

Scalable Biomechanical Modeling Through Federated Learning and
Resource Federation

by

Bowen Song

A Dissertation Presented to the
FACULTY OF THE USC GRADUATE SCHOOL
UNIVERSITY OF SOUTHERN CALIFORNIA

In Partial Fulfillment of the
Requirements for the Degree
DOCTOR OF PHILOSOPHY
(COMPUTER SCIENCE)

June 2026

Copyright 2026 Bowen Song

Acknowledgements

I want to begin by thanking my parents and my family, the cornerstone of my life. Their steady support carried me through this PhD, especially when life became difficult financially, emotionally, and personally. My mother, Hanzhen Fan, came to visit me many times and was there for me through some of the hardest periods of my life. My father, Jianzhi Song, continued working even after retirement age so that our family could stay afloat. This dissertation exists because they kept believing in me, supporting me, and making sacrifices long before any of this work became visible.

I am especially grateful to my advisor, Professor Leana Golubchik. This dissertation took a long time, and each paper required patience, persistence, and careful collaboration across disciplines. My work is not traditional in a narrow sense: it spans distributed systems, machine learning, privacy, cloud federation, and biomechanics. Leana supported that path with tireless patience, intellectual honesty, and steady guidance. She gave me the room to explore, the structure to finish, and the confidence to keep going when the work was difficult or uncertain. More than anyone else, she made this dissertation possible.

I also thank Dr. Marco Paolieri for his continuous guidance and for always helping push the work to become stronger. Marco was there through many stages of this dissertation, and his technical judgment, persistence, and generosity shaped both the cloud federation work and the biomechanical modeling work. I am grateful to Dr. Harper E. Stewart, whose expertise in biomechanics was essential to the success of the SER work. Harper’s insight into movement science helped turn an idea from computer science into a meaningful biomechanical study. I also thank Professor Jill L. McNitt-Gray for her leadership and domain expertise, and for making the biomechanics collaboration possible.

I thank Professor Vishal Misra at Columbia University and Professor Devavrat Shah at MIT for their influence on the SVD-based and counterfactual directions of this work. Their ideas helped set me on a path toward provenance, interpretability, and lightweight learning.

I am also grateful to Professor Carlee Joe-Wong for her insights on federated learning and for inspiration on expert combination, which influenced the FedSER work.

I thank my labmates Zhuojin Li and Ali Abbasi for the ideas, conversations, and support throughout the PhD. I am especially grateful to Zhuojin, who started at the same time as I did and shared all those years when it often felt like just the two of us were in the lab. Building the lab’s GPU cluster together by hand remains one of my favorite memories from graduate school. I also thank Wumo Yan and Sara Babakniya for being part of the lab community and for the conversations and support along the way.

I am grateful to the students I had the privilege to mentor. I thank Eli Friedman for building the queueing simulator and for spending two summers in our lab. Watching him grow into a full-fledged researcher at the University of Utah has been a privilege, and I am proud of him. I thank Amy (Qinming) Zhang for her work on the biomechanics dataset. I was fortunate to co-mentor Amy with Harper, and I learned a great deal from Harper’s mentoring style while watching Amy grow toward her next chapter at MIT Sloan.

I thank the lab alumni Dr. Sung-Han Lin, Dr. Leslie Cheung, and Dr. Yuan Yao. Even after graduating, they continued to support my career growth, offer advice, and help me whenever opportunities or changes came along. Seeing their careers grow into such successful paths has made the future feel brighter.

I thank Dr. Bill Cheng for giving me the opportunity to TA CSCI 353, his networking course. I learned a great deal about teaching, course operations, and the design of grading systems. Automating parts of that system by hand taught me how much care real teaching infrastructure requires. I also thank Professor Ramesh Govindan for an unforgettable networking class, which shaped how I think about systems.

Finally, I thank my closest friends. I thank Anish Gupta for years of friendship and support. Anish flew with me from the East Coast to Los Angeles when I first moved here, helped me settle down and find housing, and continued to support me from afar through all these years. I thank Dr. Hesam Mohammadi for being the best roommate, for sharing two of

the best years of my PhD life, and for being the kind of friend who opened my world to many good people. Through Hesam, I met friends including Dr. Zalan Fabian and Professor Mahdi Soltanolkotabi, and those connections became an important part of my life at USC. Hesam also helped turn me toward weight training, which became important far beyond fitness. I thank Evan Wilde and Dr. Matt Griffiths for being wonderful friends since undergraduate days. Even though we have lived far apart and across the world, they have made distance feel smaller and home feel closer. I also congratulate Matt and his family on the birth of his baby boy. I thank Sam (Sang) Yang and Zhoujian (Edmond) Zhao, my childhood friends, for the long friendship that has stayed with me across distance, time, and every stage of life.

I thank Professor Mahdi Soltanolkotabi for continuing to work out with me during my final year, when the stress was at its highest. Those mandatory daily workouts helped me through one of the hardest times of this PhD, and his constant drive to improve inspired me in research and in life. I thank Dr. Alex Bisberg for friendship and support. I also thank Dr. Sina Malakouti and Raaghav Ravishankar, friends I made at eBay. I am grateful to eBay for giving me the opportunity to work on recommendation systems as an intern and to gain real infrastructure experience over two unforgettable summers.

To all of you: I love you, and I am deeply grateful.

Table of Contents

Acknowledgements	ii
List of Tables	viii
List of Figures	xi
Abstract	xv
Chapter 1: Introduction	1
Chapter 2: Related Work	4
2.1 Wearable Biomechanical Signal Estimation	4
2.1.1 Ground Reaction Forces and Joint Moments	4
2.1.2 Laboratory Measurement and Its Limitations	5
2.1.3 Wearable Inertial Sensing and the Estimation Problem	6
2.1.4 Estimation Approaches	6
2.1.5 Low-Rank Signal Modeling	7
2.1.6 Comparison with Prior GRF Estimation Studies	8
2.2 Federated Multi-Task Biomechanical Estimation	11
2.2.1 Multi-Task Learning	11
2.2.2 Federated Learning	11
2.2.3 Distributed Computation of the SVD	12
2.2.4 Privacy in Federated Learning	13
2.2.5 Federated Multi-Task Biomechanical Estimation	13
2.2.6 Privacy-Preserving Collaboration	14
2.3 Hybrid Cloud Federations and QoS	14
2.3.1 Cloud Computing and Private Clouds	15
2.3.2 Hybrid Clouds and Cloud Federations	15
2.3.3 Quality of Service and Service Level Agreements	16
2.3.4 Queueing Models and Continuous-Time Markov Chains	16
2.3.5 Cooperative Game Theory and the Shapley Value	17
2.3.6 Comparison with Prior Cloud Federation Studies	17
Chapter 3: Estimating Ground Reaction Forces From Inertial Sensors . . .	19
3.1 Introduction	19
3.2 Notation and Dataset	22

3.3	Estimation Tasks and Metrics	25
3.3.1	Estimation Tasks and Hyperparameter Selection	25
3.3.2	Error Metrics for Estimated GRF Waveforms	26
3.3.3	Error Metrics for Estimated Biomechanical Variables	27
3.4	Methods	28
3.4.1	SVD Embedding Regression (SER)	28
3.4.2	k -Nearest Neighbors Regression	30
3.4.3	Long Short-term Memory Networks	36
3.5	Results and Discussion	41
3.5.1	Estimation of the Normal GRF Waveform	41
3.5.2	Artifacts in Estimated Waveforms	50
3.5.3	Estimation of Discrete Biomechanical Variables	51
3.5.4	Provenance	52
3.6	Conclusions	52
Chapter 4: FedSER for Wearable Biomechanics		54
4.1	Introduction	54
4.2	Dataset and Notation	58
4.3	Problem Setup and Evaluation	60
4.4	Proposed Approach	62
4.4.1	Background on SER	62
4.4.2	Federated SER	62
4.4.3	Federated Mixture-of-Experts Framework	67
4.4.4	Privacy Mechanism	69
4.4.5	Differentially Private Dataset Release	72
4.5	Results and Discussion	75
4.5.1	Comparison with Non-Federated Baselines	75
4.5.2	Robustness to Smaller Training Set Sizes	77
4.5.3	Expert Combination Strategies	78
4.5.4	Computational Demand	79
4.5.5	Privacy Perspective	82
4.5.6	Missing Input Channels and Projection-Based Recovery	87
4.6	Conclusions	89
Chapter 5: Hybrid Cloud Federations With QoS Requirements		91
5.1	Introduction	91
5.2	Performance Model	96
5.2.1	System Description and Notation	96
5.2.2	CTMC Model	99
5.2.3	Optimal Sharing Strategy with Homogeneous QoS	101
5.2.4	Optimal Sharing Strategy with Heterogeneous QoS	104
5.3	Revenue Model	107
5.3.1	Sharing Public Cloud Savings using Shapley Value	107
5.3.2	Overall Reduction in Operating Costs	110
5.3.3	Alternative Reward Policies	111

5.4	Experimental Evaluation	112
5.4.1	Public Cloud Savings and Operating Cost Reduction	114
5.4.2	Reward Policies	122
5.5	Conclusions	123
Chapter 6: Conclusion		127
Bibliography		130
Appendix A: SER Supporting Material		144
A.1	Aligning Signals from Different Locations by Reference Events	144
A.2	Discrete Biomechanical Variables	150
A.3	RMSE and Relative RMSE of Anterior/Posterior GRF	151
Appendix B: FedSER Algorithm Pseudocode		152
B.1	Federated SVD Algorithm	153
Appendix C: FedSER Detailed Tables		155
C.1	Baseline and Modality Results	155
C.2	Data-Scarcity Results	157
Appendix D: FedSER Confidence Intervals and Supplementary Figures . .		163
D.1	Confidence Interval Methodology	163
D.2	Per-Action RMSE under LOAO	164
D.3	Action-Within-Training RMSE Improvement	166
D.4	RMSE Improvement under Differential Privacy	166

List of Tables

2.1	Comparison with representative prior studies on GRF estimation. Results are reported for each of our methods (SER, KNN, LSTM) when using ALL input signals in their best scenarios of application (PERSONAL, EVERYONE, OTHERS, respectively). Protocols differ across papers, so the table is not a direct ranking; where possible we note the closest available train/test protocol (often closest to OTHERS).	9
3.1	Combinations of input signals for the estimation tasks	23
3.2	LSTM Hyperparameter Search Space	36
3.3	RMSE (in body weight units, BW, i.e., N/kg) of vertical GRF estimations $RMSE(g_z, \hat{g}_z)$ for different input signals, data scenarios, machine learning methods (results highlighted in blue are optimal or less than 0.010 from optimal for a scenario and set of input signals)	38
3.4	rRMSE (%) of vertical GRF estimations $rRMSE(g_z, \hat{g}_z)$ for different input signals, data scenarios, machine learning methods (results highlighted in blue are optimal or less than 0.5% from optimal for a scenario and set of input signals)	39
3.5	Average inference time (in seconds) to estimate 3D GRFs of a collection (120-180 steps) for an athlete. Inference time is measured on an Intel i7-6800K CPU; for LSTM we also report average inference time using a TITAN X GPU.	40
3.6	Scenarios for the estimation tasks	41
3.7	MAPE (%) of discrete biomechanical variables estimated from ALL input signals for different scenarios and machine learning methods (results highlighted in blue are optimal or less than 0.5% from optimal for a scenario and biomechanical variable)	49
4.1	Input/Output Signals (all with x, y, z axes)	58

4.2	RMSE for hip and knee flexion moments and vertical GRF under the LOAO protocol with full training data. Hip and knee moments are in N·m/kg and vertical GRF is in body-weight units (BW). The lowest value per setting and target is bolded.	74
4.3	Per-sample mobile inference time on a Pixel 8 Pro with optimizations and quantization, averaged over 100 runs after ten warm-up inferences.	82
4.4	Average hip-moment RMSE reduction from projection-based recovery relative to zero-filled FedSER inputs. Positive values indicate lower RMSE after recovery.	88
5.1	Forwarding rate to public clouds before (η_0) and after (η_F) joining the federation and relative cost reduction (δ_1, δ_2) for different arrival rates when $\vec{n} = \vec{m} = (50, 50)$, $\vec{Q} = (0, 0)$	115
5.2	Relative cost reduction with heterogeneous QoS.	121
A.1	RMSE (in body weight units, BW, i.e., N/kg) of anterior/posterior GRF estimations $RMSE(g_y, \hat{g}_y)$ for different input signals, data scenarios, machine learning methods	148
A.2	rRMSE (%) of anterior/posterior GRF estimations $rRMSE(g_y, \hat{g}_y)$ for different input signals, data scenarios, machine learning methods	149
C.1	RMSE of joint moments at hip and knee (N·m/kg) and vertical GRF (BW) across input sensor modalities, data scenarios, and machine learning methods. Results are reported for tasks included in the training set and for tasks excluded during training under the Leave-One-Action-Out (LOAO) protocol. Best (lowest) RMSE per <i>modality</i> , <i>data case</i> , <i>target</i> is highlighted; values within 0.01 of the minimum are treated as ties and highlighted as well.	156
C.2	Action-Within-Training evaluation. RMSE for IMU-only models when the test action is included in training, across data scarcity levels. Hip and knee moments are in N·m/kg and vertical GRF is in BW. FedSER variants are labeled by gating strategies. SER and FedSER (Global) have the same results and are consolidated into one. Best (lowest) RMSE per <i>case</i> , <i>scarcity level</i> , <i>target</i> is highlighted; values within 0.01 of the minimum are treated as ties and highlighted as well.	158

C.3	Leave-One-Action-Out (LOAO) evaluation. RMSE for IMU-only models when the test action is withheld from training for all participants, across data scarcity levels. Hip and knee moments are in N·m/kg and vertical GRF is in BW. FedSER variants are labeled by gating strategies. SER and FedSER (Global) have the same results and are consolidated into one. Best (lowest) RMSE per <i>case</i> , <i>scarcity level</i> , <i>target</i> is highlighted; values within 0.01 of the minimum are treated as ties and highlighted as well.	159
-----	--	-----

List of Figures

3.1	Example running signals from treadmills and wearable IMUs	32
3.2	Organization of input (IMU) and output (GRF) matrices for the SER method. Each row includes data for multiple left/right steps and multiple signals; data of an athlete can span multiple rows.	33
3.3	RMSE of vertical GRF estimated with SER and KNN for different numbers of steps S in a batch	34
3.4	Estimating GRFs with <i>SVD Embedding Regression</i>	35
3.5	RMSE of vertical GRF for different numbers of neighbors k using KNN regression	37
3.6	LSTM model architecture. The inputs are the mean, standard deviation, and range calculated from IMU signals (ALL as shown in the figure) for a time window, in addition to the weight and running speed of the athlete; a bidirectional LSTM layer is followed by dropout regularization and three fully connected layers.	37
3.7	Comparing estimations (blue) and measurements (orange) of bodyweight-normalized vertical GRF $g_z(t)$ for runners with different foot-strike patterns, using all input signals (ALL). In each subgraph, the columns are the scenarios (OTHERS, EVERYONE, PERSONAL) and the rows are the machine learning methods (SER, KNN, LSTM).	45
3.8	Effect of input signal subsets for Athlete 4 (Major Impact Peak): estimations (blue) and measurements (orange) of bodyweight-normalized vertical GRF $g_z(t)$ using all signals (ALL), sacrum acceleration only (SAC/ACC), and three-dimensional acceleration (SAC/ACC3D). Columns are the scenarios (OTHERS, EVERYONE, PERSONAL) and rows are the methods (SER, KNN, LSTM).	46

4.1	Hip flexion moment estimation error (RMSE) using IMU-only inputs under different training set sizes, with models trained on all actions except the target action	57
4.2	Experimental setup and data: (a) placement of wearable IMU sensors measuring input signals; (b) estimation outputs, hip and knee joint moments (JM, top and middle row) and ground reaction force (GRF, bottom row), during different actions (jump, turn and step, and cutting). Lines show the average measured signal, while shaded bands indicate $\pm$ the standard deviation at each time point.	59
4.3	Training examples of each action in the dataset	60
4.4	Overview of MoE framework	67
4.5	RMSE across training set sizes under IMU-only inputs, LOAO, and FEDERATED setting, where the training set size is varied by retaining varying percentages of each client’s training samples.	77
4.6	RMSE improvement of FedSER (MoE+Global) over KNN and TCN across training set sizes under IMU-only inputs, LOAO, and the FEDERATED setting. Positive values indicate lower RMSE for FedSER; error bars show 95% confidence intervals. We retain confidence intervals only in this figure as a representative example; the CI methodology and additional analyses appear in Appendix D.1.	78
4.7	RMSE across training set sizes for FedSER expert combination strategies and KNN, using only IMU inputs, in the FEDERATED setting. FedSER variants include the global expert, best expert, MoE, and MoE+Global.	79
4.8	Average training time versus the percentage of training data retained for SER, FedSER, and TCN. The vertical axis is logarithmic. SER and FedSER are measured on CPUs; TCN is a centralized A100-GPU measurement and therefore provides an optimistic lower bound for federated TCN training.	80
4.9	Hip flexion moment RMSE under DP in the IMU-only, FEDERATED, LOAO setting with full training data. RMSE is plotted against the privacy budget ϵ for a fixed δ under the Gaussian mechanism of Section 4.4.4, with ϵ split between feature and target releases; larger ϵ gives a weaker privacy guarantee (less injected noise).	84

4.10	Per-action hip-moment RMSE for TCN (squares) and FedSER MoE+Global (triangles) under IMU-only, FEDERATED, LOAO evaluation. Values are normalized by the non-private training-output mean for each action, shown by the dashed line at 1. The two panels use privacy budgets $\epsilon \approx 10$ and $\epsilon \approx 1000$ (fixed δ ; ϵ split between feature and target releases as in Section 4.4.4). . . .	86
4.11	Missing input channels and projection-based recovery before FedSER estimation.	88
4.12	Hip-moment RMSE versus number of randomly dropped IMU channels, with and without projection-based recovery, for action-within-training (left) and LOAO (right) evaluation.	89
4.13	Input-recovery and downstream output RMSE versus the number of projection iterations.	90
5.1	Federation of Hybrid Clouds	93
5.2	Forwarding rates to public clouds for different federations	95
5.3	Queueing Model of a Hybrid Cloud Federation	97
5.4	Transition rates for the CTMC model	100
5.5	Queueing model after combining the private clouds	103
5.6	Transition rates of an $M/M/(s + s_F)/(c + c_F)$ queue	103
5.7	Transition rates of an $M/M/s/c$ queue forwarding overflow to a $G/M/s_F/c_F$ queue (separate queues)	106
5.8	Sharing Strategies with Heterogeneous QoS	108
5.9	Rate of requests sent to public clouds by a federation of two clouds with $n_1 = n_2 = m_1 = m_2 = 50$ servers and $Q_1 = Q_2 = 0$, for different arrival rates λ_1 and λ_2	113
5.10	Reduction of requests sent to public clouds by two federated clouds with $n_1 = n_2 = 50$ servers, $Q_1 = Q_2 = 0$, $\lambda_1 = 10$, $\lambda_2 = 150$ for different sharing strategies m_1 and m_2	116
5.11	Reduction of requests sent to public clouds by two federated clouds with $n_1 = n_2 = 50$ servers, $Q_1 = Q_2 = 0$, $\lambda_1 = \lambda_2 = 150$ for different sharing strategies m_1 and m_2	117
5.12	Forwarding to public cloud before (dashed lines) and after (solid lines) when clouds with different QoS share all resources.	118

5.13	Comparison of different reward policies.	125
5.14	Reward distributed for different federations.	126
A.1	Examples of overlapping the magnitude of GRFs with the magnitude of acceleration signals from shanks and sacrum. Blue is from left shank, green is from right shank, orange is from sacrum, and grey is from GRFs. (a) Aligned jump reference.	146
A.2	Examples of overlapping the magnitude of GRFs with the magnitude of acceleration signals from shanks and sacrum. Blue is from left shank, green is from right shank, orange is from sacrum, and grey is from GRFs. (b) Consecutive left steps under a constant speed.	147
C.1	RMSE for multi-task estimation of hip flexion moment from IMU-only inputs, versus the percentage of data used in training. Top: action-within-training ; bottom: LOAO . Each row compares SER, KNN, TCN, and FedSER with different gating strategies across Personal , Others , and Everyone data cases.	160
C.2	RMSE for multi-task estimation of knee moment from IMU-only inputs, versus the percentage of data used in training. Top: action-within-training ; bottom: LOAO . Panel layout matches Fig. C.1.	161
C.3	RMSE for multi-task estimation of vertical GRF from IMU-only inputs, versus the percentage of data used in training. Top: action-within-training ; bottom: LOAO . Panel layout matches Fig. C.1.	162
D.1	Per-action RMSE under IMU-only inputs, LOAO, and the FEDERATED setting, for hip moment, knee moment, and vertical GRF at 10% and 100% training data. Each bar is the per-action mean RMSE (averaging over subjects, experimental runs, and speed/intensity variants within an action); error bars show 95% confidence intervals (Section D.1).	165
D.2	RMSE improvement of FedSER (MoE+Global) over KNN and TCN across training-set sizes under action-within-training evaluation with IMU-only inputs and federated training. Positive values indicate lower RMSE for FedSER; error bars show 95% action-bootstrap confidence intervals (Section D.1). . .	166
D.3	RMSE improvement of FedSER (MoE+Global) over KNN and TCN versus the privacy budget ϵ for hip moment (IMU-only, FEDERATED , LOAO , full training data). Positive values indicate lower RMSE for FedSER; error bars show 95% confidence intervals (Section D.1).	167

Abstract

Ground reaction forces and joint moments are central to gait analysis, rehabilitation, injury prevention, and sports performance: the former measure external load at the ground, and the latter measure internal mechanical demand (net torque) at the joints. Clinicians, athletes, coaches, and researchers rely on them, but direct measurement usually requires expensive lab equipment. Wearable sensors could bring this to far more people, including everyday users who track workouts through health and fitness apps, but they make the machine learning harder: the model must estimate these signals from indirect measurements, work with limited data, protect private data, and run efficiently on small edge devices. Conventional centralized deep learning often needs large pooled datasets, expensive training, extensive hyperparameter tuning, and cloud-scale resources, which is a poor fit for these constraints.

This dissertation develops lightweight, federated, and low-rank methods for these settings, evaluated on real biomechanics data and edge hardware. We first introduce SVD Embedding Regression (SER), a lightweight method that estimates ground reaction forces (GRFs) and biomechanical variables derived from them, such as contact time and impulse, from inertial measurement unit (IMU) signals by learning a linear map between low-rank SVD embeddings of the input and output signals. On running data, SER and k -nearest neighbors (KNN) match or exceed Long Short-Term Memory (LSTM) network accuracy at a fraction of the cost: SER estimates 10 running steps in 0.3 ms on a smartphone, more than four orders of magnitude faster than an LSTM (about 15 s), and personal training data reduces error further, especially for the derived variables.

We then extend SER to Federated SER (FedSER), a federated multi-task method for estimating GRFs and hip and knee joint moments across participants and actions. FedSER learns shared SVD bases and embedding regressors collaboratively, without centralizing raw data, and combines action-specific experts with a global expert through a federated task classifier. On a public biomechanics dataset, FedSER matches centralized SER and outper-

forms Temporal Convolutional Networks (TCNs) on hip and knee joint moments and vertical ground reaction forces, including under data scarcity and leave-one-action-out evaluation. It is also far more deployable: in an 11-client experiment with one physical Google Pixel 8 Pro and ten simulated clients, the phone completed FedSER training in about 3 minutes, versus 40.8 minutes for a centralized TCN on an A100 GPU. FedSER runs inference in 0.16 ms per sample on the Pixel hardware, versus 71 ms for the TCN. Secure aggregation hides individual updates without changing accuracy; FedSER is also compatible with partially homomorphic encryption in principle (not separately timed here), and it retains accuracy comparable to that of non-private TCN under differential privacy. When individual IMU channels are missing at test time, projection onto the learned input subspace reduces error if the test action was seen in training; we show this for hip flexion moment as an example.

We also study a related systems problem. Clinics, research labs, and similar places often keep their data local, yet still need bursty compute for work such as training or maintaining machine learning models, and they may want to share computing resources when demand is high instead of buying enough machines for the peak or sending every overflow job to a public cloud. We model a federation of hybrid clouds as a continuous-time Markov chain and ask when members benefit from sharing under quality-of-service (QoS) requirements. We prove that sharing all resources is best when members have the same QoS requirements, show that this is not always true when requirements differ, and use Shapley value to divide the public-cloud savings in a way that rewards both workload and resource contributions and discourages free riding. This cloud study is separate from the biomechanics experiments.

Chapter 1

Introduction

Ground reaction forces (GRFs) and joint moments describe the external loading and internal mechanical demand of human movement. Clinicians, coaches, and researchers use them for gait analysis, rehabilitation, injury-risk assessment, sports performance, and assistive-device control. In the laboratory they are measured with force plates, instrumented treadmills, motion capture, and inverse dynamics. Those systems are accurate, but they are expensive and hard to use outside controlled settings.

Wearable inertial measurement units (IMUs) measure acceleration and angular velocity from body-mounted sensors and can be worn during training, rehabilitation, and everyday movement. They do not measure GRFs or joint moments directly, so those targets must be estimated from noisy, indirect signals. The learning problem is demanding: labels are limited, subjects and activities vary, and useful systems often need fast inference on phones and other edge devices. At the same time, raw wearable and clinical signals are sensitive, so many clinics, research labs, and sports performance centers cannot pool them on a central server.

This dissertation studies lightweight and federated methods for that estimation problem, and a related systems question about sharing computers rather than data. The biomechanics chapters keep data local when it cannot be pooled. The cloud chapter asks when independent organizations benefit from sharing machines under quality-of-service (QoS) constraints. The two parts share a practical audience, but they are separate studies: the cloud analysis is not a FedSER experiment on cloud queues.

Chapter 2 introduces the background for wearable biomechanical estimation, federated multi-task learning and privacy-preserving collaboration, and hybrid cloud federations, and surveys the prior work that frames the technical chapters that follow.

Contributions. This dissertation makes three contributions.

1. *Lightweight biomechanical signal estimation.* Chapter 3 develops SVD Embedding Regression (SER), a low-rank method that estimates GRFs, and biomechanical variables derived from them such as contact time and impulse, from IMU signals by learning a linear map between compact SVD embeddings of the input and output waveforms. On running data, SER and k -nearest neighbors (KNN) are as accurate as or more accurate than Long Short-Term Memory (LSTM) baselines while estimating 10 running steps in 0.3 ms on a smartphone, more than four orders of magnitude faster than an LSTM, and with far lower training cost and energy.
2. *Federated multi-task biomechanical learning.* Chapter 4 develops Federated SER (FedSER), a federated multi-task extension of SER for estimating GRFs and joint moments across participants and actions. FedSER learns shared low-rank bases and embedding regressors without centralizing raw data, and combines action-specific experts with a global expert. It matches centralized SER and outperforms Temporal Convolutional Network (TCN) baselines on hip and knee joint moments and vertical ground reaction forces, including under data scarcity and leave-one-action-out evaluation. In an 11-client experiment with one physical Google Pixel 8 Pro and ten simulated clients, the phone completes FedSER training in about 3 minutes, versus 40.8 minutes for a centralized TCN on an A100 GPU, and runs inference in 0.16 ms per sample versus 71 ms. FedSER supports secure aggregation (and is compatible with partially homomorphic encryption in principle), and retains accuracy comparable to that of non-private TCN under differential privacy. The same chapter also examines projection-based recovery when individual IMU channels are missing at test time, using hip flexion moment as an example, along with per-action privacy results and training time across data-scarcity levels.
3. *Hybrid cloud federations under QoS requirements.* Chapter 5 studies when independent

organizations benefit from sharing resources in a federation of hybrid clouds under QoS constraints. Using a continuous-time Markov chain model, we prove that sharing all resources is best when members have the same QoS requirements, show that this can hurt when requirements differ, and use Shapley value to divide savings in a way that discourages free riding. Clinics, labs, and similar places that keep data local are a natural example of who might want this kind of sharing, but the chapter is a systems study and does not use the biomechanics workloads.

Chapter 2

Related Work

This chapter covers the background for the technical chapters and the prior work closest to them. The organizing thread runs from wearable sensing for biomechanical estimation, through federated learning when data cannot be pooled, to compute sharing when organizations keep resources local. Section 2.1 covers ground reaction forces and joint moments, how they are measured and estimated from wearable sensors, and related wearable estimation studies. Section 2.2 covers multi-task learning, federated learning, distributed SVD, and privacy, including federated and multi-task biomechanical learning. Section 2.3 covers cloud computing, hybrid clouds, cloud federations, quality-of-service requirements, queueing models, and Shapley value for sharing benefits.

2.1 Wearable Biomechanical Signal Estimation

This section covers the biomechanical signals used later in the dissertation, how they are measured, and how they are estimated from wearable sensors. We start with ground reaction forces and joint moments, then laboratory measurement and its limits, wearable inertial sensing, physics-based and data-driven estimation, and low-rank signal modeling. A quantitative comparison with prior GRF estimation studies is included below; Chapter 3 builds on this background.

2.1.1 Ground Reaction Forces and Joint Moments

The ground reaction force (GRF) is the force exerted by the ground on the body during contact. It characterizes the external mechanical loading applied to the body during movement, and it is commonly reported as three components: mediolateral, anterior-posterior,

and vertical [1]. Repeated loading contributes to the stress response of bone and soft tissue [2], and GRF analysis has been used to study the factors that lead to bone stress injuries in runners [3, 4, 5, 6, 7, 8]. The GRF waveform is also useful beyond its peak values: domain experts derive biomechanical variables from it, such as contact time, impulse, and the change in center of mass velocity, which provide context for how forces relate to movement and to injury risk [9, 10, 11].

Joint moments describe the net torque around a joint during movement, and thus measure the internal mechanical demand placed on the body rather than the external load captured by the GRF [1, 12]. Joint moments are central to clinical gait assessment, rehabilitation monitoring, and the control of assistive systems such as exoskeletons and prosthetics, where the controller adjusts assistance based on the estimated demand at the joint [13, 14, 15]. GRFs and joint moments are therefore complementary: the first describes the loading applied at ground contact, and the second describes how that loading is distributed across the joints. Both are used in this dissertation as estimation targets.

2.1.2 Laboratory Measurement and Its Limitations

GRFs and joint moments are traditionally obtained in a laboratory. GRFs are measured directly using force plates or instrumented treadmills, which contain load cells that record the three force components during contact [16, 17, 18]. Joint moments are not measured directly; they are computed through inverse dynamics, which combines the measured GRF with body-segment motion recorded by motion capture and with estimates of segment masses and inertias to solve for the net torque at each joint [1, 19]. More detailed estimates can be obtained from musculoskeletal models that represent muscles and joints explicitly, but these models require per-subject calibration and additional measurements, and they are computationally demanding [19, 20].

These methods are treated as the gold standard because they are accurate and well validated [1]. However, they share the same practical limitations: the equipment is expensive,

it is confined to the laboratory, and it constrains the participant to a small set of controlled movements. As a result, these methods are difficult to scale to large populations or to monitoring during everyday activity outside the laboratory. Wearable GRF sensors have been studied as a direct alternative, but reliable devices are not widely available, and their validity and reliability depend on the specific system, sensor location, and movement speed [21, 22]. This motivates estimating GRFs and joint moments from sensors that are cheaper and easier to deploy.

2.1.3 Wearable Inertial Sensing and the Estimation Problem

Inertial measurement units (IMUs) provide a practical sensing alternative. An IMU is a small, low-cost, body-mounted device that measures linear acceleration and angular velocity, and IMUs are now widely available in consumer wearables [23]. Because they can be worn continuously, IMUs can collect data during training, rehabilitation, and daily living, rather than only during controlled laboratory sessions.

The difficulty is that IMUs do not measure GRFs or joint moments directly: they capture localized acceleration and angular velocity at the sensor, not the forces at the ground or the torques at the joints. Recovering the biomechanical signals of interest from these indirect measurements is therefore an estimation problem, and it is a central challenge in biomechanics and remote monitoring [23]. The problem is made harder by several factors that recur throughout this dissertation: the inputs are noisy and only indirectly related to the target signals, labeled ground truth is expensive to collect and therefore limited, participants differ in movement patterns and sensor placement, multiple movement types must be supported, and the resulting models are often intended to run on devices with limited computing power.

2.1.4 Estimation Approaches

Two broad families of methods estimate biomechanical signals from wearable data. Physics-based methods use forward dynamics, optimal control, or musculoskeletal models to re-

construct the target signals from the measured motion; they can be accurate, but they typically require per-subject calibration, additional sensors, and iterative simulation with hand-tuned cost functions for each movement, which makes them compute-intensive [24, 19, 20]. Data-driven methods instead learn the mapping from sensor signals to biomechanical targets directly from data, avoiding an explicit physical model [25, 23, 26].

Within the data-driven family, most recent approaches use deep learning. Convolutional neural networks (CNNs), long short-term memory networks (LSTMs), and temporal convolutional networks (TCNs) have all been applied to estimate GRFs and joint moments from wearable and laboratory inputs [27, 28, 29, 13, 15, 30]. These models can produce accurate estimates, in some cases by combining IMUs with additional inputs such as motion capture [15]. Their accuracy, however, comes at a cost that matters for the deployment settings studied here: they require long training times and extensive hyperparameter searches, their compute, memory, and energy demands grow with the size of the data and the model, and they behave as black boxes that provide limited insight into the relationship between the inputs and the estimated signals [31, 32, 33, 34]. These costs are particularly restrictive when training or inference must run on edge devices, which motivates the lightweight methods developed in this dissertation.

2.1.5 Low-Rank Signal Modeling

The estimation method developed in Chapter 3 is built on the observation that biomechanical waveforms are highly structured and can be represented compactly in a low-dimensional space. The singular value decomposition (SVD) factorizes a data matrix into orthogonal directions ordered by how much of the variation they explain, and keeping only the leading directions gives a low-rank approximation that retains most of the signal while discarding noise [35]. Principal component analysis (PCA) applies the same idea to variance-based dimensionality reduction [36], and principal component regression regresses a target on these leading directions rather than on the raw signal [37]. Low-rank approximations of this kind

are effective for time-series signals because a small number of directions captures the shape of the waveform over time [38, 39]. Chapter 3 uses this structure to learn a linear map between low-rank SVD embeddings of the input and output waveforms, an idea related to transduction between structured input and output spaces [40].

2.1.6 Comparison with Prior GRF Estimation Studies

We next position the method of Chapter 3 relative to prior work on GRF estimation. In Table 2.1, we report a summary of representative prior studies on GRF estimation, their settings, and estimation errors.

Paper	GRF Measurement	Sensor Type	Sensor Locations	Estimation Model	RMSE (BW)			rRMSE (%)		
					x	y	z	x	y	z
[28]	Instrumented treadmill, multiple speeds and slopes	IMU (acceleration)	Sacrum	LSTM	—	—	0.16	—	—	6.4
[41]	In-sole sensors, multiple speeds	IMU (acceleration)	Left/right shanks	MLP	—	—	0.15	—	—	—
[27]	Force plate, multiple speeds	IMU (acceleration, angular velocity)	Sacrum, right thigh, shank, foot	Simulation + CNN	—	—	—	—	2	6
[42]	Instrumented force plates and treadmills	Video, IMU (acceleration)	Sacrum, left/right shanks, thighs	CNN	—	—	—	21.6	17.1	13.9
[43]	Instrumented treadmill, multiple speeds	Video, IMU (acceleration)	Sacrum, left/right shanks	MLP	—	—	0.27	—	—	—
[44]	Instrumented treadmill, rearfoot strike runners	IMU (acceleration)	Feet, (proximal) tibias, thighs, pelvis, and trunk	Physical model	0.05	0.07	0.18	10.8	7.8	6.6
Ours	Instrumented treadmill, multiple speeds	IMU (acceleration, angular velocity)	Sacrum, left/right shanks	SER (PERSONAL)	—	0.05	0.13	—	5.0	4.2
				KNN (EVERYONE)	—	0.05	0.12	—	4.6	3.9
				LSTM (OTHERS)	—	0.06	0.13	—	7.1	4.2

Table 2.1: Comparison with representative prior studies on GRF estimation. Results are reported for each of our methods (SER, KNN, LSTM) when using ALL input signals in their best scenarios of application (PERSONAL, EVERYONE, OTHERS, respectively). Protocols differ across papers, so the table is not a direct ranking; where possible we note the closest available train/test protocol (often closest to OTHERS).

Like the SER method in Chapter 3, the approach of [45] uses the general idea of transduction [40] between the embeddings of input IMU data and output GRF data. We observe the following critical differences between SER and [45]:

- SER uses a different organization of the training data, where the running steps and their IMU and GRF signals are split into *batches*, as shown in Fig. 3.3. Batch size is a critical hyperparameter to consider intra-step interactions (optimized with a validation set), while using a single step without differentiating left and right foot (as in [45]) results in higher estimation error. This suggests that the biomechanics of one leg may be different from that of the other.
- SER uses SVD instead of PCA (used in [45]), i.e., it does not normalize each input variable of IMU or GRF time series across the entire dataset. This difference allows us to preserve the patterns of the step signals over time.
- SER uses least squares regression to predict the output embedding instead of neural networks (used in [45]); in our experiments, neural networks (with up to 5 layers and 100 neurons per layer) resulted in higher estimation error and slower training, due to the limited available data and additional hyperparameter optimization. For instance, using signal input ALL, neural networks predicting output embedding achieve RMSE of 0.220 BW for OTHERS (0.023 BW higher than least squares regression), 0.189 BW for EVERYONE (0.002 BW higher), and 0.190 BW for PERSONAL scenario (0.060 BW higher).

In the OTHERS scenario on our data, SER with least-squares embedding regression also trains much faster than neural-network predictors of the same embeddings (see the ablation above); a head-to-head numeric comparison with the published Pogson et al. error rates is not reported here because the datasets and protocols differ.

Other studies consider different types of model inputs or outputs. In [28, 46], IMU signals are used to estimate only the vertical GRF, while Chapter 3 also estimates the

anterior-posterior GRF. A higher number of IMU sensors is used in [44] and [43] (8 and 17 IMUs, respectively). Notably, Chapter 3 evaluates the error reductions obtained from different combinations of input IMU signals.

2.2 Federated Multi-Task Biomechanical Estimation

The estimation methods of the previous section assume that all training data can be pooled in one place and that a single model serves every movement. Neither assumption holds when biomechanical data are collected across many participants and institutions. This section covers multi-task learning, federated learning, distributed SVD, and privacy, then the related work closest to Chapter 4.

2.2.1 Multi-Task Learning

Biomechanical monitoring rarely involves a single movement. Participants walk, climb stairs, squat, lunge, and jump, and each movement produces different signal patterns. Multi-task learning trains a single model to solve several related tasks at once, so that shared structure learned from one task improves the others and data are used more efficiently than when a separate model is trained per task [47, 48]. In this dissertation, each movement action is treated as a separate regression task, and the model is expected to estimate the biomechanical signals for every action. A common way to combine tasks is a mixture of experts, in which several specialized models (the experts) are trained and their outputs are combined, so that the model can specialize where it has data while still generalizing to actions with little or no data [49]. Chapter 4 uses this idea by combining action-specific experts with a global expert.

2.2.2 Federated Learning

Biomechanical data are distributed and sensitive. Ground truth GRFs and joint moments are expensive to collect, so individual participants usually have limited data, and some

movement actions may be missing entirely from a participant’s data [23, 13, 15, 14]. At the same time, these data are often too sensitive to be pooled in a central dataset shared across organizations, as in other medical settings [50, 51]. For example, athletes may be willing to share performance data with their own team but not with other teams or third parties.

Federated learning (FL) addresses this setting by letting participants train a shared model without sharing their training data [52]. Instead of sending raw signals to a server, each participant trains a local model on its own data and sends only model updates; the server combines these updates into a new shared model, which is sent back to the participants for the next round. FL increases the effective amount and diversity of training data while keeping raw data local, and it has been applied across mobile devices and in digital health [53, 54, 51]. It also introduces two challenges that recur in this dissertation. First, data are not identically distributed across participants, because movement patterns and collection processes differ, which motivates methods that combine shared knowledge with participant- or task-specific adaptation [13, 55, 53]. Second, mobile and edge devices have limited computation, memory, and communication, which makes lightweight methods desirable [51, 53, 56].

2.2.3 Distributed Computation of the SVD

Bringing the low-rank method of Section 2.1.5 into a federated setting requires computing the SVD without pooling the data, because the SVD in Chapter 3 is computed on the full training matrix. A federated SVD must satisfy several requirements at once: it must not share local data, it must recover approximately the same basis as a centralized SVD so that accuracy is preserved, it must converge quickly and predictably, and it must run within the limited memory and compute of edge devices. Existing distributed SVD methods do not satisfy all of these requirements, and Chapter 4 compares against them directly, as surveyed below.

2.2.4 Privacy in Federated Learning

Keeping raw data on the device is a first step toward privacy, but it is not a guarantee on its own, because a server can still inspect the individual model updates a participant sends and infer information about that participant’s data [57, 58, 59]. Two risks are relevant for biomechanical data, which can reveal a person’s health status, injury risk, or movement patterns: membership inference, where an adversary tries to determine whether a particular participant’s data was used in training, and model inversion, where an adversary tries to reconstruct sensitive input from the model or its updates [60]. Three complementary mechanisms are used to reduce these risks. Secure aggregation hides each participant’s update inside the combined sum, so the server sees only the aggregate [61]. Homomorphic encryption allows the server to aggregate values while they remain encrypted [62]; both of these add computation or communication overhead [63]. Differential privacy provides a formal guarantee by bounding how much any single protected unit can affect a released result, which requires specifying the unit being protected, how its contribution is clipped, and how the privacy budget is tracked [64, 65]. These mechanisms are the ones evaluated in Chapter 4, and they are surveyed in more detail in Section 2.2.6.

2.2.5 Federated Multi-Task Biomechanical Estimation

We next survey prior work that combines these ideas for wearable biomechanical estimation, and position the contribution of Chapter 4.

Centralized estimators for multiple movements predominantly use CNNs, LSTMs, and TCNs [27, 29, 13, 15]. These models can use wearable and laboratory sensing inputs, but their training, hyperparameter search, memory, and energy costs make on-device retraining difficult [32, 33, 34]. Several federated methods combine shared knowledge with participant- or task-specific adaptation. FedHealth combines federated aggregation with transfer learning for personalized wearable healthcare [66]. ProtoHAR adapts prototypes to individual clients, while personalized semi-supervised federated learning uses both labeled and unlabeled local

activity data [67, 68]. FATHOM uses hierarchical attention for federated multivariate sensor analytics [69]. FedMAT learns shared and participant-specific representations for cross-individual activity recognition [70], and FedOpenHAR applies federated multi-task transfer to activity-recognition and device-position tasks, including datasets with incomplete label sets [71].

These studies primarily address classification using deep networks. Chapter 4 instead studies waveform regression for GRFs and joint moments, treats movement actions as regression tasks, and emphasizes training and inference on resource-constrained devices.

Among distributed SVD methods, approaches that compute an SVD on each client and combine the decompositions at the server can differ from centralized SVD and require substantial local computation [72, 73]. Distributed power methods construct several singular vectors together, increasing client memory and requiring all vectors to continue until the slowest one converges [74, 75]. Chapter 4 uses sequential orthogonalized updates to recover the basis one vector at a time and then trains the action classifier and waveform regressors in the resulting embedding spaces.

2.2.6 Privacy-Preserving Collaboration

Section 2.2.4 introduced secure aggregation, partially homomorphic encryption, and differential privacy as complementary protections beyond keeping data local. Linear models trained on fixed features can retain more utility than end-to-end deep networks under comparable privacy budgets on image benchmarks [76], while DP-SGD provides a general training mechanism for deep networks [77]. Chapter 4 states the threat model and DP dataset-release protocol in Section 4.4.4, and reports the privacy–utility results in Section 4.5.5.

2.3 Hybrid Cloud Federations and QoS

Beyond keeping training data local, clinics, research labs, and similar places may also want to share computing resources when demand is high. This section introduces cloud computing,

public and private clouds, hybrid clouds and cloud federations, quality-of-service requirements, queueing models, and cooperative game theory for sharing benefits, as background for Chapter 5.

2.3.1 Cloud Computing and Private Clouds

Cloud computing lets organizations rent computing resources on demand instead of buying and operating their own hardware. In the Infrastructure-as-a-Service (IaaS) model, public cloud providers such as Amazon Web Services (AWS), Google Cloud, and Microsoft Azure offer hardware resources (for example CPUs, GPUs, and machine learning accelerators) that are accessed remotely through virtual machines and paid for by usage time [78]. This lets organizations scale their resources up or down to match changing workloads, which reduces the risk of provisioning too little capacity (and violating service requirements) or too much (and paying for idle resources). Large organizations also run their own private clouds and data centers, using software platforms such as OpenStack [79], Apache CloudStack [80], and OpenNebula [81].

2.3.2 Hybrid Clouds and Cloud Federations

A private cloud that owns a fixed amount of hardware must decide how much capacity to provision for peak demand. Provisioning for the peak wastes resources most of the time, while provisioning below the peak fails to meet demand during busy periods. A hybrid cloud resolves this by keeping a private pool of resources and forwarding part of its workload to a public cloud when local demand is high [82, 83]. This keeps the private pool small while still meeting demand during peaks, at the cost of paying for public cloud resources when they are used.

A cloud federation is a complementary strategy in which several private clouds agree to share part of their resources to serve each other's requests [84, 85]. When one member is temporarily overloaded, it can use resources shared by other members instead of forwarding

requests to a public cloud; when a member has idle resources, it can put them to use serving other members' requests. Members that contribute resources or workload must be rewarded, and different mechanisms have been studied for this: at one extreme, the members operate as a single business and maximize total profit, which is then split according to a cooperative game solution; at the other extreme, the members remain independent and are coordinated by a pricing mechanism for the shared services [86, 87]. Resources may be shared with the whole federation [86] or with individual members [87], and owners may or may not keep priority over their shared resources. Chapter 5 studies a federation of hybrid clouds that combines both ideas: members forward overflow traffic first to a shared federation pool and then, if necessary, to a public cloud.

2.3.3 Quality of Service and Service Level Agreements

Cloud providers commit to quality-of-service (QoS) requirements through service level agreements, which specify the performance a customer's requests must receive. In this dissertation, a QoS requirement is expressed as the maximum mean waiting time that a member's requests can tolerate before being served [88, 86]. This choice matters for federations: when members have different QoS requirements, a shared pool that is acceptable for one member may be unacceptable for another, and this heterogeneity is a central difficulty analyzed in Chapter 5.

2.3.4 Queueing Models and Continuous-Time Markov Chains

The performance of a cloud under a variable workload is naturally described by queueing theory [89]. A cloud is modeled as a set of servers with a finite waiting room: requests arrive over time, wait if all servers are busy, and are rejected (here, forwarded elsewhere) when the waiting room is full. Requests are commonly assumed to arrive according to a Poisson process and to have exponentially distributed service times, giving the standard $M/M/n/m$ queue with n servers and total capacity m , whose blocking probability has a closed form. The

QoS requirement above translates directly into a limit on the capacity of each queue. When several such queues forward their overflow traffic to a shared pool, the combined system is analyzed as a continuous-time Markov chain (CTMC), whose states track the number of requests at each cloud and at the shared pool and whose steady-state probabilities give the rate of requests forwarded to the public cloud [88, 90, 91]. Chapter 5 builds such a CTMC model and uses existing results on resource sharing in queueing networks [92] to reason about the optimal sharing strategy.

2.3.5 Cooperative Game Theory and the Shapley Value

A federation only works if its members are rewarded fairly for what they contribute, otherwise members that benefit without contributing (free riders) undermine the incentive to share. Cooperative game theory studies exactly this problem of dividing the value created by a coalition among its members. The Shapley value assigns each member a share based on its average marginal contribution across all the ways the coalition could be formed, and it is the standard tool for this kind of fair division [93]. It has been used to distribute rewards in cloud federations [87, 94]. Chapter 5 keeps each member’s public cloud cost equal to what it paid before joining and distributes the resulting savings according to the Shapley value, which discourages free riding.

2.3.6 Comparison with Prior Cloud Federation Studies

We next position the model of Chapter 5 relative to prior work on hybrid clouds and cloud federations.

Representative studies of hybrid cloud architectures include Sotomayor et al. [82], who describe their operation, and Weinman [83], who analyze their advantages in reducing cost under high workload variability; representative studies of cloud federations include [84, 85].

Different models of private cloud federations are presented in [87], where members of the federation are modeled as $M/M/1$ queues sharing either requests or capacity; “reward-

driven” or “joint-business” mechanisms of cooperation are considered to evaluate the effects on the profit of federation members. A model of federated small clouds similar to federations of hybrid clouds is presented in [86], where a CTMC model is adopted to evaluate performance metrics used in a repeated game among federation members. Notably, Chapter 5 uses a more detailed model with a simpler cooperation mechanism: private clouds are modeled as $M/M/n/m$ queues (in contrast with $M/M/1$ queues of [87]) where members do not need priority over their shared resources (in contrast with [86]); our cooperation mechanism is based on *sharing of cloud savings*, while [87, 86] assume either a joint business or a pricing model for services offered to members of the federation, or to external customers. Shapley value [93] is adopted as in [87] and in many works on coalitions.

Another notable difference is that a number of papers (for example [95]) do not consider QoS, while others (for example [96]) do not consider hybrid clouds, thus leading to different models of cooperation within the federation (e.g., borrowing resources at a price from individual members instead of optimizing the amount of shared resources).

While several works use CTMC models to evaluate the performance of hybrid or federated cloud systems [86, 88, 91, 97, 90], Chapter 5 uses theoretical results [92] to prove the advantage of hybrid cloud federations with homogeneous QoS, together with their optimal policies.

Finally, while many works [98, 99, 100, 101] present *analytical* models of performance and revenue in cloud federations to maximize profit of their members, we focus on stochastic models accounting for workload variability.

Chapter 3

Estimating Ground Reaction Forces

From Inertial Sensors

3.1 Introduction

Ground reaction force (GRF), the force exerted by the ground on a body during contact, is a key measurement used in biomechanics to study the whole body dynamics of human movement. It characterizes the mechanical loading of the body, which contributes to the stress response of bone and soft tissue [2]. Analysis of GRFs has been used as a means to identify factors that lead to bone stress injuries for runners [3, 4, 5, 6, 7, 8]. Determining the cause of running-related injuries continues to be challenging in part because of the inability to account for the mechanical loading experienced by an individual during multiple foot contacts within and across training sessions. Despite conflicting findings [102], recent studies [103] continue to explore the role of GRFs in identifying mechanisms contributing to lower extremity injuries. Understanding GRF characteristics is important for improving performance (e.g., greater net impulse translates to greater changes in body momentum) and may provide insights into mechanisms like bending moments imposed on the lower extremity during ground contact, which could help further identify combinations of factors leading to injury. To this end, domain experts find that the GRF waveform and the biomechanical variables derived from it, such as contact time, impulse, and change in center of mass velocities, provide meaningful context for understanding how GRFs cause observed body movements and contribute to stress responses [9, 10, 6, 11].

Direct measurement of GRFs is typically performed using force plates or instrumented treadmills in a laboratory environment [16, 17, 18]. Wearable GRF sensors have been studied [21] but there is a lack of reliable sensors on the market. For instance, authors in [22]

note that overall validity and reliability of these devices appears to be system, location, and speed dependent. Machine learning methods can mitigate the challenges and costs associated with direct data collection by estimating signals of interest from other signals more readily accessible and from cost-effective sources.

Our study evaluates machine learning approaches to estimate GRFs from inertial measurement unit (IMU) signals collected on regular treadmills. Given the high cost of instrumented treadmills, this approach can provide more people with valuable biomechanical data about their performance at a much lower cost, thus enabling similar studies with more frequent data collections while lowering barriers for athletes, coaches, and researchers. This data can help elucidate the relationship between GRFs and performance to advance the overall understanding of biomechanics and potential injury interventions.

The estimation of GRFs from IMU sensor data is considered in previous works [104, 27, 42, 105, 28] to overcome the difficulty of direct GRF measurement. State-of-the-art approaches use deep learning to estimate GRFs; for example, [27] uses convolutional neural networks to estimate GRFs from acceleration and angular velocity waveforms (collected using low-cost wearable IMUs), while [28] uses LSTM neural networks to estimate GRFs from acceleration waveforms (collected on regular treadmills). Drawbacks of these approaches include their often prohibitive inference times on edge devices, requirement of expensive resources to support long training times and hyperparameter optimization, and the black box nature of the resulting models, which provide limited insights for the study of the relationship between IMUs and GRFs, or to identify and remove bad training data (this is defined by provenance as explained in Section 3.5.4).

In this work, we address these limitations by exploring lightweight alternatives to deep-learning methods to facilitate training and inference on devices with limited computing power. We also analyze the improvements resulting from the use of training data collected for the target athletes at multiple body locations. Specifically, we compare LSTM neural networks used in prior GRF estimation work with two lightweight approaches: *SVD Em-*

bedding Regression (SER), which estimates GRFs through linear regression between singular value decomposition (SVD) embeddings of IMU data (input) and GRF data (output); and *k-Nearest Neighbors* (KNN) regression. We evaluate the errors of these techniques in estimating GRFs and derived biomechanical variables when using training data collected (1) from different athletes, (2) from the same athlete, or (3) both. In each scenario, we explore the use of *acceleration and angular velocity data* from sensors positioned at *different locations* (sacrum, left and right shanks, which are positioned directly above the left and right lateral malleolus). This data can be easily collected given the wide availability of wearable IMUs measuring linear acceleration, angular velocity, and magnetic fields concurrently.

To evaluate the efficacy of different machine learning methods under a variety of scenarios and input sensors, we use an existing set of deidentified data collected by domain experts working with collegiate distance runners in the NCAA Pac-12 conference. Details on the data collection and preprocessing are described in Section 3.2, while the different estimation tasks and metrics are defined in Section 3.3.

Our work provides the following contributions.

1. We introduce SER, an approach to estimate GRFs from IMU measurements (Section 3.4.1), as an alternative to KNN (Section 3.4.2) and LSTM neural networks (Section 3.4.3).
2. Through our experimental results (Section 3.5), we show that simple machine learning methods such as SER and KNN can be similarly accurate or more accurate than LSTM neural networks, requiring fewer computing resources and energy, while allowing much faster training times and hyperparameter optimization.
3. By carrying out the evaluation of all machine learning methods in all scenarios using only acceleration, only angular velocity, or both, we allow a direct comparison on the same dataset and show that angular velocity measurements (collected by IMU sensors) reduce GRF estimation error when combined with acceleration measurements.

4. We show that GRF estimation error is reduced when using sensors at both sacrum and shanks, especially with LSTM.
5. We illustrate how personal training data significantly reduces GRF estimation error of KNN and SER for all combinations of input sensors.

Notably, when personal training data is available, SER and KNN achieve rRMSE lower than 5% for vertical GRF; as a reference, 6% rRMSE is considered to be acceptable for the study of whole body dynamics in [24]. Background on wearable biomechanical estimation is in Chapter 2. Chapter 4 extends this work to federated multi-task estimation of GRFs and joint moments.

3.2 Notation and Dataset

Acronym	Input Signals	Description
ALL	$\ \vec{a}_s(t)\ , \ \vec{a}_{lr}(t)\ , \ \vec{\omega}_s(t)\ , \ \vec{\omega}_{lr}(t)\ $	L2 norm of all acceleration and angular velocity signals
ACC	$\ \vec{a}_s(t)\ , \ \vec{a}_{lr}(t)\ $	L2 norm of acceleration signals (sacrum, left/right shanks)
ANG	$\ \vec{\omega}_s(t)\ , \ \vec{\omega}_{lr}(t)\ $	L2 norm of angular velocity signals (sacrum, left/right shanks)
SACRUM	$\ \vec{a}_s(t)\ , \ \vec{\omega}_s(t)\ $	L2 norm of acceleration and angular velocity at the sacrum
SHANK	$\ \vec{a}_{lr}(t)\ , \ \vec{\omega}_{lr}(t)\ $	L2 norm of acceleration and angular velocity at left/right shanks
SAC/ACC3D	$\vec{a}_s(t)$	x, y, z components of acceleration signal at the sacrum
SAC/ACC	$\ \vec{a}_s(t)\ $	L2 norm of acceleration signal at the sacrum

Table 3.1: Combinations of input signals for the estimation tasks

We use an existing Pac-12 dataset consisting of 44 competitive collegiate distance runners (25 female and 19 male) from University of Colorado Boulder, University of Oregon, and Stanford University in accordance with the Institutional Review Board for research involving human participants.¹ The dataset includes 114 collections (1–6 per participant) where participants ran on instrumented treadmills at multiple speeds: male participants ran at 7, 6.5, and 5 min/mi (3.8, 4.1, 5.4 m/s) and female participants ran at 7 and 5.5 min/mi (3.8, 4.9 m/s); GRF data was collected by the treadmills at 1,000 Hz, while wearable IMU sensors collected acceleration and angular velocity data at the sacrum, left shank, and right shank at 500 Hz. Including all athletes, collections, and running speeds, the dataset provides 276 running intervals, each with at least 60 steps (approximately 15 seconds). Examples of our collected signals are shown in Fig. 3.1.

We synchronize data using events from various sensors to divide each running interval into individual foot contacts. To minimize noise commonly found in IMU and GRF data [28], we employ a 4th order Butterworth low-pass filter. This filter has a cutoff frequency of 20 Hz for acceleration and angular velocity signals, and 30 Hz for GRF signals. Applying the same filters allows us to make a fair quantitative comparison with related work. A detailed description of data preprocessing is provided in Appendix A.1.

The integration of the data from all athletes and their data collections at different speeds resulted in a dataset of 16,000 steps (after splitting the IMU/GRF signals using 400 ms windows aligned using cross-correlation). Each step is characterized by 15 time-series signals including the 3 components (x, y, z) of the GRFs $\vec{g}(t)$, sacrum acceleration $\vec{a}_s(t)$, sacrum angular velocity $\vec{\omega}_s(t)$, left/right shank acceleration $\vec{a}_{lr}(t)$, and left/right shank angular velocity $\vec{\omega}_{lr}(t)$. Each signal is sampled at 500 Hz with a fixed 400 ms time window (which results in 200 time points per window and a single step per window).

¹This study is approved by the University of Oregon on April 27, 2021, protocol number 05162017.019. A subset of this dataset is used in [105].

3.3 Estimation Tasks and Metrics

3.3.1 Estimation Tasks and Hyperparameter Selection

We consider different data scenarios for the estimation of GRF data: similarly to related work [28] using leave-one-subject-out for evaluation, we estimate GRFs of a target athlete using subject-independent training data collected only from other athletes (we refer to this scenario as “OTHERS”); in addition, we consider subject-dependent scenarios where training data from the target athlete is used exclusively (scenario “PERSONAL”) or in conjunction with data from other athletes (scenario “EVERYONE”). For each scenario, we consider different input signal cases to estimate GRFs as listed in Table 3.1.

Data scenarios, input signal cases, and machine learning methods define *estimation tasks*, each identified by a tuple $(scenario, sensors, method)$ with

$$\begin{aligned} scenario &\in \{\text{OTHERS}, \text{PERSONAL}, \text{EVERYONE}\}, \\ sensors &\in \{\text{ALL}, \text{ACC}, \text{ANG}, \text{SACRUM}, \text{SHANK}, \text{SAC/ACC3D}, \text{SAC/ACC}\}, \\ method &\in \{\text{SER}, \text{KNN}, \text{LSTM}\}. \end{aligned}$$

To allow error comparisons across estimation tasks, we select 10 athletes with the largest amount of data and use their last collections (including multiple running speeds) as test data. Specifically, let $Test_i, i = 1, \dots, 10$ represent the last collections of these athletes, $Train_i$ their other collections, and $Train_{REST}$ the data of the remaining athletes.

- In the OTHERS scenario, we leave out one of the $Test_i, i = 1, \dots, 10$ as testing set and use all $Train_j$ and $Test_j$ with $j \neq i$, and $Train_{REST}$ as training set to select hyperparameters (with k -fold cross validation; each fold has the data of 8 of the 43 remaining athletes), resulting in 10 different models.
- In the PERSONAL scenario, we leave out one of the $Test_i, i = 1, \dots, 10$ as testing set and use only $Train_i$ as training set to select hyperparameters (with k -fold cross validation;

each fold has the data of 1 collection of the athlete), also resulting in 10 different models.

- In the EVERYONE scenario, we leave out all of $Test_i, i = 1, \dots, 10$ as testing set and use all of $Train_i, i = 1, \dots, 10$ and $Train_{REST}$ as training set to select hyperparameters (with k -fold cross validation; each fold has the data of 20 collections), resulting in a single model.

For each estimation task, after hyperparameter optimization with k -fold cross validation, the entire training set is used for training. Reported estimation error is the average obtained by models $i = 1, \dots, 10$ on $Test_i$ in the OTHERS and PERSONAL scenarios, or by the single model of EVERYONE on $Test_i, i = 1, \dots, 10$. Note that test data of a model is never used for training nor hyperparameter selection; data with multiple running speeds is included during training and hyperparameter optimization, and also during testing. For LSTM training and hyperparameter optimization, we use early stopping with 30-epoch patience (i.e., select the best parameters observed during training, which continues until no improvements in validation error are observed for 30 epochs).

We also present results for an additional scenario of interest for the LSTM method, where hyperparameter selection and training are carried out as in the OTHERS scenario, but the resulting models are then fine-tuned using $Train_i$ (as in the PERSONAL scenario, using k -fold cross validation to select the number of fine-tuning epochs) before evaluating their estimation error on $Test_i$; this scenario, which results in a model for each athlete, is particularly common for neural networks with many parameters, where using a pretrained model mitigates the issue of data scarcity.

3.3.2 Error Metrics for Estimated GRF Waveforms

The machine learning methods estimate, for each step, the components of the GRFs $\vec{g}(t) = (g_x(t), g_y(t), g_z(t))$ at each time point $t = 1, \dots, T$. We indicate the estimations by $\hat{g}_x(t)$,

$\hat{g}_y(t)$, and $\hat{g}_z(t)$, respectively, and we evaluate the Root Mean Squared Error (RMSE) and the Relative Root Mean Squared Error (rRMSE) for each component $d \in \{x, y, z\}$. In the main results we focus on the vertical component g_z , which is the primary target in related wearable GRF studies; anterior–posterior (g_y) errors appear in Appendix A.3, while mediolateral (g_x) is estimated by the same models but is not tabulated separately.

$$RMSE(g_d, \hat{g}_d) = \sqrt{\frac{\sum_{t=1}^T [g_d(t) - \hat{g}_d(t)]^2}{T}} \quad (3.1)$$

$$rRMSE(g_d, \hat{g}_d) = \frac{RMSE(g_d, \hat{g}_d)}{[\text{RANGE}(g_d) + \text{RANGE}(\hat{g}_d)]/2} \quad (3.2)$$

where $\text{RANGE}(v) = \max_{t=1, \dots, T} v(t) - \min_{t=1, \dots, T} v(t)$. We compute the average of these error metrics across estimated steps. For reference, related literature has reported $RMSE(g_z, \hat{g}_z)$ in the range of 0.14 BW (i.e., N/kg) to 0.21 BW [106, 27] for vertical GRF estimations and $rRMSE(g_z, \hat{g}_z)$ in the range of 6% to 14% [24, 42]; we note that the differences in results are partly due to differences in participants, sensors, and data curation between the datasets used in related works. Note also that GRFs are normalized by body weight in our dataset; even when omitted, RMSE errors are relative to body weights of the athletes.

3.3.3 Error Metrics for Estimated Biomechanical Variables

GRF waveforms measured during foot contact are frequently used by domain experts to calculate discrete biomechanical variables representing different characteristics of a running step. We consider the biomechanical variables *Loading Rate*, *Contact Time*, *Braking Time*, *Braking Percentage*, *Active Peak*, *Average Vertical Force*, *Net Vertical Impulse*, and *A/P Velocity Change* (defined in [2] and Appendix A.2). We evaluate each biomechanical variable f from the estimated GRF waveforms $\hat{g}_x(t), \hat{g}_y(t), \hat{g}_z(t)$ and from their actual values $g_x(t), g_y(t), g_z(t)$ for $t = 1, \dots, T$, and we compute the mean absolute percentage error (MAPE), i.e., the mean of $|f(\hat{g}_x, \hat{g}_y, \hat{g}_z) - f(g_x, g_y, g_z)| / |f(g_x, g_y, g_z)|$ across different steps. In addition to these metrics, we also study the effects of different estimation models on the

resulting waveforms and their interpretability in Section 3.5.

3.4 Methods

3.4.1 SVD Embedding Regression (SER)

As a lightweight alternative to deep learning methods for the estimation of GRFs from IMU signals (acceleration and angular velocity at different body locations), SER uses linear regression between SVD embeddings of input (IMU) and output (GRF) data; to reconstruct the GRF signals from the predicted output embedding (the *pre-image problem*), we use the right singular vectors of the training data. This approach (which can be viewed as a natural generalization of *Principal Component Regression* at a high dimension [37]) is similar to transduction of structured data [40], but pre-image calculation is very fast, providing a lightweight alternative to deep learning methods.

SVD Embedding of IMU and GRF Signals

We organize our training data into two matrices, A_{IMU} Fig. 3.2a and A_{GRF} Fig. 3.2b. Each row of these matrices corresponds to a different batch of S consecutive running foot contacts from a measurement (i.e., steps of an athlete running at a given speed); for each running step in a batch and time point t (200 per step), the columns of A_{IMU} include the IMU signals (e.g., the L2 norm of acceleration and angular velocity signals $\|\vec{a}_s(t)\|, \|\vec{a}_{lr}(t)\|, \|\vec{\omega}_s(t)\|, \|\vec{\omega}_{lr}(t)\|$ in the ALL case), while the columns of A_{GRF} include the components of the GRFs, i.e., $g_x(t), g_y(t), g_z(t)$.

To obtain low-dimensional embeddings of the training data, we compute the SVD decomposition of the matrices $A_{IMU} \in \mathbb{R}^{n \times m}$ and $A_{GRF} \in \mathbb{R}^{n \times p}$, i.e.,

$$\begin{aligned} A_{IMU} &= U_{IMU} \Sigma_{IMU} V_{IMU}^T \\ A_{GRF} &= U_{GRF} \Sigma_{GRF} V_{GRF}^T \end{aligned} \tag{3.3}$$

where: $U_{IMU} \in \mathbb{R}^{n \times n}$ and $U_{GRF} \in \mathbb{R}^{n \times n}$ are orthogonal matrices (with left singular vectors as columns); $\Sigma_{IMU} \in \mathbb{R}^{n \times m}$ and $\Sigma_{GRF} \in \mathbb{R}^{n \times p}$ are rectangular diagonal matrices (with singular values in descending order on the diagonal); $V_{IMU} \in \mathbb{R}^{m \times m}$ and $V_{GRF} \in \mathbb{R}^{p \times p}$ are orthogonal matrices (with right singular vectors as columns).

We obtain low-rank approximations by keeping only the first r singular values of the SVD decomposition, i.e., the first r columns of the U and V matrices, and the first r rows/columns of Σ :

$$\begin{aligned} A_{IMU} &\approx \bar{U}_{IMU} \bar{\Sigma}_{IMU} \bar{V}_{IMU}^T \\ A_{GRF} &\approx \bar{U}_{GRF} \bar{\Sigma}_{GRF} \bar{V}_{GRF}^T \end{aligned} \tag{3.4}$$

with $\bar{U}_{IMU} \in \mathbb{R}^{n \times r_{IMU}}$, $\bar{\Sigma}_{IMU} \in \mathbb{R}^{r_{IMU} \times r_{IMU}}$, $\bar{V}_{IMU} \in \mathbb{R}^{m \times r_{IMU}}$ and $\bar{U}_{GRF} \in \mathbb{R}^{n \times r_{GRF}}$, $\bar{\Sigma}_{GRF} \in \mathbb{R}^{r_{GRF} \times r_{GRF}}$, $\bar{V}_{GRF} \in \mathbb{R}^{p \times r_{GRF}}$. On our dataset, we use ranks $r_{IMU} = r_{GRF} = 6$, which retain at least 95% of the energy of Σ_{IMU} and Σ_{GRF} , respectively (i.e., the sum of the squares of the retained singular values is at least 95% of the sum of the squares of all the singular values). The rank 6 approximation of GRFs has an average RMSE of 0.079 BW or rRMSE of 2.1%; the same accumulative energy is also chosen in the literature [38, 39]. We also select the number of steps per row $S \in \{2, 3, 5, 6, 10, 12, 15, 20, 30, 60\}$, using a validation set. Larger ranks r_{IMU} and r_{GRF} work similarly well, while the method is sensitive to S , as illustrated in Fig. 3.3.

Training and Estimation using SVD Embeddings

After low-rank approximation, each row of matrices $\bar{U}_{IMU}$ and $\bar{U}_{GRF}$ is a vector with r_{IMU} and r_{GRF} components representing the embeddings (i.e., the features) of IMU (input) and GRF (output) signals, respectively, for a batch of S running steps in the training set. We train a predictor for each component of the GRF embedding using *least squares regression* with elastic net regularization, i.e., we select, for each $j = 1, \dots, r_{GRF}$, the parameters

$\beta_j \in \mathbb{R}^{r_{IMU}}$ and $\alpha_j \in \mathbb{R}$ minimizing the loss

$$\sum_{i=1}^n [(\bar{U}_{GRF})_{ij} - ((\bar{U}_{IMU})_{i*}\beta_j + \alpha_j)]^2 + \lambda_2 \|\beta_j\|_2^2 + \lambda_1 \|\beta_j\|_1$$

where $(\bar{U}_{IMU})_{i*} \in \mathbb{R}^{r_{IMU}}$ represents the i th row of $\bar{U}_{IMU}$ and $(\bar{U}_{GRF})_{ij} \in \mathbb{R}$ represents component j of the GRF embedding for the i th training example. The regularization weights $\lambda_1 \geq 0$ and $\lambda_2 \geq 0$ are selected for each estimation task using a validation set.

Given the IMU signals $x \in \mathbb{R}^{1 \times m}$ of a new sequence of S steps, we estimate the GRF signals $y \in \mathbb{R}^{1 \times p}$ by (Fig. 3.4):

1. Calculating the embedding $\tilde{x} \in \mathbb{R}^{1 \times r_{IMU}}$ of the new IMU signals as $\tilde{x} = x \bar{V}_{IMU} \bar{\Sigma}_{IMU}^{-1}$;
2. Predicting the embedding $\tilde{y} \in \mathbb{R}^{1 \times r_{GRF}}$ of the corresponding GRF signals as $(\tilde{y})_j = \tilde{x} \beta_j + \alpha_j$ for each $j = 1, \dots, r_{GRF}$;
3. Reconstructing the estimated GRF signals $y \in \mathbb{R}^{1 \times p}$ as $y = \tilde{y} \bar{\Sigma}_{GRF} \bar{V}_{GRF}^T$.

3.4.2 k -Nearest Neighbors Regression

As a lightweight baseline, we apply k -Nearest Neighbors regression (KNN), where the GRFs are estimated by combining the GRF signals of the k training examples with most similar IMU signals. Specifically, given the IMU signals $x \in \mathbb{R}^{1 \times m}$ for a new sequence of S steps, we estimate its GRF signals $y \in \mathbb{R}^{1 \times p}$ by:

1. Sorting the sequences of S steps of the training set, $x_i \in \mathbb{R}^{1 \times m}$ for $i = 1, \dots, n$, by their Euclidean distances $d(x, x_i) = \|x - x_i\|_2$;
2. Selecting the indices $\mathcal{K} \subseteq \{1, \dots, n\}$ of the k training sequences with lowest distances;

3. Estimating $y \in \mathbb{R}^{1 \times p}$ as

$$y = \frac{\sum_{i \in \mathcal{K}} w_i y_i}{\sum_{i \in \mathcal{K}} w_i}, \quad w_i = \frac{1}{d(x, x_i) + \varepsilon}$$

where $y_i \in \mathbb{R}^{1 \times p}$ are the GRF signals associated with the IMU signals x_i in the training set, for $i = 1, \dots, n$, and $\varepsilon > 0$ is a small constant that avoids division by zero when a neighbor coincides with x .

For each estimation task, we select the number of neighbors k and the number of consecutive steps S in a sequence using a validation set. While k has a minor effect on estimation error (additional neighbors after $k = 10$ have lower weights and provide minor improvements, as illustrated in Fig. 3.5), S can have an important effect for some estimation tasks, as illustrated in Fig. 3.3.

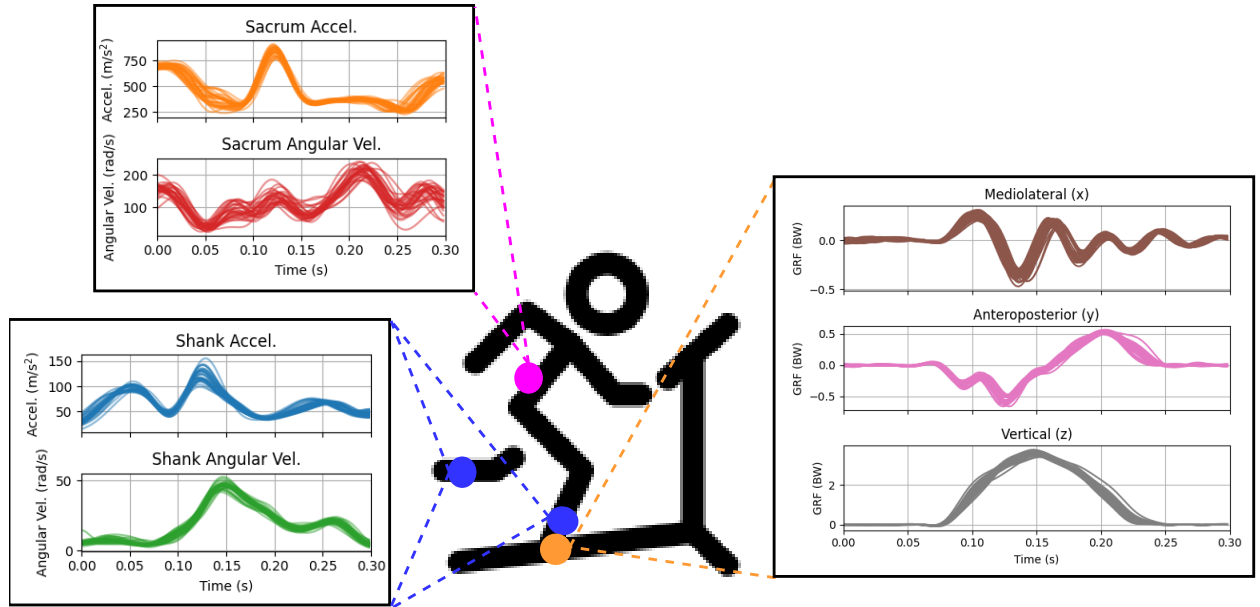
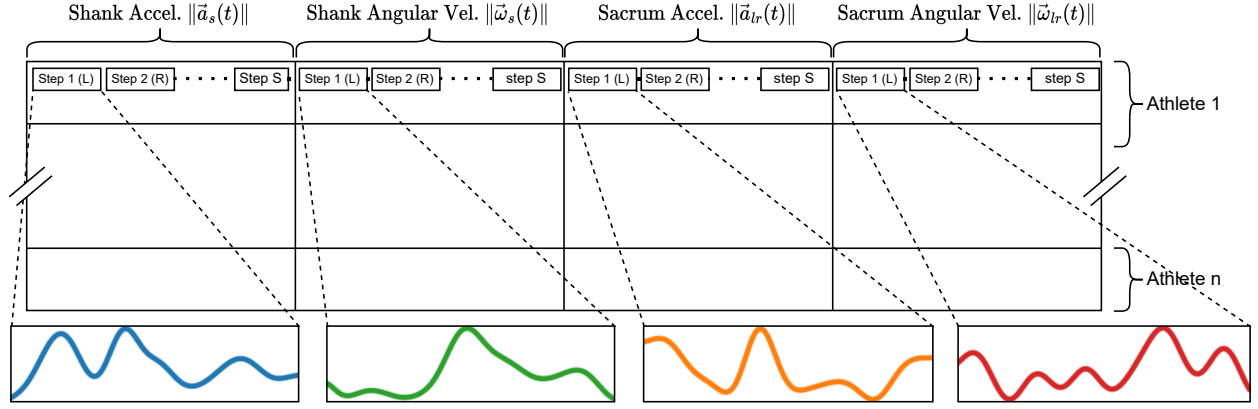
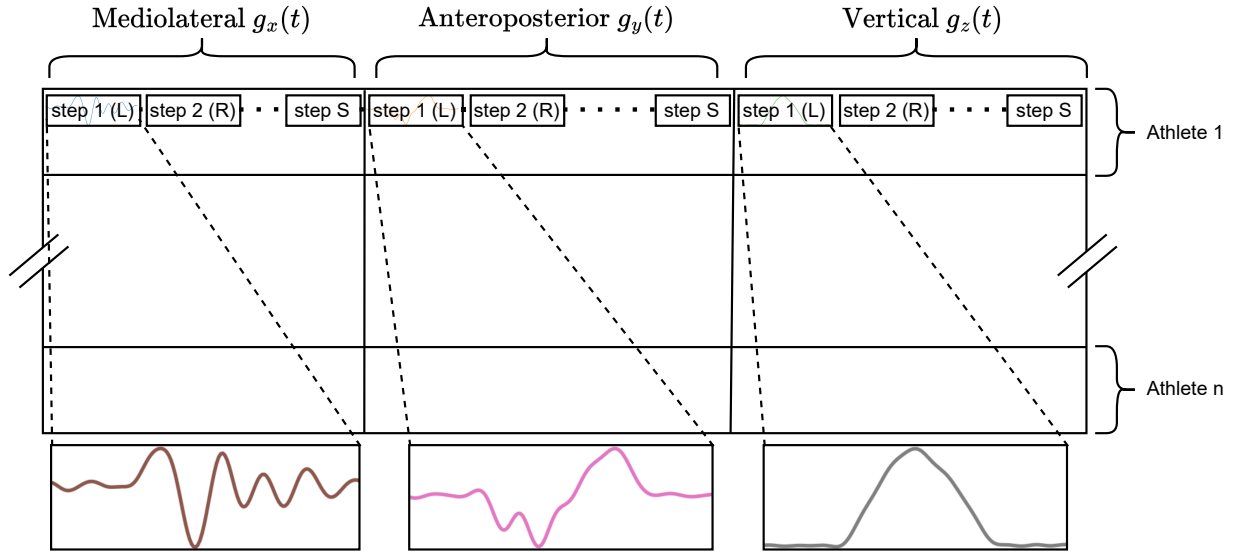


Figure 3.1: Example of signals measured during running from the respective sensors. (Right) 3 components of GRFs, i.e., the x, y, z axes (mediolateral, anterior-posterior, vertical, respectively), measured from instrumented treadmills. (Left, Top) magnitude of IMU signals including acceleration and angular velocity from the sacrum. (Left, Bottom) magnitude of IMU signals including acceleration and angular velocity from the left and right shanks

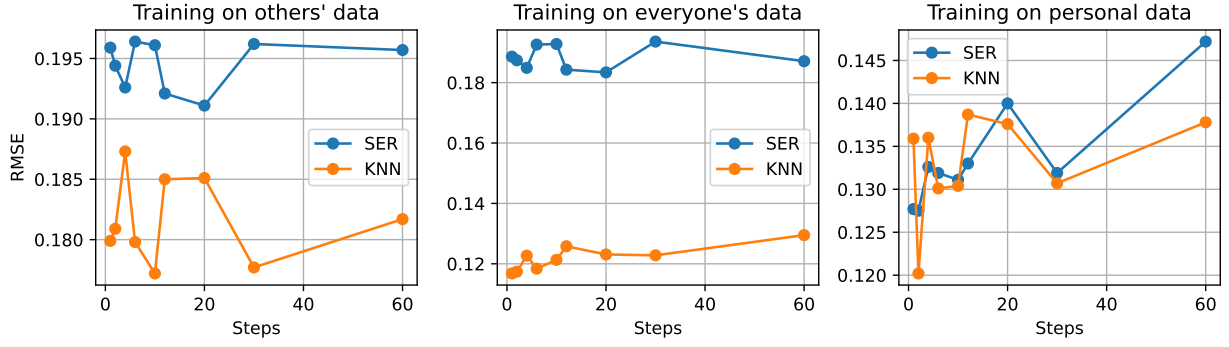


(a) A_{IMU}

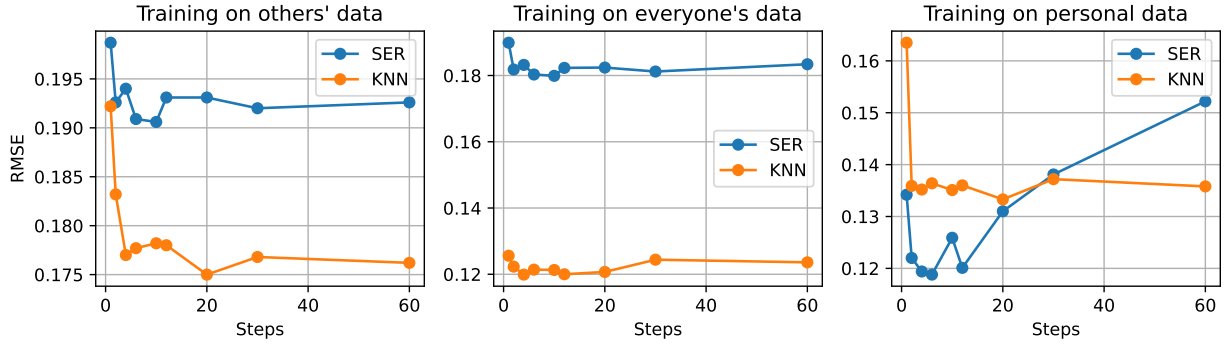


(b) A_{GRF}

Figure 3.2: Organization of input (IMU) and output (GRF) matrices for the SER method. Each row includes data for multiple left/right steps and multiple signals; data of an athlete can span multiple rows.



(a) Input from ALL signals



(b) Input from SAC/ACC3D signals

Figure 3.3: RMSE of vertical GRF estimated with SER and KNN for different numbers of steps S in a batch

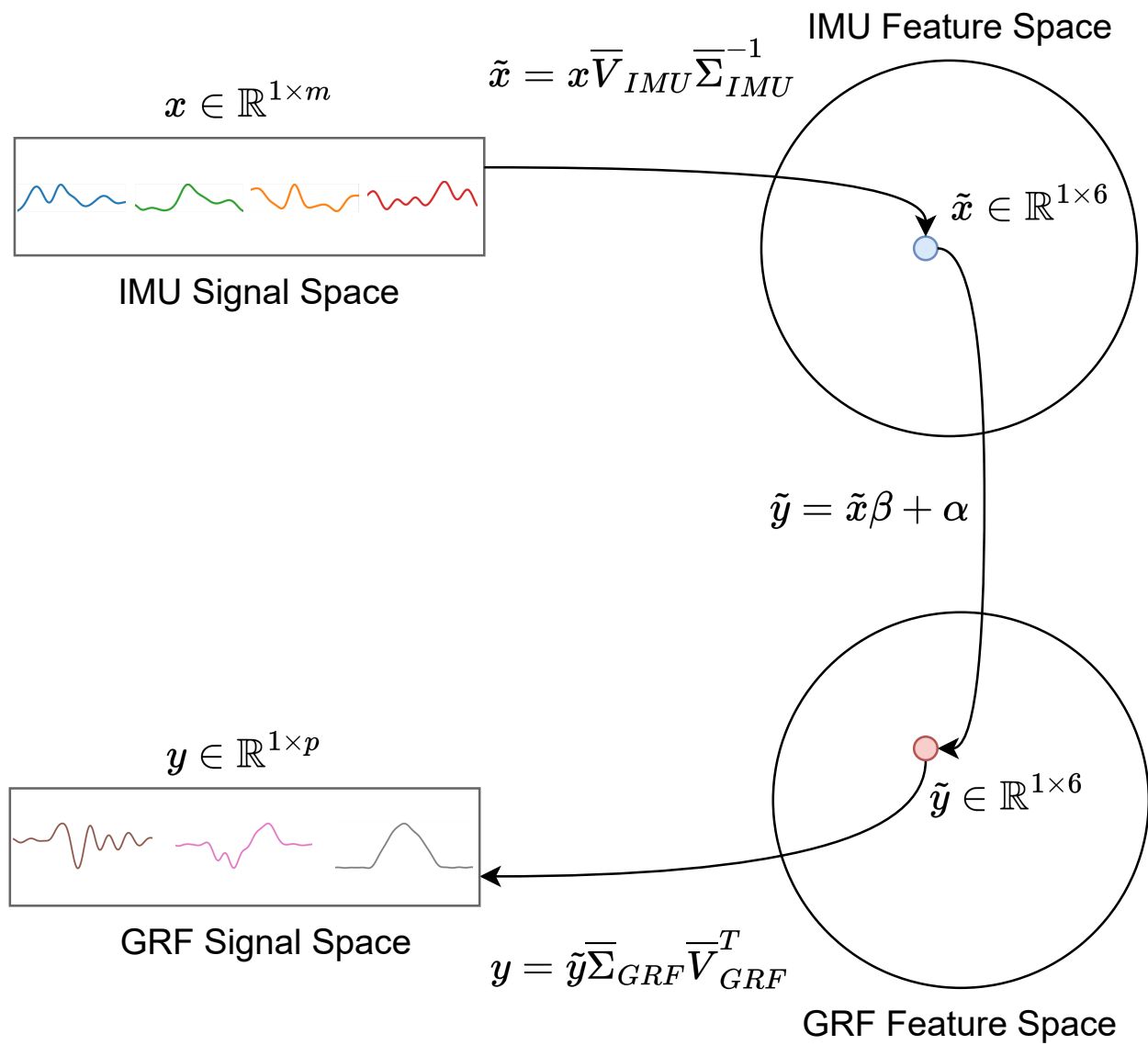


Figure 3.4: Estimating GRFs with *SVD Embedding Regression*

Hyperparameter	Values
LSTM Units	125, 250, 300, 500, 800, 1000
Layer 1 Neurons	5, 10, 20, 35, 50
Layer 2 Neurons	10, 20, 35, 50
Layer 3 Neurons	5, 10, 20, 35, 50
Batch Size	8, 16, 32
Learning Rate	0.0001, 0.0003, 0.0005, 0.0007, 0.001
Dropout Rate	0.2, 0.4

Table 3.2: LSTM Hyperparameter Search Space

3.4.3 Long Short-term Memory Networks

As a deep learning baseline, we adapt the *long short-term memory* (LSTM) neural network of [28] to estimate *all components* of the GRFs (instead of the only vertical component estimated in [28]). The model estimates $\vec{g}(t) = (g_x(t), g_y(t), g_z(t))$ from the IMU signals in the time window $[t - W, t]$ of size $W > 0$ (e.g., in the ALL case, $\|\vec{a}_s(u)\|, \|\vec{a}_{lr}(u)\|, \|\vec{\omega}_s(u)\|, \|\vec{\omega}_{lr}(u)\|$ for $u \in [t - W, t]$). Similarly to [28], we also provide the mean, standard deviation and range of IMU signals over the window $[t - W, t]$ as inputs, together with the running speed and the weight of the athlete.

The architecture of the network is depicted in Fig. 3.6: after a bidirectional LSTM layer with *tanh* activations and dropout (a regularization approach), we use three fully-connected layers with ReLU activations. The model is trained using the *Adam* optimizer as a standard choice and the mean square error as loss function. We select the number of units of each layer, batch size, learning rate, and dropout rate using a validation set; the search space is reported in Table 3.2. Notably, this machine learning approach has much higher training times than SER and KNN (up to 1 hour for each combination of hyperparameters) and the largest search space (18,000 combinations of hyperparameters). Due to memory limitations of the GPUs used for training (NVIDIA Titan X, 12 GB of RAM [107]), we were not able to train models with more than 1000 LSTM units, although hyperparameter selection suggests that a higher number of LSTM units could be beneficial, albeit quite costly.

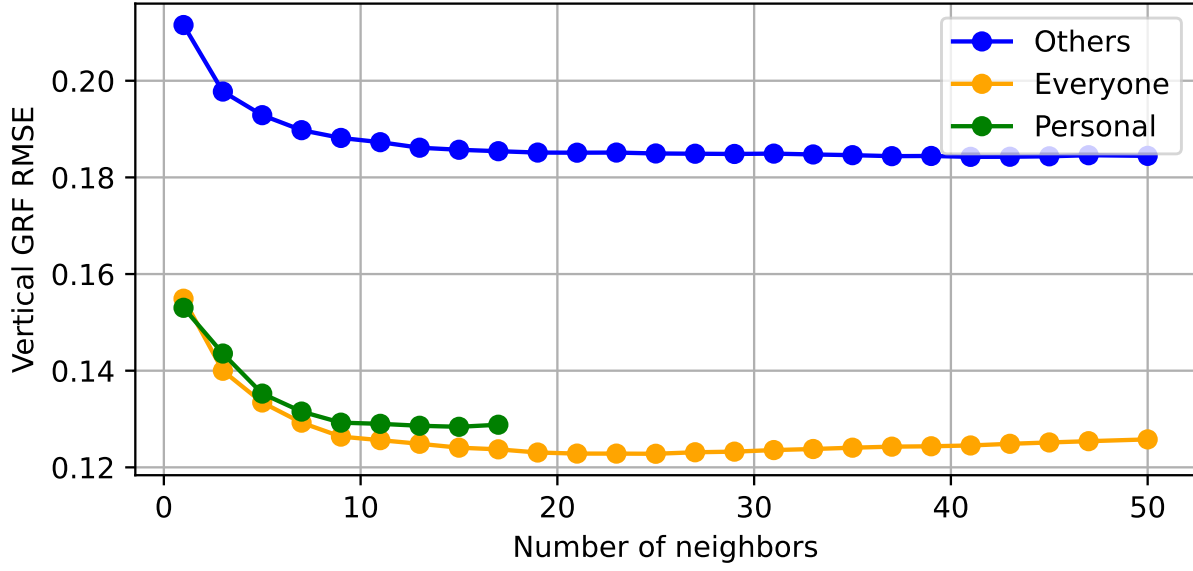


Figure 3.5: RMSE of vertical GRF for different numbers of neighbors k using KNN regression

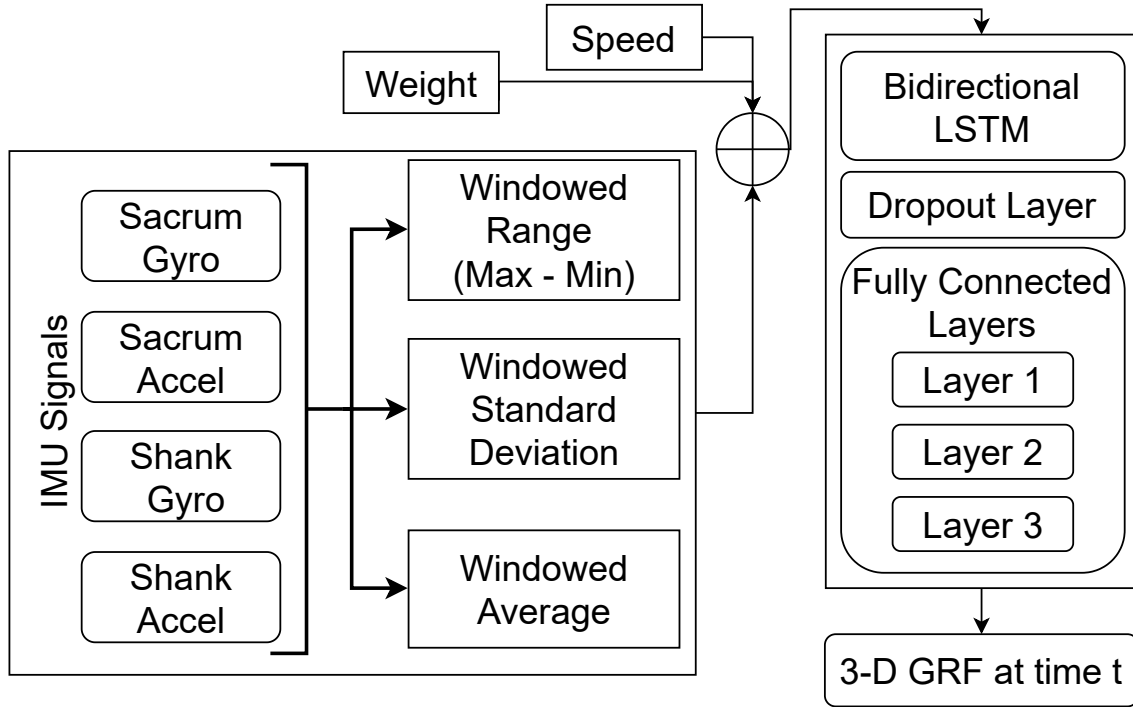


Figure 3.6: LSTM model architecture. The inputs are the mean, standard deviation, and range calculated from IMU signals (ALL as shown in the figure) for a time window, in addition to the weight and running speed of the athlete; a bidirectional LSTM layer is followed by dropout regularization and three fully connected layers.

Input Signals	Scenario OTHERS			Scenario EVERYONE			Scenario PERSONAL		
	SER	KNN	LSTM	SER	KNN	LSTM (FINE-TUNED)	SER	KNN	LSTM
ALL	0.197	0.180	0.126	0.187	0.118	0.124 (0.117)	0.130	0.122	0.134
ACC	0.220	0.197	0.177	0.210	0.125	0.175 (0.151)	0.127	0.127	0.143
ANG	0.197	0.187	0.183	0.190	0.130	0.183 (0.180)	0.133	0.132	0.171
SHANK	0.215	0.210	0.206	0.205	0.149	0.209 (0.190)	0.139	0.134	0.188
SACRUM	0.217	0.210	0.289	0.205	0.129	0.286 (0.270)	0.136	0.137	0.184
SAC/ACC3D	0.194	0.181	0.171	0.185	0.122	0.177 (0.160)	0.128	0.132	0.178
SAC/ACC	0.198	0.190	0.187	0.190	0.130	0.206 (0.177)	0.129	0.133	0.193

Table 3.3: RMSE (in body weight units, BW, i.e., N/kg) of vertical GRF estimations $RMSE(g_z, \hat{g}_z)$ for different input signals, data scenarios, machine learning methods (results highlighted in blue are optimal or less than 0.010 from optimal for a scenario and set of input signals)

Input Signals	Scenario OTHERS			Scenario EVERYONE			Scenario PERSONAL		
	SER	KNN	LSTM	SER	KNN	LSTM (FINE-TUNED)	SER	KNN	LSTM
ALL	6.5	6.0	4.2	6.2	3.9	4.2 (3.9)	4.2	4.0	4.3
ACC	7.4	6.6	6.0	7.0	4.1	5.9 (5.2)	4.1	4.1	4.8
ANG	6.5	6.2	6.1	6.3	4.3	6.0 (6.0)	4.3	4.3	5.9
SHANK	7.2	7.0	7.0	6.8	4.9	7.0 (6.7)	4.5	4.4	6.5
SACRUM	7.2	6.9	10.1	6.8	4.2	9.8 (9.6)	4.4	4.5	6.3
SAC/ACC3D	6.4	6.0	5.8	6.2	4.0	5.9 (5.4)	4.1	4.3	6.0
SAC/ACC	6.6	6.3	6.3	6.3	4.3	6.9 (6.1)	4.2	4.4	6.5

Table 3.4: rRMSE (%) of vertical GRF estimations $rRMSE(g_z, \hat{g}_z)$ for different input signals, data scenarios, machine learning methods (results highlighted in blue are optimal or less than 0.5% from optimal for a scenario and set of input signals)

Input Signals	Scenario OTHERS			Scenario EVERYONE			Scenario PERSONAL		
	SER	KNN	LSTM (GPU)	SER	KNN	LSTM (GPU)	SER	KNN	LSTM (GPU)
ALL	0.001	2.256	4.221 (0.497)	0.001	2.088	3.219 (0.472)	0.001	0.042	4.922 (0.558)
ACC	0.003	1.165	4.325 (0.521)	0.002	1.089	3.251 (0.473)	0.001	0.022	5.139 (0.586)
ANG	0.004	1.161	4.605 (0.543)	0.0004	1.064	3.180 (0.474)	0.001	0.025	4.179 (0.516)
SHANK	0.005	1.165	4.014 (0.503)	0.002	1.062	3.216 (0.476)	0.001	0.021	4.739 (0.554)
SACRUM	0.004	1.156	4.557 (0.545)	0.0004	1.067	3.295 (0.480)	0.001	0.023	4.945 (0.563)
SAC/ACC3D	0.001	1.716	3.132 (0.439)	0.002	1.574	5.160 (0.579)	0.001	0.030	0.784 (0.248)
SAC/ACC	0.005	0.518	3.221 (0.450)	0.002	0.411	3.153 (0.461)	0.001	0.013	0.862 (0.272)

Table 3.5: Average inference time (in seconds) to estimate 3D GRFs of a collection (120-180 steps) for an athlete. Inference time is measured on an Intel i7-6800K CPU; for LSTM we also report average inference time using a TITAN X GPU.

Acronym	Training Data of the Scenario
OTHERS	IMU and GRF data of other athletes
PERSONAL	IMU and GRF data of the same athlete
EVERYONE	IMU and GRF data of all athletes

Table 3.6: Scenarios for the estimation tasks

3.5 Results and Discussion

3.5.1 Estimation of the Normal GRF Waveform

First, we focus on the estimation of RMSE and rRMSE for the vertical GRF, i.e., $RMSE(g_z, \hat{g}_z)$ and $rRMSE(g_z, \hat{g}_z)$, obtained by different machine learning methods for each set of input signals and scenario. The vertical component of the GRF is particularly important for the study of stress response in bone and soft tissue [3, 4] and provides the information used to compute several discrete biomechanical variables (discussed in Section 3.5.3). Results are reported in Tables 3.3 and 3.4, while input signals and scenarios of different estimation tasks are summarized in Tables 3.1 and 3.6, respectively. Although limited to our dataset, this evaluation allows us to make the following observations regarding the use of machine learning methods for the estimation of GRFs from IMU signals.

Use of Acceleration and Angular Velocity Signals

We observe that, in all scenarios and for all machine learning methods, the use of both acceleration and angular velocity signals is preferable, since RMSE and rRMSE are similar or significantly lower. In particular, using only acceleration (case ACC) or angular velocity signals (case ANG) is generally worse than using both (case ALL). Since most IMU sensors collect both signals, these improvements can be obtained without additional costs of the data collection process, despite the focus of related work on the exclusive use of acceleration signals [28].

Sensor Locations

We observe that, in all scenarios and for all machine learning methods, the use of sensors at both sacrum and left/right shank (case ALL) is preferable, since RMSE and rRMSE are either similar or significantly lower than when each sensor location is used exclusively (cases SACRUM and SHANK, respectively). While expected for deep learning methods (LSTM), this result highlights that lightweight machine learning methods such as SER and KNN can also provide effective estimation models for complex IMU data collected from multiple locations.

Use of Multidimensional Acceleration at the Sacrum

Since related literature [105, 28] focuses on acceleration signals collected using sensors located at the sacrum, we explore the benefits of a multidimensional acceleration signal $\vec{a}_s(t)$ (SAC/ACC3D) to estimate GRFs, instead of its L2 norm $\|\vec{a}_s(t)\|$ (case SAC/ACC). We observe that, in all scenarios and for all machine learning methods, the use of a multidimensional acceleration signal at the sacrum is preferable to its L2 norm, since RMSE and rRMSE are either similar or significantly lower. We also note that, when limited to the sacrum location, the use of multidimensional acceleration signals (SAC/ACC3D) is preferable to the use of the L2 norm of both acceleration and angular velocity (SACRUM), and even to the use of the L2 norm of only acceleration signals at the sacrum and left/right shanks (ACC). Our experiments indicate that the increase in the model computation cost is negligible.

Use of Lightweight Machine Learning Methods

While prior work [28] adopts deep learning methods based on LSTM neural networks, we observe that lightweight approaches can provide similar or lower error of the estimated GRFs for specific scenarios and input signals. In the scenario EVERYONE, KNN is preferable to LSTM and SER, as it provides much lower RMSE and rRMSE for most combinations of input signals; in the scenario PERSONAL, both SER and KNN are preferable to LSTM, for all combinations of input signals. Notably, in the setting of [28] (scenario OTHERS, signals SAC/ACC3D), KNN and LSTM perform very similarly on our dataset (rRMSE of 6.0% and

5.8%, respectively; as a reference, LSTM incurred an rRMSE of 6.4% in [28]).

We attribute the improved performance of KNN in the scenarios EVERYONE and PERSONAL to the inclusion of historical running data collected for the target athlete in the training set: KNN is able to exploit the patterns specific to the target athlete, while ignoring data from other athletes; in contrast, LSTM obtains similar estimation error in the OTHERS and EVERYONE scenarios, but obtains lower error when used with sensors at multiple body locations (this observation holds in the fine-tuned results). Finally, SER is able to model the GRFs of an athlete very accurately in the PERSONAL scenario. We observe that the estimation error of a machine learning method in a specific scenario depends on multiple factors, including the total amount of available data (which is significantly lower in the PERSONAL scenario, possibly resulting in higher estimation error), the lack of personal data (which may result in higher estimation error in the OTHERS scenario), and the ability of an algorithm to hone in on personal data when it is mixed with that of other athletes (as in the EVERYONE case).

Note that our dataset is relatively small (as is common in this field) and not all gait patterns are well-represented; estimation error improvements from the inclusion of personal data indicate that linear models can specialize further to an individual, rather than reflecting a failure to generalize to the broader population.

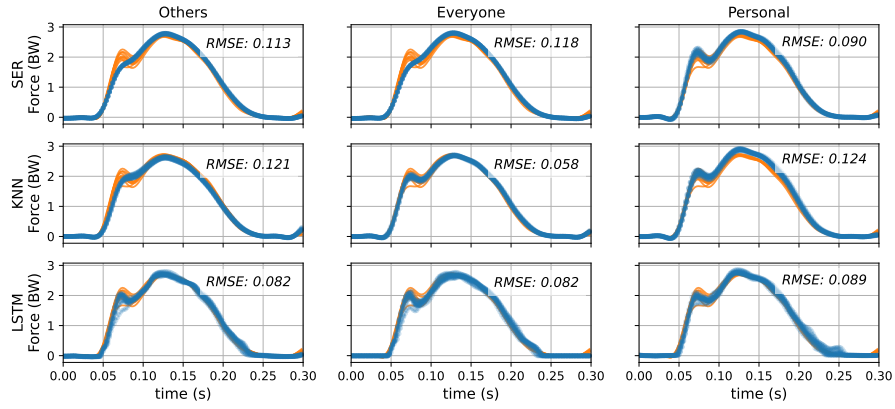
Resource and Energy Utilization

We observe that the upfront training cost of LSTM models is more than three orders of magnitude higher than that of SER, which drastically reduces GPU hours, carbon footprint, and cloud or on-premise computing costs. In our study, training of LSTM models (including architecture search and hyperparameter tuning) required parallel processing on 13 TITAN X GPUs for 30 days, while training of SER models (including exhaustive hyperparameter search) required only 5 hours on a single commodity CPU. Our wall-plug measurements of power consumption indicated 200 W for each GPU (rated at 250 W by Nvidia [108]; utilization was 90%) and only 90 W for an Intel i7-6800K CPU (rated at 140 W by Intel [109]), resulting in an energy cost of 1.9 Million Watt-Hours for LSTM compared to 450 Watt-Hours for SER, giving a reduction in energy consumption by a factor of 4,160. Moreover, there is a substantial dollar cost when training LSTMs in the cloud. We estimate over \$9,000 to train the LSTMs (e.g., using an AWS g4dn.metal instance with an on-demand hourly rate of \$7.8 [110] for 9,360 GPU hours) and only \$36 to train SER (e.g., using an AWS hpc7a.12xlarge instance with an on-demand hourly rate of \$7.20 [111] for 5 hours).

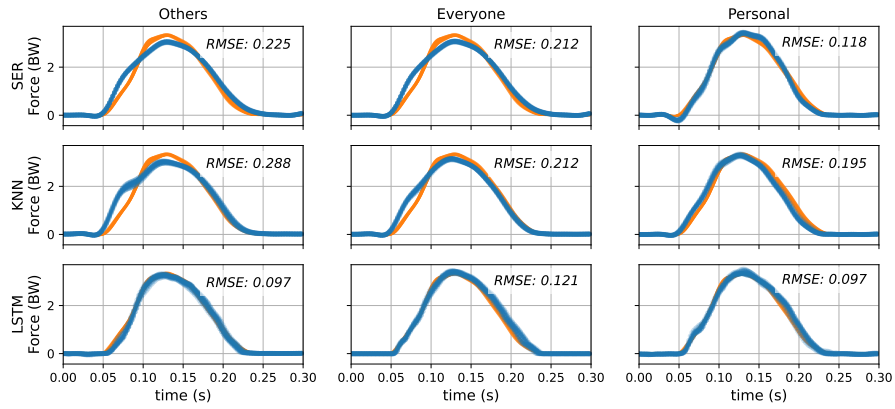
At the same time, the estimation error improvements due to LSTM are only significant in one scenario (2% rRMSE improvement when training on all IMU signals in the OTHERS scenario, while in all other cases the rRMSE improvement is at most 0.6%). Therefore, although the initial training time is a one-time cost, given that LSTM models provide limited benefits, their additional cost in terms of time, energy, and dollar expenditure is a significant drawback.

Efficient Inference for Near Real-Time Applications

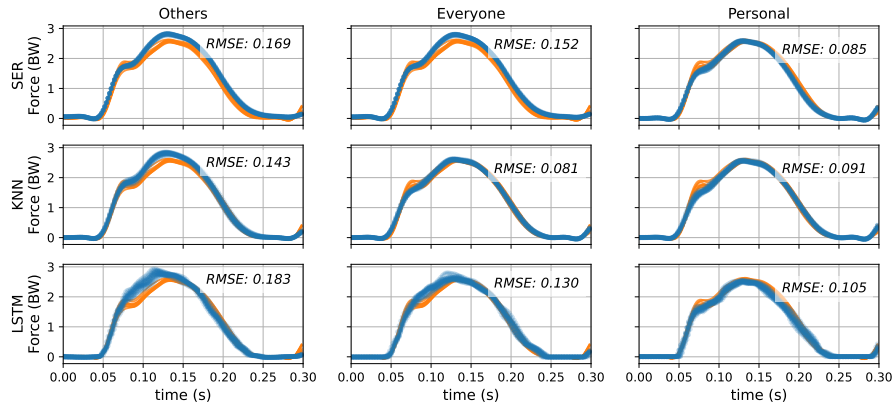
Considering inference latency for batch estimation, SER has the shortest inference time in all scenarios (Table 3.5) using CPUs. The inference time of LSTM (using CPU or GPU) depends on the size of the model (i.e., number of parameters, layers, etc.), which is determined through



(a) Athlete 1 (Major Impact Peak), Signals ALL

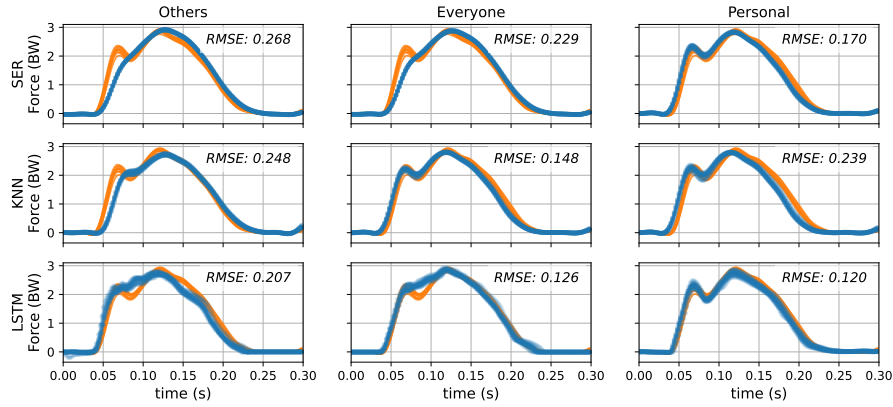


(b) Athlete 2 (No Impact Peak), Signals ALL

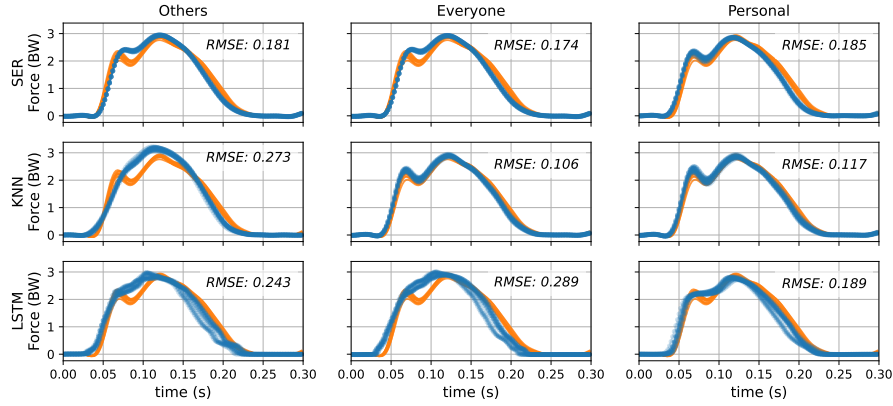


(c) Athlete 3 (Minor Impact Peak), Signals ALL

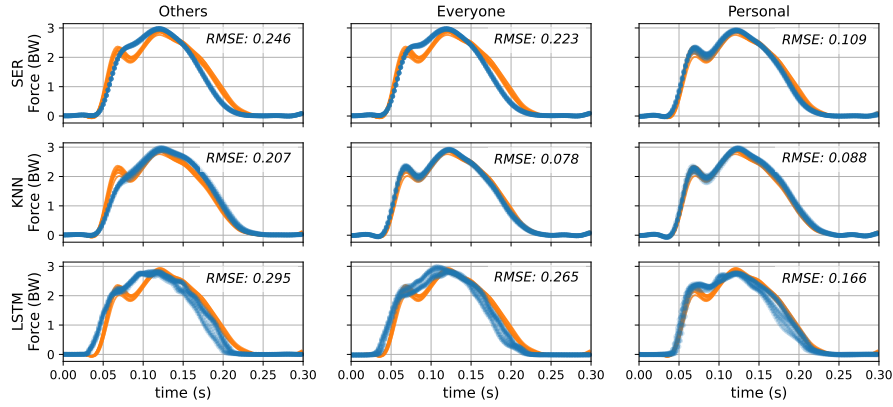
Figure 3.7: Comparing estimations (blue) and measurements (orange) of bodyweight-normalized vertical GRF $g_z(t)$ for runners with different foot-strike patterns, using all input signals (ALL). In each subgraph, the columns are the scenarios (OTHERS, EVERYONE, PERSONAL) and the rows are the machine learning methods (SER, KNN, LSTM).



(a) Athlete 4 (Major Impact Peak), Signals ALL



(b) Athlete 4 (Major Impact Peak), Signals SAC/ACC



(c) Athlete 4 (Major Impact Peak), Signals SAC/ACC3D

Figure 3.8: Effect of input signal subsets for Athlete 4 (Major Impact Peak): estimations (blue) and measurements (orange) of bodyweight-normalized vertical GRF $g_z(t)$ using all signals (ALL), sacrum acceleration only (SAC/ACC), and three-dimensional acceleration (SAC/ACC3D). Columns are the scenarios (OTHERS, EVERYONE, PERSONAL) and rows are the methods (SER, KNN, LSTM).

hyper-parameter search, whereas KNN’s inference time depends on the size of the training dataset.

We note that, while SER does not offer real-time inference, after the initial delay for the collection of 2-5 running steps (1-2 seconds) for inference, SER’s 0.3 ms inference latency (for estimating 20 steps²) is well-suited for near real-time analysis performed entirely on an edge device such as a mobile phone. This is in contrast with LSTM model inference which would require approximately 15 seconds (to estimate 20 steps) on the same mobile device (during which time an athlete would run at least another 30 steps). In order to achieve near real-time analysis (with some initial delay), inference time needs to be less than the running time for the steps being analyzed, or the analysis needs to skip some steps during inference. Another approach might be to process batches of steps. However, due to memory constraints, LSTM inference time for batch analysis grows linearly with the batch size. Yet another approach to reducing LSTM inference latency on mobile devices is to use quantization. Our experiments with quantization indicated a reduction in LSTM’s inference time down to 4 seconds, which is still more than 4 orders of magnitude higher than the inference latency of SER *without* quantization. However, this reduction in inference latency comes at the cost of an increase in RMSE; our experiments indicated an average of $\approx 10\%$ increase in RMSE, with a worst case increase of over 34% in the case of using SAC/ACC3D as input signal. Thus, LSTMs are less suitable for near real-time analysis on edge devices.

We note that, to achieve 0.5 or 4 seconds inference latency for LSTM reported in Table 3.5 requires use of cloud GPUs or CPUs, respectively. Note also that on-device prediction facilitates preservation of data privacy. The reduced cost and energy consumption of simpler methods such as SER also facilitates development of GRF estimation methods on embedded components which can be integrated into treadmills in an inexpensive manner (e.g., using embedded CPUs). This opens the way to integration of estimation techniques into standard (inexpensive) treadmills receiving data from IMU sensors, providing an affordable alternative

²We report the latency for 20 steps, to make a meaningful comparison to LSTM, which uses 20 steps as in related literature.

to instrumented treadmills while allowing broader data collection.

As an application scenario (where such near real-time prediction on embedded devices would be useful), we consider a coach making suggestions to a group of (e.g., a dozen) distance runners training on treadmills. Given SER’s fast inference, the coach can simultaneously monitor GRFs and biomechanical variables (e.g., on a tablet) for the entire group of athletes and adjust their running styles based on recent steps (estimation is available after every step with SER as opposed to after a few minutes with LSTM). For example, the coach can identify fatigue from GRFs [112] and adjust the athlete’s pace to maintain good running form as a means to prevent injuries.

To quantitatively motivate such applications, we deployed the GRF estimation methods presented in this chapter on a Samsung S20 smartphone (Exynos CPU, 2.73 GHz, using a Mongoose M5 core) and measured latency and memory usage during inference for 10 steps (this is the minimum possible number of steps for the LSTM models). On average, LSTM inference required 14.9 seconds, while SER required only 0.3 ms and KNN required 36 ms; memory usage was 180 MB for LSTM, 170 MB for KNN for OTHERS and EVERYONE models and 9 MB for PERSONAL, and only 4 MB for SER.

When integration on mobile or embedded devices is not required, limited networking and privacy requirements also pose challenges to the use of cloud resources needed by LSTMs. Even when using cloud resources is an option, their cost would be orders of magnitude lower for KNN and SER.

Biomechanical Variable	Scenario OTHERS			Scenario EVERYONE				Scenario PERSONAL		
	SER	KNN	LSTM	SER	KNN	LSTM	LSTM FINE-TUNED	SER	KNN	LSTM
Loading Rate	19.4	19.6	11.6	19.4	6.5	13.6	8.1	7.2	4.9	10.0
Contact Time	9.9	10.9	8.8	12.1	4.7	6.4	4.8	4.3	4.1	4.7
Braking Time	3.9	3.5	5.9	5.9	2.0	4.3	3.4	3.1	1.7	4.5
Braking Percentage	9.5	7.4	8.6	9.2	3.9	5.1	6.0	4.6	4.3	5.2
Active Peak	5.6	4.7	1.6	5.3	2.8	1.3	2.2	2.9	2.6	2.6
Average Vert. Force	5.9	4.9	3.2	6.2	2.5	3.1	3.2	2.4	2.2	2.8
Net Vertical Impulse	10.5	8.3	5.6	10.6	4.3	5.6	6.1	4.1	4.3	8.4
A/P Velocity Change	12.8	11.1	11.4	10.0	5.4	9.3	6.1	7.3	6.4	8.7

Table 3.7: MAPE (%) of discrete biomechanical variables estimated from ALL input signals for different scenarios and machine learning methods (results highlighted in blue are optimal or less than 0.5% from optimal for a scenario and biomechanical variable)

3.5.2 Artifacts in Estimated Waveforms

We observe that each machine learning method can result in different anomalies in the estimated GRFs, which are significant for its evaluation by domain experts.

For example, the estimations in Fig. 3.7a are for a runner who initiates ground contact with their heel (rearfoot strike) which leads to a pronounced impact peak in the vertical component of the GRF. In the scenario OTHERS, only LSTM estimates a pronounced impact peak, while KNN and SER estimate smoother waveforms due to the averaging of data from midfoot and forefoot runners (initiating ground contact with their mid or forefoot) with less pronounced impact peaks. Both LSTM and KNN are able to use data of rearfoot strike runners in the scenarios EVERYONE and PERSONAL, while SER accurately estimates this feature only in the scenario PERSONAL.

In Fig. 3.7b, we report the GRF of a runner who initiates contact with the front of their foot (forefoot strike) which results in no impact peak in the vertical GRF. In the scenarios OTHERS and EVERYONE, KNN and SER introduce a change in slope not present in the measured GRF; in contrast, LSTM introduces a change in slope in the scenario PERSONAL. When athletes have a minor impact peak as in Fig. 3.7c, KNN and SER tend to produce more representative waveforms, in all scenarios.

Finally, the athlete of Fig. 3.8a also has a pronounced impact peak. While KNN and SER estimate a smoother waveform resulting from the average of these patterns, LSTM tries to capture both the impact peak and also the active peak, resulting in inaccurate waveforms in the OTHERS and EVERYONE scenarios. The same phenomenon is observed when using only acceleration signals at the sacrum (Fig. 3.8b); when using multidimensional acceleration signals (SAC/ACC3D), similar anomalies are produced by SER (cases OTHERS and EVERYONE).

3.5.3 Estimation of Discrete Biomechanical Variables

Finally, we use the estimated GRF waveform to evaluate discrete biomechanical variables (defined precisely in Appendix A.2), which are of interest to domain experts for the detection of running anomalies that may lead to stress-responses in bone and soft tissue. MAPE values with respect to the measured GRFs are reported in Table 3.7 for GRF estimations using ALL input signals, for different scenarios and machine learning methods.

In the scenario OTHERS, LSTM achieves substantially lower MAPE for most biomechanical variables (except braking time and braking percentage), in line with the lower RMSE and rRMSE values of the estimated GRF waveform (Tables 3.3 and 3.4, row ALL). In contrast, in the scenario EVERYONE, LSTM achieves substantially higher MAPE than KNN, despite their similar RMSE and rRMSE values in Tables 3.3 and 3.4. Even with fine-tuning, where LSTM has a marginally lower RMSE, LSTM still achieves higher MAPE than KNN except for one variable (i.e.: Active Peak).

In the scenario PERSONAL, KNN and SER achieve substantially lower MAPE than LSTM for several biomechanical variables, e.g., loading rate and net vertical impulse, as expected. Notably, while the best RMSE and rRMSE achieved by KNN in the scenario PERSONAL (Tables 3.3 and 3.4, row ALL) are worse than those of the scenario EVERYONE, its MAPE values are substantially lower for most biomechanical variables; unexpected MAPE reductions are observed in the scenario PERSONAL also for LSTM (e.g., for loading rate and contact time). In general, similarly to related work [28], MAPE values are significantly higher for biomechanical variables that depend on sensitive features of the GRF waveform, e.g., the loading rate, which depends on rate of change of the vertical GRF.

This analysis shows that RMSE and rRMSE of the estimated GRFs are not always useful estimates of the error of the derived biomechanical variables (which depend on specific features of the GRF waveform), and that *personal data is especially useful when biomechanical variables are of interest*. The presence of various types of artifacts in the estimations from OTHERS’ data, despite achieving a low RMSE, highlights the potential for further improve-

ments.

3.5.4 Provenance

Given the linear nature of SER, it provides greater interpretability of GRF estimation than a highly non-linear model such as LSTM. Moreover, SER also allows tracing each part of the GRF estimation back to the training IMU/GRF data through the model parameters (KNN does this for the entire GRF estimation). This is often referred to as provenance, e.g., as in [113]. Ability to determine provenance is useful for both biomechanics research (to study the relationship between IMU data and GRFs) and practical applications (e.g., to detect which training data is causing model deterioration once it has been deployed in the field for a while). We are not aware of any provenance results in the literature for LSTM models, possibly because of the complex relationship between inputs and outputs due to long-term and short-term memory as well as the stochastic nature of the training process.

We also note that provenance is different from sensitivity analysis of the model outputs around specific inputs during inference, e.g., as in [114]. Although such sensitivity analysis could potentially be adapted to LSTMs (no such work exists to our knowledge), it does not provide a relationship between training data and inference outputs.

3.6 Conclusions

GRF waveforms measured during foot contact and their derived biomechanical variables can be accurately estimated from acceleration and angular velocity signals collected using wearable IMU sensors. To this end, depending on the training data and input signals, simple machine learning methods such as SER and KNN are similarly accurate or more accurate than LSTM neural networks, while requiring fewer computing resources and less energy, and allowing much faster inference on edge devices as well as faster training and hyperparameter optimization, as illustrated by our evaluation.

Notably, SER and KNN produce more accurate estimations of the GRF waveform when

personal training data (i.e., GRF and IMU measurements for an athlete) are available; in this case, the error of the estimated biomechanical variables is greatly improved with respect to LSTM neural networks. We also observed that all machine learning methods benefit from the use of both acceleration and angular velocity, and from the use of all components of the sacrum acceleration (instead of its L2 norm).

The results also identify two remaining limitations of this first SER formulation: the study does not evaluate injury-anomaly detection from estimated GRF waveforms, and provenance analysis is limited to illustrative examples rather than a full account of which training samples determine specific waveform events.

The next chapter extends SER to federated multi-task estimation of GRFs and joint-moment waveforms across participants and actions, without pooling raw data (Chapter 4).

Chapter 4

FedSER for Multi-Task Biomechanical Signal Estimation

4.1 Introduction

Ground reaction forces (GRFs) and joint moments (JMs) are key measurements used in biomechanics to study human movement. GRFs describe the external forces experienced during ground contact. Joint moments describe the net torque around each joint during movement. These signals support athletic performance evaluation, clinical assessment, rehabilitation monitoring, and control of assistive systems such as exoskeletons and prosthetics [15, 13, 14, 115]. Direct measurement of these signals is typically performed using force plates, instrumented treadmills, and motion capture systems [115, 15, 55]. Although these methods provide gold-standard measurements, they are expensive, laboratory-bound, and difficult to scale to large populations or to monitoring in the wild.

Wearable sensors such as inertial measurement units (IMUs) provide a low-cost alternative to monitor biomechanical variables outside the laboratory. They can collect data continuously during sports, rehabilitation, and daily living [23, 55]. However, IMUs do not measure GRFs or JMs directly: they capture localized linear acceleration and angular velocity signals, and estimating GRFs or JMs from these signals remains a key challenge in biomechanics and remote monitoring [23]. Physics-based models (e.g., forward dynamics and optimal control) can recover these quantities but typically require per-subject calibration and additional sensors for accurate estimates [24, 19, 20]. Even then, they remain compute-intensive, relying on iterative simulation and hand-crafted cost functions for each movement task [20]. This motivates data-driven estimation that learns the mapping directly from sensor signals.

State-of-the-art approaches predominantly use deep learning models, including convolutional neural networks (CNNs), long short-term memory networks (LSTMs), temporal con-

volutional networks (TCNs), transformers, and diffusion networks [28, 27, 29, 13, 15, 116]. These models can produce accurate estimates, often by leveraging richer sensing modalities, including motion capture in controlled laboratory settings [15]. However, they require costly training and extensive hyperparameter searches, while providing limited interpretability [31, 32, 115]. Their compute, energy, and memory costs grow significantly with data size and model complexity [33, 34].

In addition, due to the expensive GRFs and JMs collection process [23, 27, 55], users usually have *limited data* and some movement actions may be missing entirely from their training sets [13, 15, 14], making models trained locally less accurate. At the same time, similarly to other medical settings, biomechanical data are usually too sensitive to be shared in centralized training datasets. *Federated learning* (FL) [52] is a distributed approach that allows participants to train a model jointly, without sharing their training data: instead, they send model updates to a server, which combines them to obtain a new model that is shared with the participants; in turn, the new model is used by each participant to update a local model, and to compute new updates to send to the server. FL increases the effective training data size and the diversity of actions represented in the model, but significant challenges exist due to the different data distribution among participants (e.g., due to different movement patterns or collection processes [13, 55]). Moreover, while mobile and edge devices can collect data in the wild (e.g., using wearable sensors), they present additional challenges for FL due to their limited computation, memory, and communication capabilities [51, 53, 56]. Lightweight machine learning methods are therefore desirable.

In our previous work [115], we introduced SVD Embedding Regression (SER) as a lightweight alternative to deep learning models for the estimation of GRFs from IMU signals (e.g., collected using inexpensive wearable devices). SER uses SVD to project input and output signals into low-rank embedding spaces, where it learns a linear mapping. SER matched or exceeded the accuracy of deep LSTM baselines with substantially faster training and inference, which allows its use on mobile and edge devices.

To adapt SER to federated learning, a distributed SVD approach is required that (1) does not share local data, (2) recovers approximately the same basis as centralized SVD (to preserve accuracy), (3) converges quickly and predictably, and (4) runs within the limited memory and compute of mobile and edge devices. Several distributed SVD approaches exist, but none satisfy all these requirements. The approach of [72, 73] runs a local SVD on each client and combines the SVD decompositions at the server; notably, this approach does not construct the same SVD decomposition as a centralized SVD and requires clients to compute SVD decompositions locally, which can be expensive. The distributed power method of [74, 75] constructs all the left singular vectors associated with k largest singular values together; for large k , this approach requires significantly more memory at the clients (since all k vectors are used during the main iteration) and suffers from long execution times (since different vectors converge at different rates, but the approach updates all vectors until the slowest converges). As an alternative to distributed SVD, centralized approaches using cryptography [117, 118] or differential privacy [119, 120, 121] provide privacy, but are either compute-intensive and rely on strong assumptions about trusted parties, or result in lower accuracy due to the noise added for differential privacy.

In this chapter, we overcome these limitations by proposing Federated SER (FedSER), a federated multi-task framework based on SER that enables private training between mobile/edge devices and improves accuracy for actions missing from the training dataset. We estimate multiple biomechanical quantities, including GRFs and JMs, across diverse actions (collected in different conditions and with different movement intensities). Our multi-modal setting ranges from wearable-only inputs to combinations of wearable and laboratory-based input signals. Fig. 4.1 shows the benefits of SER and FedSER for hip flexion moment estimation. When participants do not collaborate and use only personal data, SER (blue line) outperforms both KNN [115] (yellow, a simple baseline) and TCN [15, 13] (red) across training set sizes (obtained by subsampling the training data). By utilizing data from all 11 participants, FedSER (green line) reduces the error further; even with similar total data

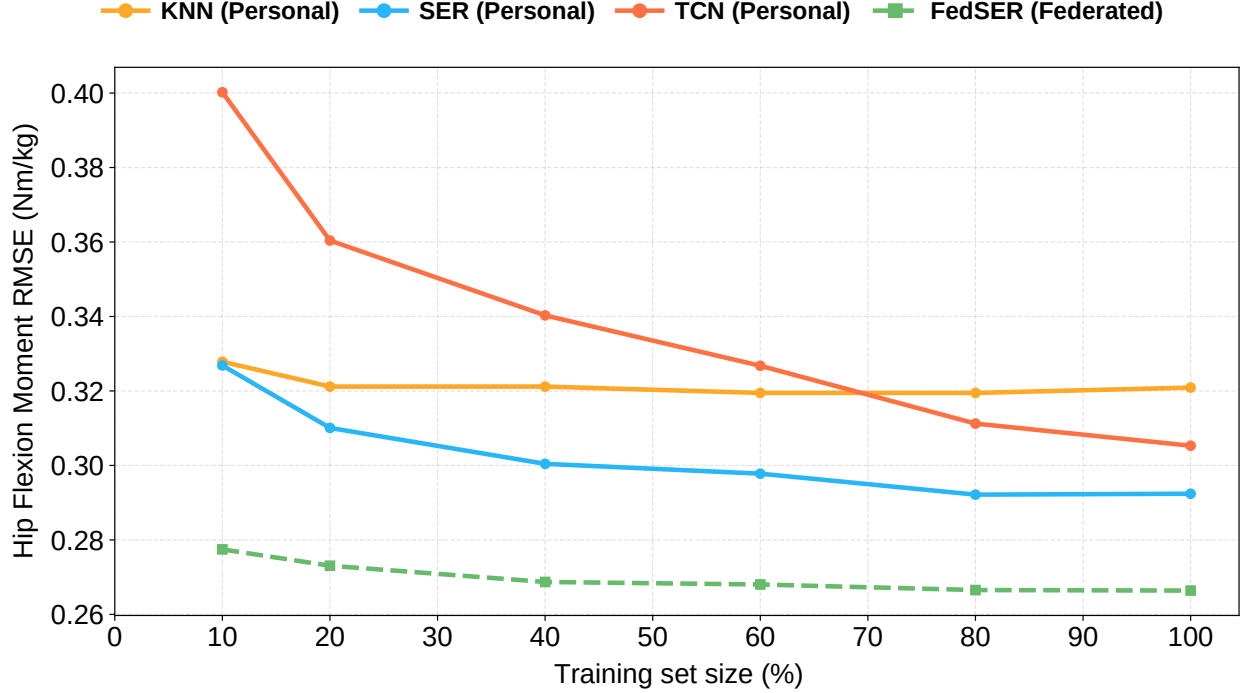


Figure 4.1: Hip flexion moment estimation error (RMSE) using IMU-only inputs under different training set sizes, with models trained on all actions except the target action

(e.g., using 10% of the data of each participant), FedSER outperforms other methods (e.g., SER using 100% of the data) due to our approach combining experts trained on specific actions.

Our contributions are as follows: (1) We show that our proposed method FedSER achieves better accuracy than deep learning models (specifically, TCNs) under multiple forms of data scarcity (limited training set sizes, missing actions in the training data, missing sensing modalities); see Sections 4.5.1, 4.5.2 and 4.5.6. (2) We evaluate the performance of FedSER on a mobile smartphone, showing orders-of-magnitude improvements with respect to deep learning models (TCNs) in both training (about 3 minutes in an 11-client experiment with one physical Google Pixel 8 Pro and ten simulated clients, instead of 40 minutes on an A100 cloud GPU) and inference (0.16 ms instead of 71 ms on the same smartphone), without compromising estimation accuracy; see Section 4.5.4. (3) We evaluate the privacy-utility tradeoff of FedSER, showing that under the same differentially private dataset release it

Table 4.1: Input/Output Signals (all with x, y, z axes)

Notation	Signal	Source
ACC, VEL	Linear acceleration; angular velocity	Directly measured using wearable IMUs on the pelvis, thigh, shank, and foot (Fig. 4.2a)
JA, JV	Joint angles; joint angular velocities	Motion-capture system, with marker trajectories converted via inverse kinematics to hip, knee, and ankle angles
GRFS	Ground reaction forces	Directly measured by force plates and instrumented treadmills
JMS	Joint moments	Inverse dynamics combining JA, JV, and GRFS to obtain hip and knee moments

retains accuracy comparable to that of non-private TCN and outperforms TCN and KNN across privacy budgets on hip flexion moment; see Section 4.5.5.

4.2 Dataset and Notation

We use a publicly available biomechanics dataset [55]. As described in Table 4.1, the dataset includes aligned timeseries of IMU signals (from wearable sensors placed on several body parts, see Fig. 4.2a), ground reaction forces and joint angles/velocities (from laboratory measurements), and joint moments (derived using inverse dynamics). Movement actions (each treated as a learning task) include cyclic movements (e.g., running, stair ascent/descent, incline/decline walking) and non-cyclic ones (e.g., sit-to-stand, squats, lunges, cutting, jumping, ball toss). We consider 16 shared action categories across 11 participants, grouping individual action and intensity variants into broader tasks (e.g., stair ascent and descent as *stairs*, and walking at 0.6, 1.2, and 1.8 m/s as *walking*). One participant with insufficient overlap in shared actions was removed during preprocessing so that all selected actions are present across the remaining 11 participants; the number of training examples per action varies, as illustrated in Fig. 4.3. Each participant contributes approximately 1,000 training examples in total.

Model Inputs

We use IMU signals (ACC and VEL) as inputs. Here VEL denotes angular velocity from the IMUs (the same physical quantity labeled ANG in Chapter 3); JV denotes joint angu-

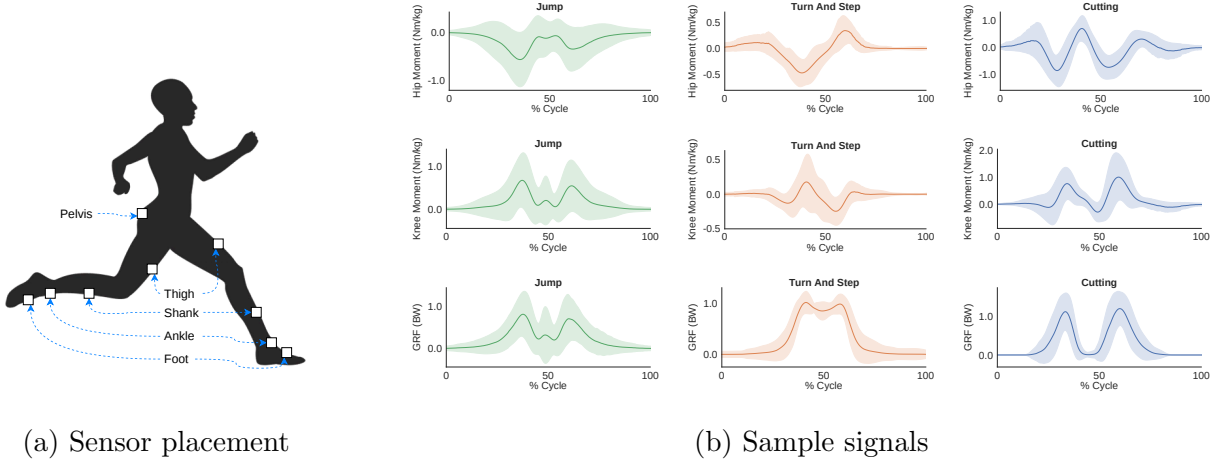


Figure 4.2: Experimental setup and data: (a) placement of wearable IMU sensors measuring input signals; (b) estimation outputs, hip and knee joint moments (JM, top and middle row) and ground reaction force (GRF, bottom row), during different actions (jump, turn and step, and cutting). Lines show the average measured signal, while shaded bands indicate $\pm$ the standard deviation at each time point.

lar velocity from motion capture. We additionally include motion-capture (JA and JV) as inputs to TCNs to compare estimation using only IMUs with approaches that rely on laboratory motion-capture signals [15]. We also report IMU&EMG results in the full modality comparison (Table C.1) to contrast how methods use added sensing channels: TCN tends to improve with almost any additional modality, whereas SER and KNN improve mainly when the added modality is informative—EMG helps less than IMU with motion capture relative to IMU-only. An existing study on this dataset found that EMG did not improve joint moment estimation [122], and the remaining experiments in this chapter use IMU-only inputs.

Model Outputs

The model estimates GRF and JM signals at the hip and knee joints, as defined in Table 4.1. Although FedSER estimates all components of GRFs and JMs, our evaluation focuses on the directions of primary biomechanical relevance, namely vertical GRF and sagittal-plane hip and knee JM. Representative examples of these biomechanical signals are shown in Fig. 4.2b for multiple actions, illustrating distinct waveform characteristics across tasks and outputs.

Data Preprocessing

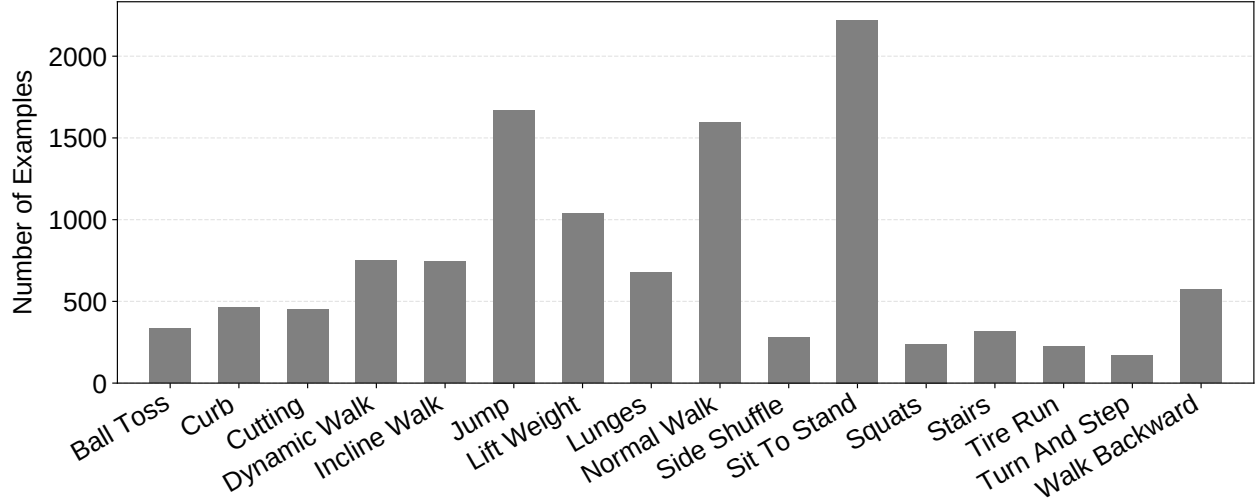


Figure 4.3: Training examples of each action in the dataset

The dataset provides temporally synchronized signals collected at 200 Hz, with trial boundaries defined using dataset timestamps. Since trial durations vary across actions and participants, signals of shorter actions are symmetrically zero-padded to the same temporal length. Padding does not alter the original signal content, and padding boundaries are explicitly tracked so that padded regions are excluded during the evaluation of estimation errors (consistent with prior work using this dataset).

4.3 Problem Setup and Evaluation

We consider a federated multi-task regression problem in which C clients collaboratively learn a model to estimate biomechanical signals (GRFs and JMs) from IMU signals (ACC and VEL). Each client $c \in \{1, \dots, C\}$ holds a private dataset

$$\mathcal{D}_c = \{(\mathbf{x}_i^{(c)}, \mathbf{y}_i^{(c)}, a_i^{(c)})\}_{i=1}^{n_c}$$

where n_c denotes the number of examples available at client c , the input $\mathbf{x}_i^{(c)} \in \mathbb{R}^{d_x}$ represents the IMU signals of an action (a vector concatenation of all components of ACC and VEL signals

over the given time window), the output $\mathbf{y}_i^{(c)} \in \mathbb{R}^{d_y}$ denotes the corresponding biomechanical signals (GRF and JM, represented as a vector concatenation of all of their components), and the label $a_i^{(c)} \in \mathcal{A}$ denotes the action type (e.g., walking, stairs, squatting, lunging, jumping). Each action type corresponds to a distinct regression task (we use the two terms interchangeably) and all clients share identical input and output dimensions, d_x and d_y .

Estimation Objective

The goal is to learn a predictor $f: \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_y}$ that estimates target biomechanical signals $\mathbf{y}$ from wearable sensor inputs $\mathbf{x}$ using data distributed across C clients. To assess accuracy of a prediction $\hat{\mathbf{y}} = f(\mathbf{x})$, we use the root mean squared error $\text{RMSE}(\mathbf{y}, \hat{\mathbf{y}}) = \sqrt{\frac{1}{d_y} \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2}$.

During training, each sample is associated with an action label $a \in \mathcal{A}$, which specifies the underlying movement type. At test time the action label is not available, and test examples may correspond to actions not observed during training. Accuracy is then evaluated across all actions. We also consider a Leave-One-Action-Out (LOAO) setting where training is repeated by leaving each action out of the training set and using it for testing.

Data Scenarios

We consider two data scenarios: in the **PERSONAL** scenario, each client c trains a model using only its local dataset $\mathcal{D}_c$; in the **FEDERATED** scenario, all clients train a shared model together from $\mathcal{D} = \bigcup_{c=1}^C \mathcal{D}_c$ (but without sharing their data). The detailed per-case results reported in the appendix use the equivalent labels **Personal** and **Everyone** for these two scenarios, and additionally report an **Others** case in which a client’s model is trained only on the pooled data of the other clients (excluding its own). To examine robustness to different training set sizes, we randomly subsample the examples within each client-action combination. For each scenario and training set size, hyperparameters are selected using cross-validation; test data are never used for training and hyperparameter selection.

4.4 Proposed Approach

4.4.1 Background on SER

SER [115] learns a linear mapping between low-dimensional SVD spaces of the input and output signals. Let $\mathbf{M}_x \in \mathbb{R}^{n \times d_x}$ and $\mathbf{M}_y \in \mathbb{R}^{n \times d_y}$ be the input and output signal matrices formed by stacking n training examples. Let

$$\mathbf{M}_x \approx \mathbf{U}_x \Sigma_x \mathbf{V}_x^\top \quad \text{and} \quad \mathbf{M}_y \approx \mathbf{U}_y \Sigma_y \mathbf{V}_y^\top$$

denote their truncated SVDs of ranks r_x and r_y , respectively, where rows of the left-singular-vector matrices $\mathbf{U}_x \in \mathbb{R}^{n \times r_x}$ and $\mathbf{U}_y \in \mathbb{R}^{n \times r_y}$ contain the embeddings of the corresponding input and output examples (the rows of $\mathbf{M}_x$ and $\mathbf{M}_y$). SER learns a mapping between $\mathbf{U}_x$ and $\mathbf{U}_y$ using ridge linear regression, i.e., $\mathbf{B}^* = \arg \min_{\mathbf{B} \in \mathbb{R}^{r_x \times r_y}} \|\mathbf{U}_y - \mathbf{U}_x \mathbf{B}\|_F^2 + \lambda \|\mathbf{B}\|_F^2 = (\mathbf{U}_x^\top \mathbf{U}_x + \lambda \mathbf{I})^{-1} \mathbf{U}_x^\top \mathbf{U}_y$, where $\|\cdot\|_F$ is the Frobenius norm, $\lambda \geq 0$ is the regularization parameter, and $\mathbf{B}^*$ is the learned linear mapping from input to output embeddings.

At inference time, a new input signal $\mathbf{x}$ is first projected into the input embedding space using the forward SVD mapping $\mathbf{u}_x = \Sigma_x^{-1} \mathbf{V}_x^\top \mathbf{x}$. The corresponding output embedding is estimated as $\hat{\mathbf{u}}_y = \mathbf{B}^{*\top} \mathbf{u}_x$ and mapped back to the signal domain using the backward SVD reconstruction $\hat{\mathbf{y}} = \mathbf{V}_y \Sigma_y \hat{\mathbf{u}}_y$.

4.4.2 Federated SER

In FedSER, multiple clients learn the SVD and linear mapping of SER without sharing their training data, exchanging only aggregate statistics.

Federated SVD via Gradient Descent and Deflation

We adapt the gradient-based SVD approach of [123] to develop a federated SVD algorithm in which clients iteratively share updates derived from local data. We denote by $\mathbf{M}_c \in \mathbb{R}^{n_c \times d}$ the signal matrix (with either training inputs or outputs as rows) of client $c \in \{1, \dots, C\}$,

where n_c is the number of local training examples and d is the size of input or output examples. Our goal is to find the matrix $\mathbf{V} \in \mathbb{R}^{d \times k}$ of right singular vectors associated with the k largest singular values of the matrix $\mathbf{M} = [\mathbf{M}_1; \dots; \mathbf{M}_C] \in \mathbb{R}^{n \times d}$ with $n = \sum_{c=1}^C n_c$, i.e., where all examples are stacked on rows. To do this in a distributed way (without sharing training data between clients), we consider the matrix

$$\mathbf{A} := \mathbf{M}^\top \mathbf{M} = \sum_{c=1}^C \mathbf{M}_c^\top \mathbf{M}_c \in \mathbb{R}^{d \times d}$$

whose eigenvectors are the right singular vectors of $\mathbf{M}$ and whose eigenvalues are the squared singular values of $\mathbf{M}$ [35]. This decomposition lets each client c compute its update with $\mathbf{M}_c^\top \mathbf{M}_c$ independently, while full matrices $\mathbf{A}$ or $\mathbf{M}$ are never constructed.

Distributed SVD Algorithm. The complete server–client procedure is provided in Algorithm 1. The columns of $\mathbf{V}$ are computed in blocks, in decreasing order of their associated singular values. Starting with $b_0 = 1$, the effective block width is $b_{\text{eff}} = \min(b, k - \ell)$, where ℓ columns have already been recovered, and b is doubled after each block is accepted. The leading singular directions have the largest singular values and capture the most signal variance, so the smaller early blocks receive finer refinement. Later directions have smaller singular values and less influence on the final low-rank representation; processing more of them together reduces the number of sequential convergence stages. The block width therefore trades per-round client computation, memory, and communication against the number of stages required to reach rank k .

The convergence tolerance δ is a tuning parameter in the same tradeoff. We use the fixed value $\delta = 10^{-8}$ in the reported implementation, following [123]. A block-dependent schedule δ_ℓ can instead use a tight tolerance for the leading directions and a more relaxed tolerance for later blocks, whose contribution to the retained variance is smaller.

Initially, $\mathbf{V}$ contains no recovered columns. To compute the next block, the server samples $\mathbf{N} \in \mathbb{R}^{d \times b_{\text{eff}}}$ with $(\mathbf{N})_{ij} \sim \mathcal{N}(0, 1)$ and sets $\mathbf{X}_0 = \text{orth}(\mathbf{N})$. Here, $\text{orth}(\mathbf{Z})$ denotes an

orthonormal basis for the column space of $\mathbf{Z}$; our implementation uses a reduced QR factorization. The Gaussian start has a nonzero projection onto the target eigenspace with probability one, and the usual eigengap assumptions give convergence of the subspace iteration [124]. For the first iteration, $\mathbf{X}_{-1} = \mathbf{X}_0$, so the initial look-ahead is $\mathbf{Y}_0 = \mathbf{X}_0$. For iteration t , the server and clients compute

$$\mathbf{Y}_t = \mathbf{X}_t + \mu(\mathbf{X}_t - \mathbf{X}_{t-1}), \quad (4.1)$$

$$\mathbf{U}_{c,t} = \text{ClientUpdate}(\mathbf{M}_c, \mathbf{Y}_t) = \mathbf{M}_c^\top (\mathbf{M}_c \mathbf{Y}_t), \quad (4.2)$$

$$\mathbf{U}_t = \sum_{c=1}^C \mathbf{U}_{c,t}, \quad (4.3)$$

$$\widehat{\mathbf{X}}_{t+1} = (1 - \eta)\mathbf{Y}_t + \eta\mathbf{U}_t, \quad (4.4)$$

$$\mathbf{X}_{t+1} = \text{orth}\left((\mathbf{I}_d - \mathbf{V}\mathbf{V}^\top)\widehat{\mathbf{X}}_{t+1}\right). \quad (4.5)$$

The Nesterov-style look-ahead $\mathbf{Y}_t$ extrapolates along the previous iterate displacement before the next client query. It is intended to reach a stable block in fewer communication rounds; setting $\mu = 0$ recovers the unaccelerated update. After every update, $\text{orth}(\cdot)$ restores $\mathbf{X}_{t+1}^\top \mathbf{X}_{t+1} = \mathbf{I}_{b_{\text{eff}}}$, controls numerical scaling, and prevents all columns of a block from collapsing onto the single largest-eigenvalue direction. It is computed once at the server because the server already holds the aggregated update and the previously recovered columns in $\mathbf{V}$.

The server stops refining a block when

$$\varepsilon_t^u = \left\| \frac{\mathbf{X}_{t+1}}{\|\mathbf{X}_{t+1}\|_F} - \frac{\mathbf{X}_t}{\|\mathbf{X}_t\|_F} \right\|_F < \delta, \quad (4.6)$$

or when the maximum number of iterations is reached. This criterion measures the normalized change of the basis iterate rather than its magnitude, which is removed by orthogonalization and recovered separately through the singular values. It is evaluated completely at the server and therefore does not require another client operation or communication round. The

maximum iteration count remains a safeguard when the requested tolerance is not reached.

Relation to Gradient Descent and Deflation. For the unaccelerated case ($\mu = 0$), consider the reconstruction loss $\mathcal{L}(\mathbf{X}) = \frac{1}{4}\|\mathbf{A} - \mathbf{X}\mathbf{X}^\top\|_F^2$. Its gradient is $\nabla\mathcal{L}(\mathbf{X}) = (\mathbf{X}\mathbf{X}^\top - \mathbf{A})\mathbf{X}$, which reduces to $\mathbf{X} - \mathbf{A}\mathbf{X}$ when $\mathbf{X}^\top\mathbf{X} = \mathbf{I}_{b_{\text{eff}}}$. The normalized update of [123] divides this gradient by $\|\mathbf{X}\|_F^2 = b_{\text{eff}}$. After absorbing $1/b_{\text{eff}}$ into η , the resulting update is Eq. (4.4), because $\mathbf{U}_t = \sum_c \mathbf{M}_c^\top \mathbf{M}_c \mathbf{Y}_t = \mathbf{A}\mathbf{Y}_t$. The server then applies projection deflation with $\mathbf{P}_\mathbf{V} = \mathbf{V}\mathbf{V}^\top$. The projection $(\mathbf{I}_d - \mathbf{P}_\mathbf{V})$ removes every component in the span of the previously recovered directions before the new block is accepted. When $\mathbf{V}$ contains exact eigenvectors of the symmetric matrix $\mathbf{A}$, this projector commutes with $\mathbf{A}$ and is equivalent to applying the deflated operator on the orthogonal complement [124, Section 4.2], [125, Section 7.5.4]. Keeping deflation and orthogonalization at the server gives all clients the same orthogonal complement and avoids adding another client-side operation.

Single Client Primitive and Matrix-Free Evaluation. Every federated SVD round uses the same client-side primitive in Eq. (4.2). It is one client operation type, although locally it contains the two matrix multiplications $\mathbf{M}_c\mathbf{Z}$ and $\mathbf{M}_c^\top(\mathbf{M}_c\mathbf{Z})$. Evaluating the expression in this order avoids constructing either the local $d \times d$ normal matrix $\mathbf{M}_c^\top\mathbf{M}_c$ or the global matrix $\mathbf{A}$. The same primitive is used with a block iterate during refinement and with $\mathbf{V}$ during the final singular-value calculation, which reduces client code and provides one uniform interface for aggregation and privacy mechanisms. The matrix-free primitive can also be evaluated over row blocks $\{\mathbf{B}_{c,j}\}_j$ of local data,

$$\mathbf{M}_c^\top(\mathbf{M}_c\mathbf{Z}) = \sum_j \mathbf{B}_{c,j}^\top(\mathbf{B}_{c,j}\mathbf{Z}), \quad (4.7)$$

so a client can stream its samples without retaining a normal matrix. When sparse storage and sparse multiplication kernels are used, only nonzero entries are visited, giving $\mathcal{O}(\text{nnz}(\mathbf{M}_c)q)$ computation for a query $\mathbf{Z} \in \mathbb{R}^{d \times q}$. For an iteration block, $q = b_{\text{eff}}$, the additional working memory and communication payload are $\mathcal{O}(db_{\text{eff}})$, and the payload does

not depend on the number of client examples [126, 127].

Singular Values Estimation. After computing $\mathbf{V}$, the singular values of $\mathbf{M}$ are estimated using Rayleigh quotients [124]. For a unit-norm recovered direction $\mathbf{v}_i$,

$$\mathbf{v}_i^\top \mathbf{A} \mathbf{v}_i = \|\mathbf{M} \mathbf{v}_i\|_2^2 \approx \sigma_i^2. \quad (4.8)$$

Therefore, each client computes $\mathbf{H}_c = \mathbf{V}^\top \text{ClientUpdate}(\mathbf{M}_c, \mathbf{V}) = \mathbf{V}^\top \mathbf{M}_c^\top \mathbf{M}_c \mathbf{V}$, and the server obtains

$$\mathbf{s} = \sqrt{\text{diag}\left(\sum_{c=1}^C \mathbf{H}_c\right)}, \quad \mathbf{\Sigma} = \text{diag}(\mathbf{s}). \quad (4.9)$$

The Rayleigh calculation is valid because $\mathbf{A} = \mathbf{M}^\top \mathbf{M}$ is positive semidefinite and its eigenvalues are σ_i^2 . It recovers the singular values with the same client primitive, without reconstructing $\mathbf{M}$, materializing $\mathbf{A}$, or requiring a separate local SVD.

Federated Linear Regression

Each client projects its local signals into the shared SVD embedding space using the transformation bases $(\mathbf{V}_x, \mathbf{\Sigma}_x)$ and $(\mathbf{V}_y, \mathbf{\Sigma}_y)$ obtained through distributed SVD. For client c , this gives $\mathbf{U}_{x,c} = \mathbf{M}_{x,c} \mathbf{V}_x \mathbf{\Sigma}_x^{-1} \in \mathbb{R}^{n_c \times r_x}$ and $\mathbf{U}_{y,c} = \mathbf{M}_{y,c} \mathbf{V}_y \mathbf{\Sigma}_y^{-1} \in \mathbb{R}^{n_c \times r_y}$. The remaining step is to learn the mapping $\mathbf{B}^* \in \mathbb{R}^{r_x \times r_y}$ without sharing the local embeddings of training data. Each client c computes $\mathbf{S}_c = \mathbf{U}_{x,c}^\top \mathbf{U}_{x,c} \in \mathbb{R}^{r_x \times r_x}$ and $\mathbf{P}_c = \mathbf{U}_{x,c}^\top \mathbf{U}_{y,c} \in \mathbb{R}^{r_x \times r_y}$ and sends them to the server. The server forms $\mathbf{S} = \sum_{c=1}^C \mathbf{S}_c$ and $\mathbf{P} = \sum_{c=1}^C \mathbf{P}_c$, then solves $\mathbf{B}^* = (\mathbf{S} + \lambda \mathbf{I})^{-1} \mathbf{P}$ where $\lambda \geq 0$ is an optional ridge regularization parameter (the regression uses no intercept term, which would be zero given that the input and output signals, and their embeddings $\mathbf{U}_{x,c}, \mathbf{U}_{y,c}$, are zero-centered [36]). Each client transmits $\mathcal{O}(r_x^2 + r_x r_y)$ values (independent of its dataset size) and receives $\mathbf{B}^*$ (an $r_x \times r_y$ matrix).

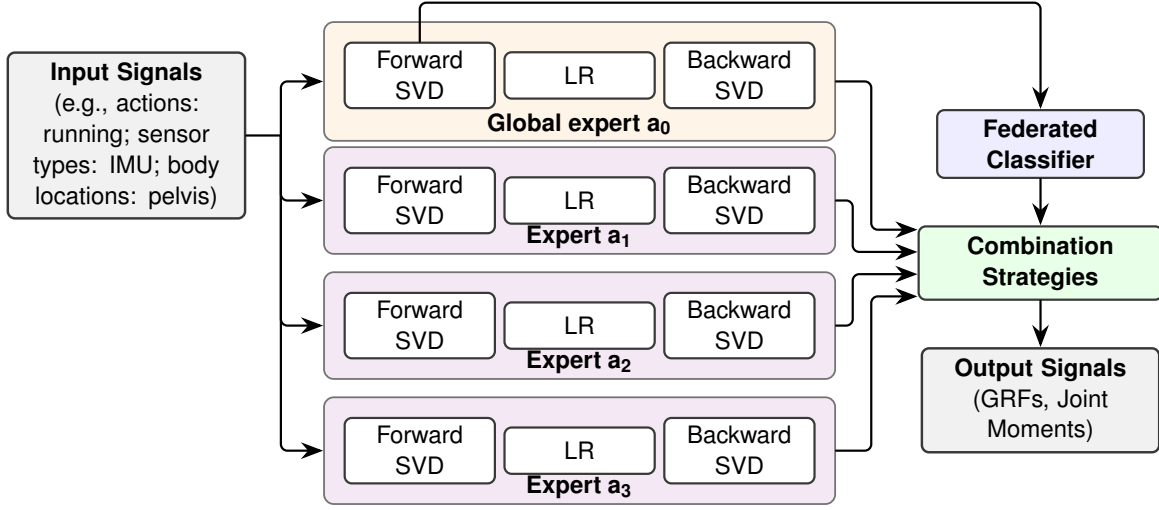


Figure 4.4: Overview of MoE framework

4.4.3 Federated Mixture-of-Experts Framework

The model developed in Section 4.4.2 is a global model for all actions. As illustrated in Fig. 4.4, in addition to this model, our mixture-of-experts (MoE) framework for multi-task estimation also trains (1) an action classification model and (2) one expert model for each action (using the same approach as in Section 4.4.2, but with data of only one action): at inference time, the input is mapped into the global SVD embedding (trained using all actions) and the action classification model is used to assign a probability to each action type; these probabilities are used to combine the outputs of action-specific expert models with that of the global model.

Federated Action Classification

To recognize the action observed in an input example, the federation trains a classifier in the SVD embedding space: given an input $\mathbf{x}$, we compute $\mathbf{u}_x = \Sigma_x^{-1} \mathbf{V}_x^\top \mathbf{x} \in \mathbb{R}^{r_x}$ where $\mathbf{V}_x$ and Σ_x are the right singular vectors and singular values learned during federated SVD (using examples of all actions). The federated classifier $\mathbf{g}: \mathbb{R}^{r_x} \rightarrow \mathbb{R}^{|A|}$ maps the embedding $\mathbf{u}_x$ to a probability score for each action. We implement $\mathbf{g}$ as a two-layer feedforward network

(with weights $\mathbf{W}_1$ and $\mathbf{W}_2$) followed by a softmax layer, i.e., $\mathbf{g}(\mathbf{u}_x) = \text{SoftMax}(\mathbf{z}/\tau)$ where $\mathbf{z} = \mathbf{W}_2\sigma(\mathbf{W}_1\mathbf{u}_x)$ are the logits, $\tau > 0$ is the temperature (optimized on held-out validation data), and σ is a pointwise ReLU. At each round, clients evaluate the gradient of the cross-entropy loss on batches of examples from their local datasets M_c ; the server aggregates these gradients using federated averaging [52].

Expert Combination Strategies

The vector of action probabilities $\mathbf{g}$ is used at inference time to combine the outputs of different experts. We index experts by $a \in \{0\} \cup \mathcal{A}$, where $a = 0$ indicates the global FedSER model and each $a \in \mathcal{A}$ is an action-specific FedSER model; we denote these models by

$$\hat{\mathbf{y}}^{(a)} := f^{(a)}(\mathbf{x}) = \left(\mathbf{V}_y^{(a)}\boldsymbol{\Sigma}_y^{(a)}\right)\mathbf{B}^{(a)\top}\left(\boldsymbol{\Sigma}_x^{(a)-1}\mathbf{V}_x^{(a)\top}\right)\mathbf{x}.$$

In our experiments, while action-specific experts achieve lower error on known actions, routing to a single expert can increase error if the action is mispredicted. The global expert $f^{(0)}(\cdot)$ is trained on all actions and is more robust when the action is uncertain. Given an input example $\mathbf{x}$, let $\mathbf{u}_x = \boldsymbol{\Sigma}_x^{(0)-1}\mathbf{V}_x^{(0)\top}\mathbf{x}$ be its embedding in the global FedSER model. We consider three expert combination strategies:

Best Expert. We use the action-specific expert with the highest probability predicted by $\mathbf{g}(\mathbf{u}_x)$:

$$\hat{\mathbf{y}}_{\text{BE}} = f^{(\hat{a})}(\mathbf{x}), \quad \hat{a} = \arg \max_{a \in \mathcal{A}} (\mathbf{g}(\mathbf{u}_x))_a.$$

Mixture of Experts (MoE). We combine all task-specific experts using classifier probabilities:

$$\hat{\mathbf{y}}_{\text{MoE}} = \sum_{a \in \mathcal{A}} (\mathbf{g}(\mathbf{u}_x))_a f^{(a)}(\mathbf{x}).$$

MoE+Global. To be robust in the presence of uncertain action predictions, we use normalized entropy to assign a weight to the global expert:

$$\lambda(\mathbf{x}) = \frac{-\sum_{a \in \mathcal{A}} (\mathbf{g}(\mathbf{u}_x))_a \log(\mathbf{g}(\mathbf{u}_x))_a}{\log |\mathcal{A}|} \in [0, 1]$$

and

$$\hat{\mathbf{y}}_{\text{MoE+Global}} = \lambda(\mathbf{x}) f^{(0)}(\mathbf{x}) + (1 - \lambda(\mathbf{x})) \hat{\mathbf{y}}_{\text{MoE}}. \quad (4.10)$$

Low entropy favors task-specific experts, while high entropy shifts estimation toward the global expert.

4.4.4 Privacy Mechanism

Beyond keeping raw biomechanical data on local devices and preserving data ownership, we use secure aggregation and differential privacy to limit what the server and other clients can infer.

First, we use Secure Aggregation (SA), which prevents the federated server from seeing any individual client’s update. Each client’s update is masked, and only the summed (combined) update can be recovered. No single client’s model gradients or raw data statistics are exposed during training.

Second, we apply Differential Privacy (DP) to protect any participant from two risks. DP defends against *membership inference*, preventing an adversary from determining whether a particular participant’s data was included in the training set, thereby weakening the link between model behavior and individual records. It also mitigates *model inversion*, making it difficult for an adversary to recover sensitive information such as movement characteristics, performance metrics, or health status of any participant.

With these mechanisms, raw signals stay on the client, individual updates are masked during aggregation, and the released model is harder to invert or use for membership in-

ference. That matters for biomechanical and wearable signals, which can reveal health status, injury risk, movement patterns, or physical condition in clinical, rehabilitation, or performance-monitoring settings.

Threat Model

Our federated approach keeps biomechanical time series from wearables (IMU signals) and biomechanical targets (e.g., GRF, joint moments) on clients. Adversaries, including all other clients in the federation or third-party eavesdroppers, may attempt to: (i) reconstruct sensitive input (partial) signals from model parameters or an individual client update (model inversion), or (ii) determine whether a specific individual’s data participated in training (membership inference). These risks are especially serious for biomechanical data, since movement patterns can reveal health conditions, injury status, or uniquely identifying motion signatures. We assume an honest-but-curious server (it follows the protocol but may try to learn extra information). We make no trust assumption about other clients in the federation (they may all collude with each other). We also protect against external eavesdroppers listening to communication.

Our evaluation uses a differentially private *dataset release*: each trial’s inputs and outputs are clipped and noised before any learning, so the privacy guarantee is at the record (trial) level on the released dataset. When each client is a single participant, this release also limits what membership in that participant’s local data can reveal, including under collusion among the other clients and an honest-but-curious server that inspects protocol messages (DP, not SA alone, covers this stronger setting; see Section 4.5.5).

In our experiments, we have a relatively small number of clients (11 participants). With fewer clients, the privacy–utility tradeoff is naturally tighter, meaning stronger privacy may reduce accuracy more noticeably.

If instead a client represents an institution (e.g., a hospital), the same record-level release protects individual trials within that institution’s data.

Secure Aggregation (SA)

We employ Secure Aggregation to protect client updates during transmission. Under SA, the server only observes the aggregated update across clients and cannot access any individual contribution. This prevents honest-but-curious servers or external eavesdroppers from inspecting per-client gradients or SVD statistics. However, SA alone does not protect against inference attacks on the final trained model.

Differential Privacy (DP)

To protect against model inversion and membership inference attacks on the released model, we incorporate Differential Privacy (DP) mechanisms. DP provides a formal guarantee that the inclusion or exclusion of any single participant has limited influence on the model output.

There are multiple perturbation strategies spanning classical and modern DP learning frameworks [65, 128, 77]:

- **Input perturbation:** noise added directly to client-side features or labels before training [65].
- **Objective perturbation:** noise incorporated into the optimization objective function [128].
- **Gradient perturbation:** noise added to the clipped gradients during the iterative training process (e.g., DP-SGD) [77].
- **Output perturbation:** noise added to the final learned model parameters before release or aggregation [128].

Rather than injecting noise separately into each FedSER pipeline component (the task classifier, the linear regression mapping in the low-rank space, and the federated SVD statistics), which would fragment the privacy budget across stages, we apply DP once through the dataset-level release described next.

Reconstruction Considerations

Recent work shows that exact reconstruction of underlying data from federated SVD updates requires a sufficient number of iterations, typically on the order of d/k (feature dimension divided by rank) or the number of recovered eigenvectors [74]. In our setting, the number of communication rounds remains well below this threshold. Moreover, what is shared are aggregated covariance statistics rather than raw signals. Even in the absence of DP noise, reconstruction would require recovering the full normal matrix before attempting signal inversion. Empirically, reconstructions from these shared statistics do not recover usable biomechanical waveforms.

Comparison to DP-SGD for TCN

As a discarded comparison, we also trained TCN with DP-SGD (clipped noisy gradients) [77]. On our biomechanics data that approach yielded higher RMSE than training TCN on the same shared (ϵ, δ) -DP dataset release used for KNN, TCN, and FedSER in Section 4.5.5, so all reported privacy results use the dataset release.

4.4.5 Differentially Private Dataset Release

We adopt a federated dataset release approach to enable differentially private (DP) learning while maintaining compatibility with both KNN regression and FedSER. KNN inherently requires direct access to training inputs and outputs. To the best of our knowledge, there is no prior work on differentially private KNN regression; existing research primarily focuses on classification, where the label space is public and mechanisms can operate on noisy vote counts or distances [128]. Adapting these techniques for continuous outputs or relying on synthetic data generation introduces severe approximation errors. Therefore, to enable KNN without exposing raw data, we release a DP-protected dataset in a federated manner and perform all downstream learning exclusively on the released data.

This dataset-level release strategy also bypasses the severe privacy budget fragmentation

that would otherwise apply to FedSER. The FedSER pipeline consists of multiple distinct stages (SVD embedding, regression, classification, and gating) operating across both global and action-specific expert models. Because privacy budgets compound, injecting fixed noise at each stage results in an unacceptably loose cumulative privacy guarantee (ϵ). Conversely, strictly bounding the total ϵ requires splitting the budget across stages, which increases the noise added to each component and reduces model accuracy. In addition, allocating a separate privacy budget to each stage consumes additional overall budget under composition across the pipeline. Instead, by applying differential privacy once at the dataset level, we can train the entire pipeline on the released data without further budget depletion.

Let the dataset consist of feature tensors and targets:

$$X \in \mathbb{R}^{N \times M_{\text{input}} \times K}, \quad y \in \mathbb{R}^{N \times M_{\text{output}} \times K}$$

where N is the number of trials, M_{input} is the number of sensor modalities, K is the time dimension, and M_{output} corresponds to targets (e.g., joint moments and ground reaction forces). To bound sensitivity, we apply per-trial ℓ_2 clipping:

$$X_{i,m} \leftarrow X_{i,m} \min \left(1, \frac{C_x[m]}{\|X_{i,m}\|_2} \right), \quad y_{i,m} \leftarrow y_{i,m} \min \left(1, \frac{C_y[m]}{\|y_{i,m}\|_2} \right)$$

The clipping thresholds $C_x[m]$ and $C_y[m]$ are estimated using the 90th percentile of per-trial norms and aggregated across subjects using a DP release of the median across clients. Both the percentile and aggregation strategy (mean, max, min, median) are treated as tunable parameters.

After clipping, we apply the Gaussian mechanism independently to the features and targets:

$$\tilde{X} = X + \mathcal{N}(0, \sigma_x^2 I), \quad \tilde{y} = y + \mathcal{N}(0, \sigma_y^2 I)$$

Table 4.2: RMSE for hip and knee flexion moments and vertical GRF under the LOAO protocol with full training data. Hip and knee moments are in N·m/kg and vertical GRF is in body-weight units (BW). The lowest value per setting and target is bolded.

Input	Method	personal			federated		
		Hip Moment	Knee Moment	Vertical GRF	Hip Moment	Knee Moment	Vertical GRF
IMU + Motion Capture	SER / FedSER (Global)	0.235	0.218	0.238	0.228	0.186	0.229
IMU + Motion Capture	KNN	0.263	0.224	0.223	0.241	0.212	0.209
IMU + Motion Capture	TCN	0.297	0.248	0.325	0.271	0.234	0.298
IMU only	SER / FedSER (Global)	0.292	0.226	0.271	0.266	0.205	0.264

where the noise scale for each release is determined by:

$$\sigma = \frac{\Delta_2 \sqrt{2 \log(1.25/\delta)}}{\epsilon}$$

with Δ_2 the ℓ_2 sensitivity of that release (Δ_X for features and Δ_y for targets). The total privacy budget (ϵ, δ) is divided between the feature and target releases. The resulting dataset $\tilde{D} = (\tilde{X}, \tilde{y})$ satisfies (ϵ, δ) -differential privacy, and any downstream learning procedure applied to $\tilde{D}$ preserves the same guarantee due to the post-processing property of differential privacy.

We release the features and targets independently rather than as a single concatenated vector, since jointly releasing (X, y) results in a combined ℓ_2 sensitivity of $\Delta_{\text{joint}} = \sqrt{\Delta_X^2 + \Delta_y^2}$. Because $\sqrt{\Delta_X^2 + \Delta_y^2} \geq \max(\Delta_X, \Delta_y)$, a joint mechanism forces the noise distribution to be calibrated to a strictly larger sensitivity. This adds unnecessary noise to the lower-sensitivity component, typically the targets in regression problems. Separate Gaussian mechanisms with a split privacy budget avoid this noise inflation while satisfying the overall (ϵ, δ) -differential privacy guarantee [64].

4.5 Results and Discussion

4.5.1 Comparison with Non-Federated Baselines

Non-federated training assumes that all clients share their data to train a global model. Although this assumption is often unrealistic for medical applications due to privacy constraints, non-federated training provides an optimistic reference point on the accuracy of federated learning because it has access to the entire dataset. We use two non-federated baselines:

Temporal Convolutional Networks (TCNs)

TCNs are widely used for regression on timeseries. They capture temporal dependencies with parallelizable convolutions and large receptive fields but require expensive hyperparameter optimization and training. Recent work [15, 13] achieved high accuracy for estimating joint moments using TCNs [30] on the same public dataset [55] used in this study. We select the TCN architecture and training hyperparameters by cross-validation over a grid centered on the configuration of [15], which uses dilated causal convolutions with a kernel size of 4 and a depth of 5 layers of 50 units, giving an effective time history of about 0.93 s at a 200 Hz sampling rate, and use MSE as a loss metric. Our grid sweeps the kernel size over $\{3, 4, 5\}$, the network depth over $\{4, 5, 6\}$ layers, the number of units per layer over $\{32, 50, 64\}$, the dropout rate over $\{0.1, 0.2, 0.3\}$, and the learning rate over $\{10^{-4}, 10^{-3}\}$.

K -Nearest Neighbors (KNN)

KNN estimates the output signals by averaging outputs of the K most similar inputs in the training set under the ℓ_2 distance metric. We select the number of neighbors K by cross-validation over $K \in \{1, \dots, 200\}$. Its accuracy depends on whether the training set contains examples that closely resemble the test examples. Although KNN can be accurate when training data is representative, it requires querying training samples at inference time, increasing privacy risk. Its privacy-preserving variants address this risk but can introduce

substantial accuracy degradation [129, 130]. In addition, its memory usage scales linearly with the dataset size, making it less attractive for large-scale or on-device deployment.

In Table 4.2, we compare a global FedSER expert with SER [115], KNN, and TCN [15] using an LOAO evaluation (we observe similar insight under action-within-training evaluation). In the PERSONAL setting, SER obtains the lowest RMSE for hip and knee moments; KNN obtains lowest RMSE for GRF by only a small margin. Both SER and KNN have lower RMSE than TCN on all three target metrics. The personal TCN results reported in [15] are 0.280 for hip and 0.240 for knee moment; although these results were evaluated on a different subset of the same dataset, they are closely aligned with our TCN results. Overall, these results show that SER is competitive with strong baselines for estimating joint moments and GRFs in this setting.

In the FEDERATED setting (Table 4.2), all methods benefit from using data of all participants, achieving lower RMSE. The global model of FedSER (trained on all actions) remains the best on hip and knee moments, while KNN remains the best on GRF. We later show in Fig. 4.7b, FedSER substantially improves under the MoE framework and outperforms KNN in GRF. The centralized SER achieves the same RMSE for all of our experiments as the global FedSER model (this result was expected, since federated SVD converges to the same decomposition as centralized SVD in our experiments). Notably, using only IMU inputs (which can be collected using wearable sensors), FedSER outperforms TCN trained with both IMU and motion capture inputs. When TCN is restricted to the same IMU-only inputs in the FEDERATED setting, its LOAO RMSE further increases (hip: 0.313, knee: 0.256, GRF: 0.333; Table C.1). Similarly, restricting KNN to IMU-only inputs increases its LOAO RMSE (hip: 0.301, knee: 0.243, GRF: 0.248). To enable estimation of JMs and GRFs in the wild, where motion capture is unavailable, the remaining experiments use only IMU inputs.

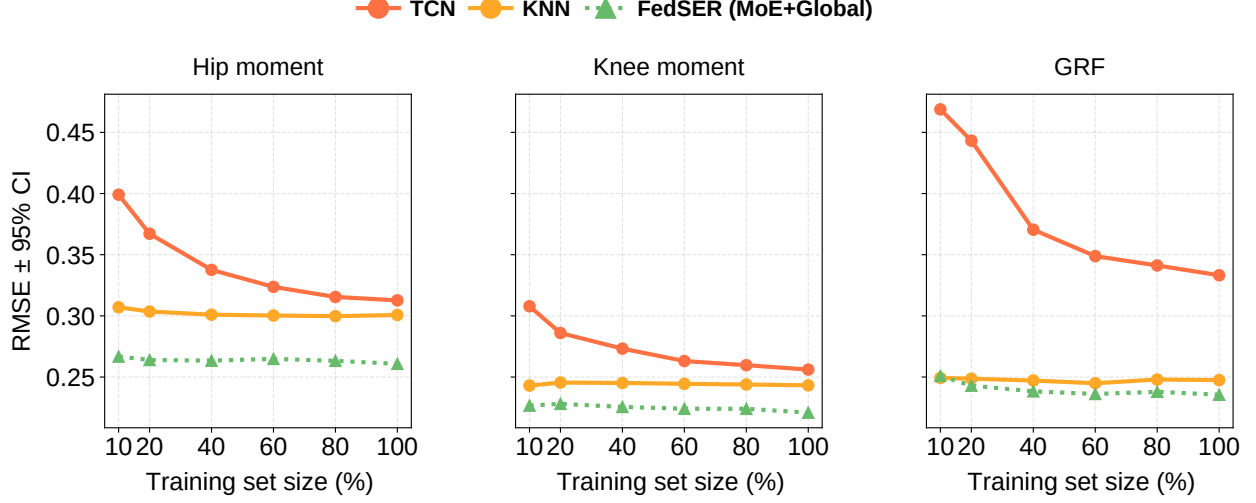


Figure 4.5: RMSE across training set sizes under IMU-only inputs, LOAO, and FEDERATED setting, where the training set size is varied by retaining varying percentages of each client’s training samples.

4.5.2 Robustness to Smaller Training Set Sizes

In Fig. 4.5, we report RMSE of FedSER (MoE+Global), TCN, and KNN when varying the amount of training data of each participant from 10% to 100% and using IMU sensors as input, the LOAO protocol, and FEDERATED training. The results show that FedSER outperforms both KNN and TCN estimating all targets for every dataset size. Note that reducing the dataset size has a smaller effect on SER and KNN than on TCN, which requires more training data.

Fig. 4.6 reports the average RMSE improvement of FedSER (MoE+Global) over KNN and TCN, with 95% confidence intervals (CIs). The average improvement is calculated from the difference of per-action mean RMSE between each baseline and the MoE+Global model (each per-action mean averages over subjects, experimental runs, and speed/intensity variants within an action), then averaged across the 16 actions. The CI is a bootstrap interval over these per-action improvements [131]. We consider action-wise RMSE differences to preserve the action-level comparison between methods. The bootstrap CI is computed over these action-wise differences, estimating uncertainty from their empirical distribution

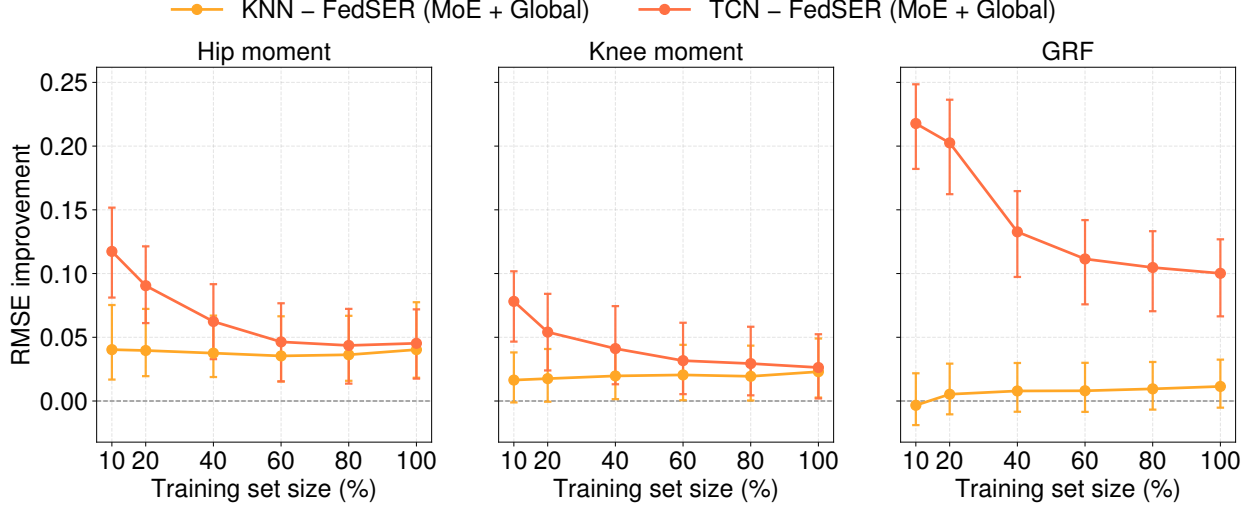


Figure 4.6: RMSE improvement of FedSER (MoE+Global) over KNN and TCN across training set sizes under IMU-only inputs, LOAO, and the FEDERATED setting. Positive values indicate lower RMSE for FedSER; error bars show 95% confidence intervals. We retain confidence intervals only in this figure as a representative example; the CI methodology and additional analyses appear in Appendix D.1.

without assuming that the differences are Gaussian [132, Chapter 12]. The improvement over TCN (orange), $\text{RMSE}_{\text{TCN}} - \text{RMSE}_{\text{MoE+Global}}$, is positive for every training set size and target, and larger when less training data are available. This shows FedSER is clearly more accurate than TCN, particularly when training data are scarce. The improvement over KNN (yellow), $\text{RMSE}_{\text{KNN}} - \text{RMSE}_{\text{MoE+Global}}$, is smaller and near zero on GRF, where KNN remains competitive (but generally KNN requires participants to share their data). Note that average RMSE shows these relationships, and the CIs confirm that they typically hold across experimental runs; detailed CI results with the same insights for all experiments appear in Appendix D.1.

4.5.3 Expert Combination Strategies

In Fig. 4.7, we compare expert combination strategies in FedSER using action-within-training and LOAO evaluations, with KNN as a baseline (TCN is omitted because its RMSE is consistently higher than FedSER in both settings, e.g., see Fig. 4.5 for LOAO). In both scenarios, the MoE+Global strategy of Eq. (4.10) achieves the most significant improvements

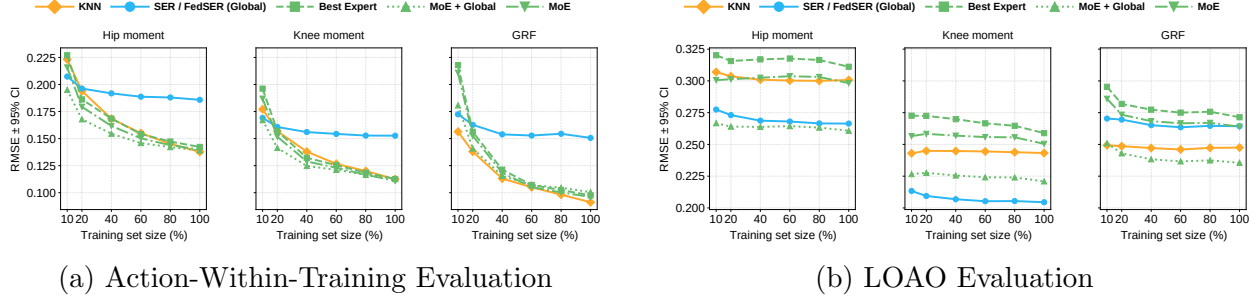


Figure 4.7: RMSE across training set sizes for FedSER expert combination strategies and KNN, using only IMU inputs, in the FEDERATED setting. FedSER variants include the global expert, best expert, MoE, and MoE+Global.

over FedSER using a single global expert. When the test action is included in training (Fig. 4.7a), the MoE+Global is also better or comparable to KNN; when using an LOAO evaluation (Fig. 4.7b), the MoE+Global strategy achieves the lowest RMSE for all training set sizes and all estimation targets except for knee moment, where FedSER using a global expert has slightly lower RMSE.

Under LOAO, FedSER (including variants) appears less sensitive to data scarcity because no action-specific expert is fitted to an unseen action, so FedSER falls back on the global model; as expected, the all-action model outperforms the single-action experts, and by pooling every action’s data the global model is far less sensitive to reduced training data. When the action is in training, FedSER instead relies on its action-specific expert, which is as data-dependent as KNN, so at 10% its small per-action slice is too sparse to be reliable and the global model does better.

4.5.4 Computational Demand

We characterize the computational demand of FedSER training and inference using both cloud and edge-device measurements. Cloud experiments use commodity CPUs and A100 GPUs [133], while edge-device experiments use a *Google Pixel 8 Pro* [134]. Inference latency is measured on the edge device for all methods that are practical to deploy (SER, FedSER, and TCN), while edge-device training is evaluated only for FedSER. In many applications of

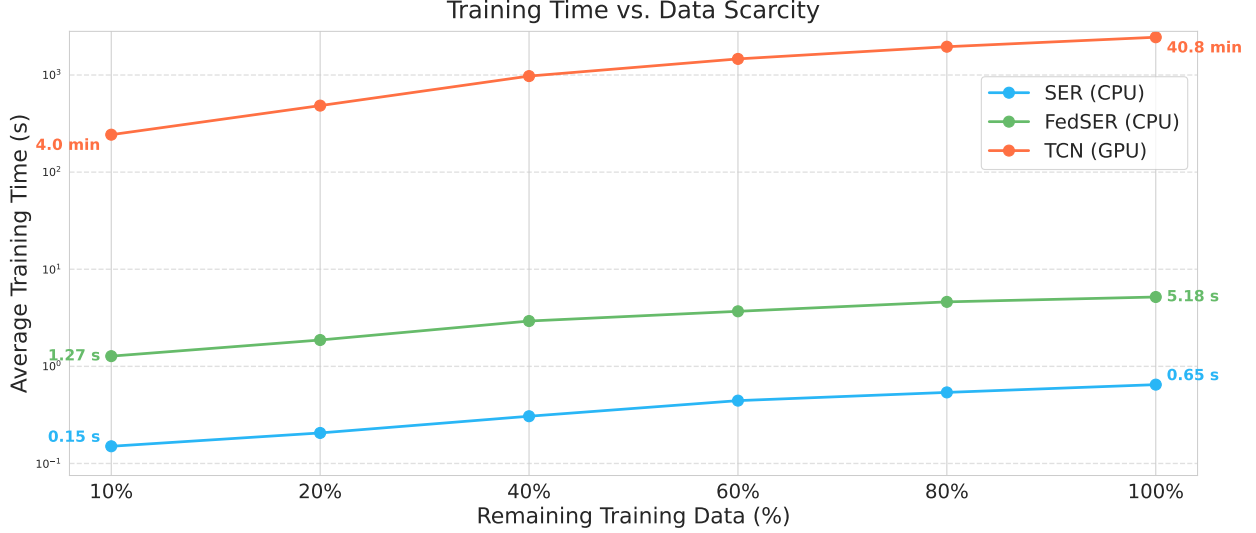


Figure 4.8: Average training time versus the percentage of training data retained for SER, FedSER, and TCN. The vertical axis is logarithmic. SER and FedSER are measured on CPUs; TCN is a centralized A100-GPU measurement and therefore provides an optimistic lower bound for federated TCN training.

interest, wearable sensors would stream IMU measurements to the smartphone of a participant. While participants can upload their data to a remote system, they can also participate in federated training directly from their smartphones, keeping sensor data local and sending model updates to the server for aggregation. For this type of applications, we report training time, per-client communication cost, and on-device inference latency. Centralized TCN training times on A100 GPUs are used as optimistic lower bounds for federated TCN training times, since training on CPU or edge devices was too slow.

Training Time. Using all available training data, training one TCN model takes 40.8 minutes on an A100 GPU, whereas FedSER training takes only 5.18 s on a CPU (about 25% of this time is spent training the federated task classifier). SER training is even faster, requiring only 0.65 s on a CPU. Thus, even under an optimistic centralized GPU setting for TCN, FedSER is around two or more orders of magnitude faster. This gap persists with smaller training set sizes. Figure 4.8 plots training time from 10% to 100% of each client’s training samples. At 10% of the training data, SER requires 0.15 s, FedSER requires 1.27 s, and TCN requires 4.0 minutes; with the full training set, the times are 0.65 s, 5.18 s, and

40.8 minutes. TCN therefore takes approximately 190 times longer than FedSER at 10% and 470 times longer at 100%. Although FedSER includes federation and task-classifier overhead relative to SER, its complete training time remains in seconds across all training-set sizes. The comparison favors TCN: its time is for centralized training on an A100 GPU, whereas FedSER uses CPUs, and a federated TCN would also exchange model updates repeatedly. The figure is also for training a single model; it does not include TCN hyperparameter search over kernel size, dilation factors, residual blocks, and related choices, whereas FedSER has few tuning parameters.

Federated Training on Edge Devices. We evaluate federated training on edge devices using one physical Google Pixel 8 Pro client communicating over 50 Mbps Wi-Fi with 10 clients simulated using a compute node. Across 10 runs, each client holds around 190 MB of training data, corresponding to a total of ~ 2.1 GB across eleven clients. Training the full FedSER model requires 97 s of computation and 89 s for data transfer over Wi-Fi, for a total active time of about 3 minutes, with a peak RAM usage of 372 MB. The edge-device implementation applying default optimizations and quantization shows no RMSE difference from the compute-node implementation.

Communication Cost. We estimate communication cost using the full training-set size by simulating each participant as a separate client on a single host and measuring the amount of data that would be transferred between each client and the server if the clients had sufficient local compute and storage to run the corresponding method. In this case, FedSER requires a per-client communication cost of 589 MB, consisting of 288 MB upload and 301 MB download. In contrast, federated KNN requires 2.14 GB per client because each client must retrieve samples from all other clients, while TCN requires approximately 1 GB per client over 100 epochs due to repeated model-update (gradient) exchange. Here, 100 epochs is the smallest setting at which TCN reached the target accuracy across the evaluated settings.

Table 4.3: Per-sample mobile inference time on a Pixel 8 Pro with optimizations and quantization, averaged over 100 runs after ten warm-up inferences.

Model	Size (MB)	Peak RAM (MB)	CPU (ms)	GPU (ms)
SER	0.146	0.149	0.126	0.116
FedSER	0.173	0.177	0.155	0.143
TCN	4.64	21.1	71.2	17.2

Inference on Edge Devices. The Google Pixel 8 Pro provides multiple inference accelerators, including 4 CPU cores, a GPU, and a neural network accelerator via NNAPI. To determine the most efficient execution strategy, we evaluated on-device inference using TensorFlow Lite (TFLite). Table 4.3 reports model size, peak runtime memory, and latency on the GPU and CPU, with CPU execution using four XNNPACK threads. We omit KNN from our inference evaluation because retaining the full training dataset (~ 2.1 GB) is impractical on a smartphone. NNAPI is also omitted due to unsupported operators and because it runs the TCN model significantly slower than using the CPU (208 ms vs. 71.2 ms).

4.5.5 Privacy Perspective

FedSER supports multiple privacy protections, each with different trust assumptions as well as computational and accuracy costs. We consider a federation of a total of C clients, including a target client c whose data we protect. We assume an *honest-but-curious* server that follows the protocol but inspects every message it receives, and we place no trust in the other $C - 1$ clients, which may collude with one another or with the server [57]. The adversary’s goal escalates in severity: it may learn whether c ’s data was used to train the model (*membership inference*) [60]; for a clinical cohort, identifying c as part of that patient group can expose them to insurance or employment discrimination [50]; infer c ’s signal attributes that indicate injury risk without accessing the raw signals (*property inference*) [58, 10]; or reconstruct c ’s signals outright (*model inversion*) [59].

In FedSER, participants keep their training data locally and share only model statis-

tics [52, 57], but this is not a formal privacy guarantee on its own: each client’s individual update is still visible to the server. From these updates, an honest-but-curious server could attempt to reconstruct the statistics of c ’s signals (their covariance), posing a model-inversion risk for federated SVD [74].

Secure aggregation (SA) [61] hides these individual updates: the honest-but-curious server observes only the aggregate over the C clients, so acting alone it cannot reconstruct any individual client’s data. SA does not change model accuracy, but it adds communication and computation overhead [63]. Partially homomorphic encryption (PHE) [62] is compatible with the same aggregation pattern in principle (the server computes over encrypted updates), but we do not separately time a PHE deployment here. The risk remains: if the other $C - 1$ clients collude, they can subtract their own contributions from an SA aggregate to recover client c ’s update [57], and SA alone does not protect against this worst-case collusion.

Differential privacy (DP) bounds how much the released data, and any model trained on that data, can reveal about clients’ membership and the properties of their data [64], even when the other $C - 1$ clients collude together with the server. To evaluate the effects of DP, we clip each client’s local time-series, including both inputs and outputs, add Gaussian noise, and release the result as a privatized (ϵ, δ) -DP dataset that KNN, TCN, and FedSER all train on [135]. Clipping caps each client’s data contribution by scaling down those whose ℓ_2 norm exceeds a threshold, set to the median across clients of each client’s p -th percentile norm (with $p = 90$ chosen from our empirical RMSE results), thereby bounding its sensitivity [77, 65]. Because no later computation on a DP release can weaken its guarantee (the post-processing property [64, Proposition 2.1]), this single release covers the entire FedSER pipeline, so we do not split a privacy budget across its stages (SVD, regression, and the MoE framework). Prior work finds simple models with linear mappings (like FedSER) more accurate under DP than deep networks trained with DP-SGD [76, 77]. Empirically, this DP data release also gave us lower RMSEs than applying DP-SGD for TCN during training. The threat model and DP dataset-release protocol are stated in Section 4.4.4.

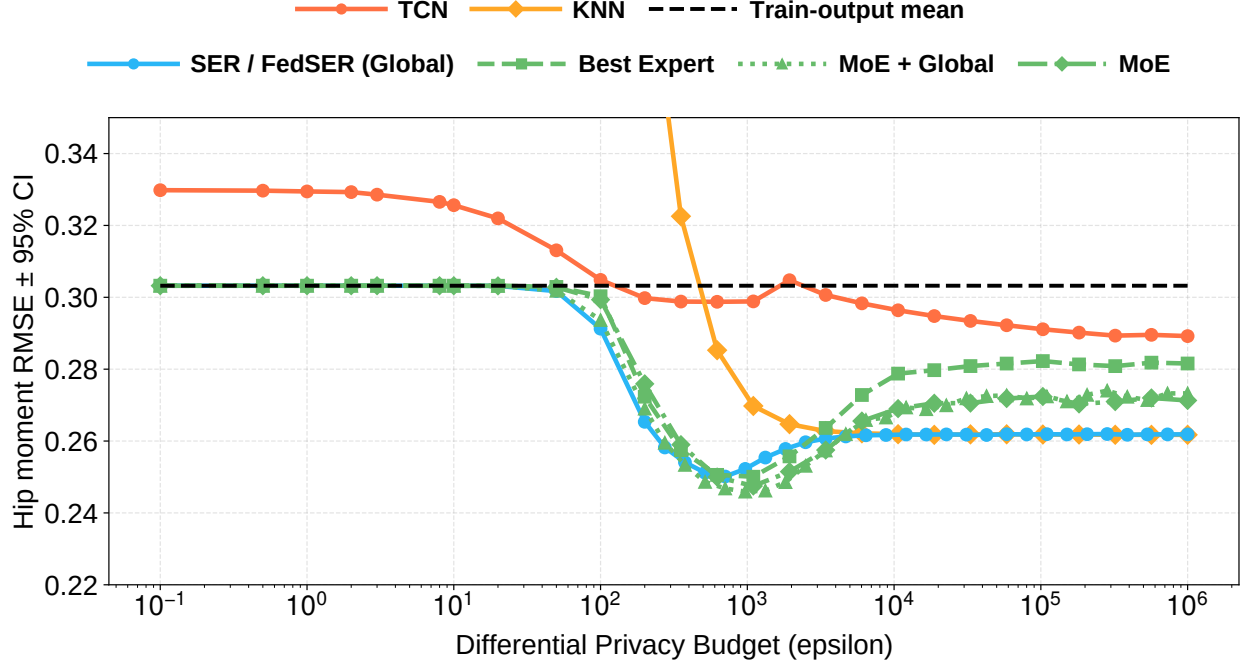


Figure 4.9: Hip flexion moment RMSE under DP in the IMU-only, FEDERATED, LOAO setting with full training data. RMSE is plotted against the privacy budget ϵ for a fixed δ under the Gaussian mechanism of Section 4.4.4, with ϵ split between feature and target releases; larger ϵ gives a weaker privacy guarantee (less injected noise).

Fig. 4.9 depicts the estimation RMSE for the hip flexion moment across privacy budgets (other targets and training set sizes are qualitatively similar). KNN is the most sensitive to noise: at smaller ϵ values (stronger privacy guarantees), its RMSE grows beyond the plotted range. As ϵ increases, all methods return to their non-private accuracy. Under stricter privacy budgets, the RMSE of TCN and FedSER eventually plateaus ($\epsilon < 10$) as the added noise dominates the useful signal, and both models essentially predict an average of the training outputs (we measure this mean at 0.305 N·m/kg, close to the FedSER plateau). Because TCN does not explicitly mitigate the injected noise, its predictions can still carry it, so its plateau RMSE is higher than FedSER’s. On the other hand, FedSER naturally denoises; even at the strictest privacy budget (smallest ϵ), the RMSE of FedSER remains comparable to that of the non-private TCN baseline. Note, this comparison is an average across all actions: under LOAO, TCN’s reported accuracy is much more dependent on the dataset’s action distribution (we observe TCN performs well on some actions and poorly on

others), so the average can change depending on how those actions are represented. The curves are not strictly monotone: moderate noise can slightly reduce RMSE, possibly due to noise acting as implicit regularization for the low-rank linear model, especially for FedSER. Across budgets, FedSER with MoE+Global has the lowest RMSE, matching its advantage in the non-private setting.

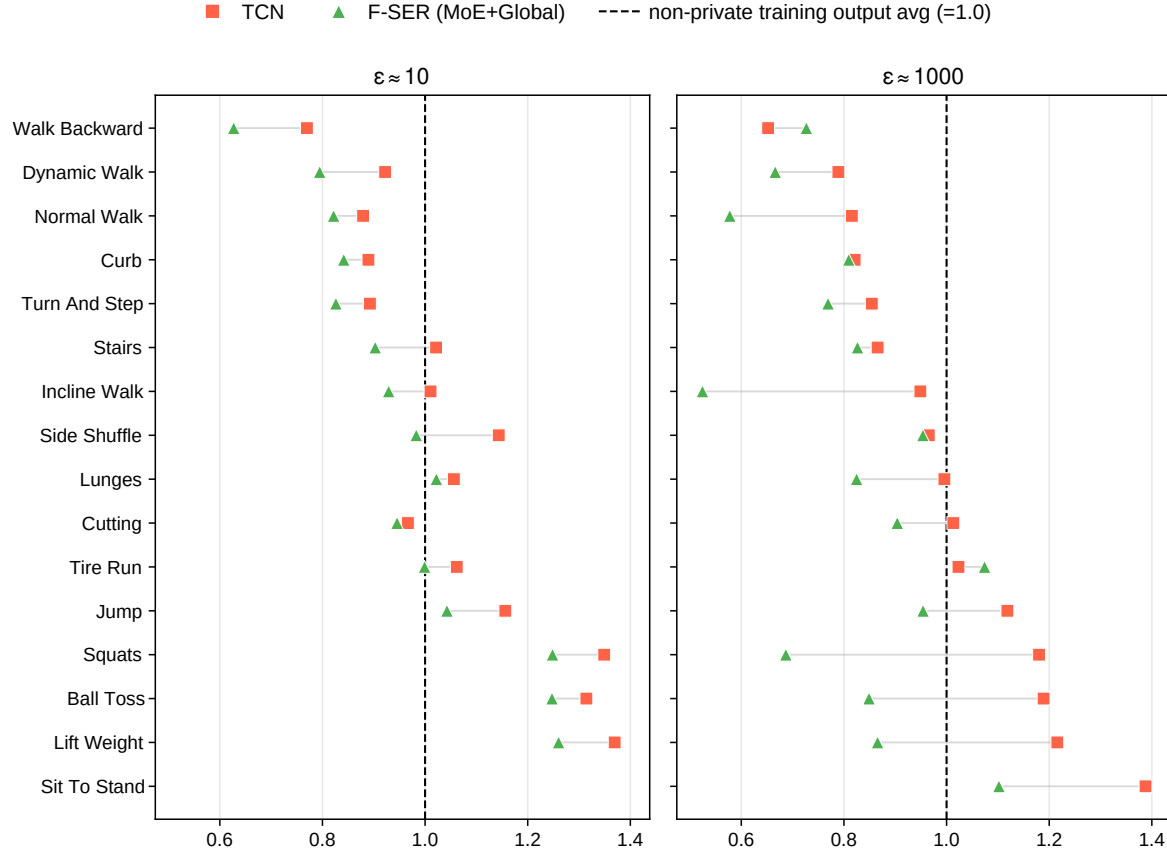


Figure 4.10: Per-action hip-moment RMSE for TCN (squares) and FedSER MoE+Global (triangles) under IMU-only, FEDERATED, LOAO evaluation. Values are normalized by the non-private training-output mean for each action, shown by the dashed line at 1. The two panels use privacy budgets $\epsilon \approx 10$ and $\epsilon \approx 1000$ (fixed δ ; ϵ split between feature and target releases as in Section 4.4.4).

Figure 4.10 separates the aggregate curves above by held-out action under the same privacy budgets. Near $\epsilon = 10$, most values cluster around the training-output mean, but the amount and direction of the error vary by action. Near $\epsilon = 1000$, the actions that are difficult for TCN are not always the same as those that are difficult for FedSER. The figure therefore does not support a claim that one method is uniformly better for every action.

The RMSE increase at small ϵ is expected: for strong privacy $\epsilon \leq 0.5$ [64], prior work reports comparable loss unless the data is far larger, on the order of 10^3 clients [136] or with much more training data ($\sim 10\times$) [76], so it reflects the scale of the 11-subject dataset rather than a limitation of FedSER. Reaching a larger population is where FedSER helps, as its lightweight computation combined with these privacy mechanisms makes larger-scale studies inexpensive and deployable on a phone.

4.5.6 Missing Input Channels and Projection-Based Recovery

Individual IMU channels may be missing or corrupted at inference time. As a proxy, we randomly drop individual channels (not whole sensors) at test time. Models are trained with all 24 IMU channels: three acceleration and three angular-velocity channels at each of four sensor locations. For each test sample, we set a random subset of channels to zero and compare the original FedSER prediction with a recovered-input prediction. We report results for hip flexion moment as an example.

Let Ω denote the observed input coordinates and let $\mathbf{V}_x$ contain the learned global input basis. Starting from a zero-filled input $\mathbf{x}^{(0)}$, each recovery iteration computes

$$\tilde{\mathbf{x}}^{(t+1)} = \mathbf{V}_x \mathbf{V}_x^T \mathbf{x}^{(t)}. \quad (4.11)$$

We then restore the observed coordinates and retain the projected values only for the missing

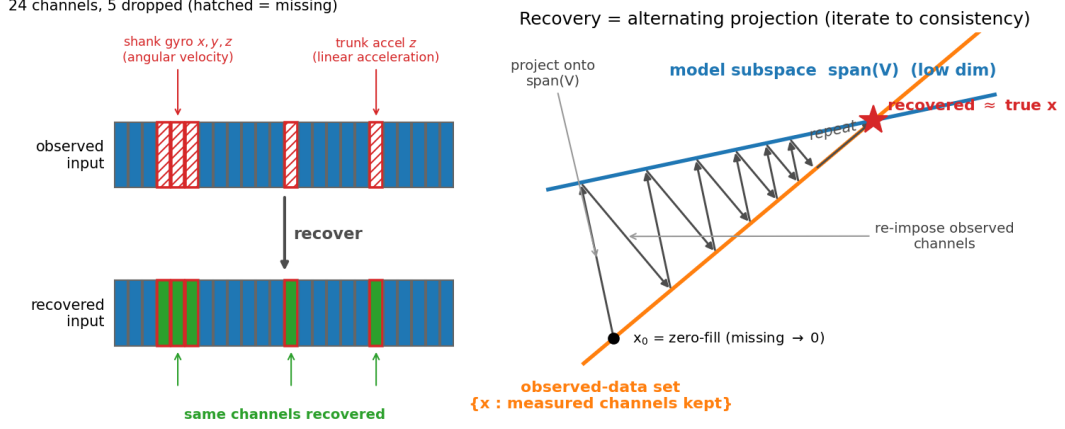


Figure 4.11: Missing input channels and projection-based recovery before FedSER estimation.

Table 4.4: Average hip-moment RMSE reduction from projection-based recovery relative to zero-filled FedSER inputs. Positive values indicate lower RMSE after recovery.

Avg. RMSE reduction	Action in training	LOAO
SER / FedSER (Global)	+7.3%	−2.6%
FedSER (MoE+Global)	+14.1%	−1.2%

coordinates:

$$x_j^{(t+1)} = \begin{cases} x_j^{\text{obs}}, & j \in \Omega, \\ \tilde{x}_j^{(t+1)}, & j \notin \Omega. \end{cases} \quad (4.12)$$

We select the iteration limit using validation output RMSE and apply the unchanged FedSER gating procedure to the recovered input. The method is iterative projection onto a learned input subspace (some figures label this “CS,” but it is not sparse compressed-sensing recovery).

Figure 4.12 and Table 4.4 summarize the hip-moment results. When the test action is included in training, recovery reduces average RMSE by 7.3% for the global expert and 14.1% for MoE+Global. Under LOAO, recovery is approximately neutral: RMSE changes by −2.6% for the global expert and −1.2% for MoE+Global. The learned input subspace cannot introduce action-specific structure that was absent from training.

Recovery error reaches a minimum after a small number of iterations and then increases

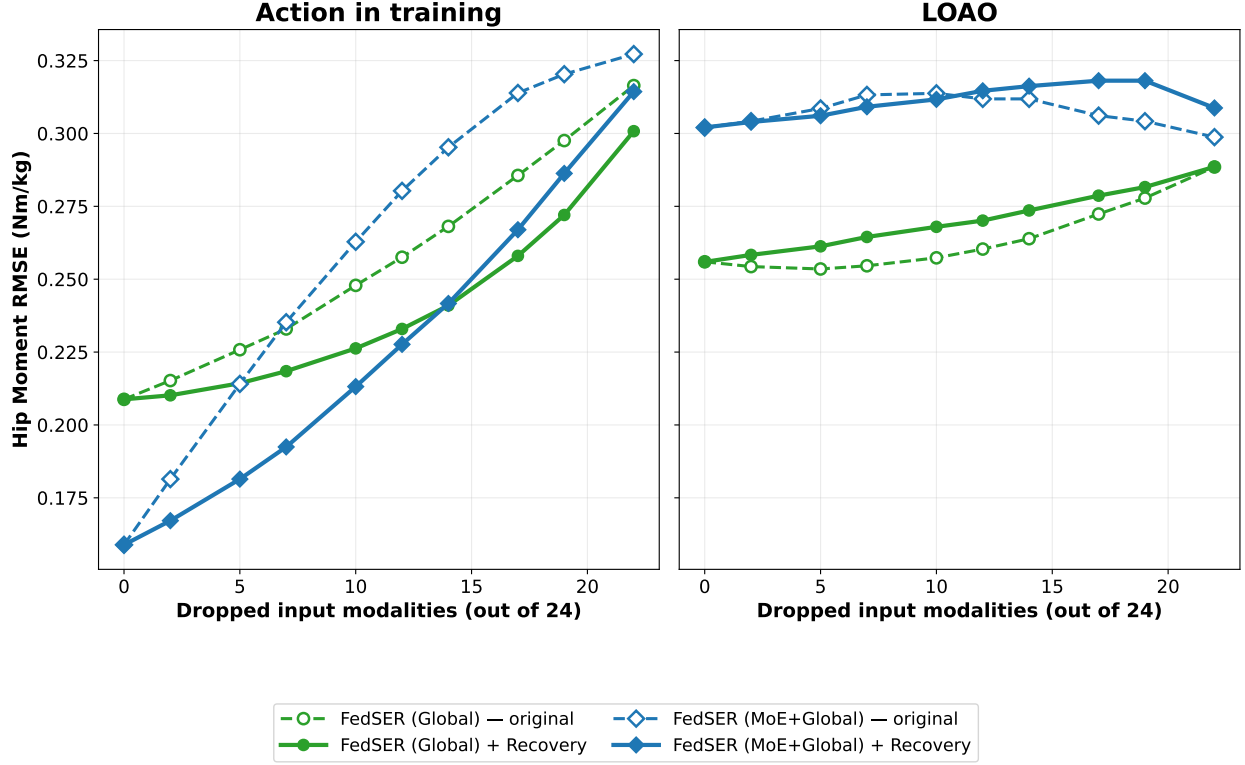


Figure 4.12: Hip-moment RMSE versus number of randomly dropped IMU channels, with and without projection-based recovery, for action-within-training (left) and LOAO (right) evaluation.

because repeated projection moves the test input toward the training subspace. We therefore select the iteration count using validation output RMSE rather than running the procedure to convergence; the iteration minimizing input reconstruction error need not minimize downstream estimation error.

4.6 Conclusions

We presented FedSER, a federated multi-task framework for estimating biomechanical signals from IMU data collected by wearable sensors. FedSER allows participants to train collaboratively without sharing training data. It matches centralized SER accuracy across biomechanical targets and data settings, and enables training directly on edge devices. Empirically, FedSER outperforms TCN in estimating hip and knee flexion moments and vertical

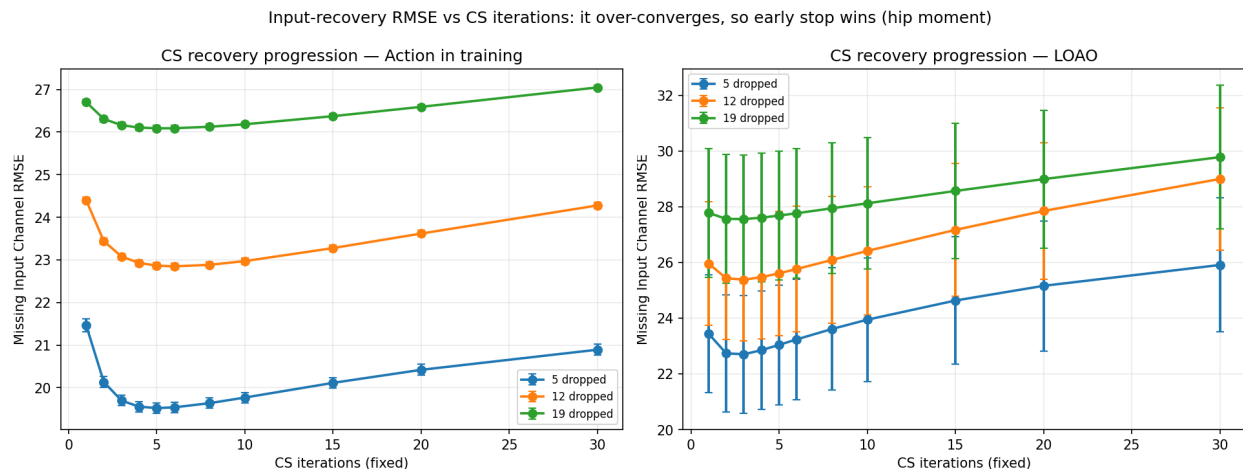


Figure 4.13: Input-recovery and downstream output RMSE versus the number of projection iterations.

ground reaction forces across all training set sizes, including both action-within-training and LOAO scenarios.

FedSER trains directly on the phones in minutes, whereas the centralized TCN requires a server-grade GPU and cannot be trained on-device at all. FedSER is also compatible with secure aggregation, which hides individual client updates at no accuracy cost, and with partially homomorphic encryption in principle (not separately timed here); it retains accuracy comparable to that of non-private TCN under differential privacy. FedSER is a practical improvement over deep models when labeled data are limited or not allowed to pool on a central server.

The low training cost of FedSER also enables on-the-fly retraining with newly collected data; we leave a detailed study of such on-device continual adaptation to future work. For hip flexion moment as an example, when individual IMU channels are missing at test time, projection onto the learned input subspace reduces RMSE if the test action was seen in training, and is about neutral under LOAO (Section 4.5.6).

The next chapter turns to a systems question that often faces the same kinds of places: when clinics, research labs, and similar organizations form a federation of hybrid clouds, when is sharing resources profitable under quality-of-service constraints (Chapter 5)?

Chapter 5

Performance and Revenue Analysis of Hybrid Cloud Federations With QoS Requirements

5.1 Introduction

The previous chapters showed how participants can train together without sharing raw biomechanical data. Clinics, research labs, and sports performance centers often need that kind of data locality, and they may also want to share computers when demand is high, without becoming one organization or sending every overflow job to a public cloud. This chapter studies that resource-sharing problem for federations of hybrid clouds. Background is in Chapter 2.

Since the introduction of public cloud services in the mid-2000s, cloud computing has radically transformed the IT industry by removing the need for upfront commitments to acquire hardware resources and expertise to operate them. In the *Infrastructure-as-a-Service* (IaaS) market, *public cloud providers* (Amazon AWS, Google Cloud, Microsoft Azure) offer a multitude of hardware resources (CPUs, GPUs, FPGAs, machine learning accelerators) remotely accessible through virtual machines (VMs). Similarly to other utilities, application developers can pay for these resources by usage time (e.g., CPU cores paid by the second) and quickly allocate them on demand to meet changes in their workloads, alleviating the risk of underprovisioning or overprovisioning to satisfy quality of service (QoS) requirements [78].

Such flexibility has had a profound impact on the design and operation of *private clouds* and data centers of large organizations: instead of acquiring enough hardware resources to satisfy QoS requirements during predicted peaks of their workloads, private clouds can *forward part of the workload to public clouds when needed*. This cloud architecture, called

hybrid cloud [82, 83], is also common for organizations that decide to avoid investing in on-premise data centers altogether and choose instead to commit to a certain amount of cloud resources for fixed periods of time, obtaining discounts with respect to on-demand prices; for example, Amazon AWS *reserved instances* offer up to 72% discounts on VM instances in case of 1-year or 3-year commitments. Given the difficulty of workload prediction, organizations can reserve an underprovisioned pool of resources at a discounted price, allocating more resources on-demand during peak loads. Software platforms for private clouds, such as OpenStack [79], Apache CloudStack [80], and OpenNebula [81], facilitate these hybrid architectures by implementing APIs compatible with those of public clouds.

In addition to hybrid architectures, *cloud federations* [84, 85] provide another appealing strategy to operate under uncertain and variable workloads: private clouds agree to *share part of their computing resources* to serve requests from other members of the federation. In so doing, underprovisioned private clouds can satisfy their QoS requirements during peak loads, while overprovisioned private clouds can put their resources to work instead of leaving them idle. Different policies have been investigated to reward members of the federation that provide resources or workload [86, 87]: at one extreme, business operations are shared entirely (e.g., members of the federation become one business organization) and resource sharing is optimized to maximize the total profit, which is then distributed according to solution concepts for cooperative games (e.g., Shapley value); at the other extreme, members are non-cooperative players maximizing their profit by sharing part of their resources or workload, while the federation is self-enforced by a pricing mechanism used for its services. Resources can be shared with the entire federation [86] or with individual members [87], and resource owners may have priority over the use of shared resources [86].

This chapter focuses on IaaS cloud architectures including both hybrid clouds and cloud federations. As illustrated in Fig. 5.1, when local queues are too large to satisfy QoS requirements (in our setting, the maximum mean waiting times allowed by service level agreements), private clouds forward user requests for VM instances (i.e., their *overflow traffic*)

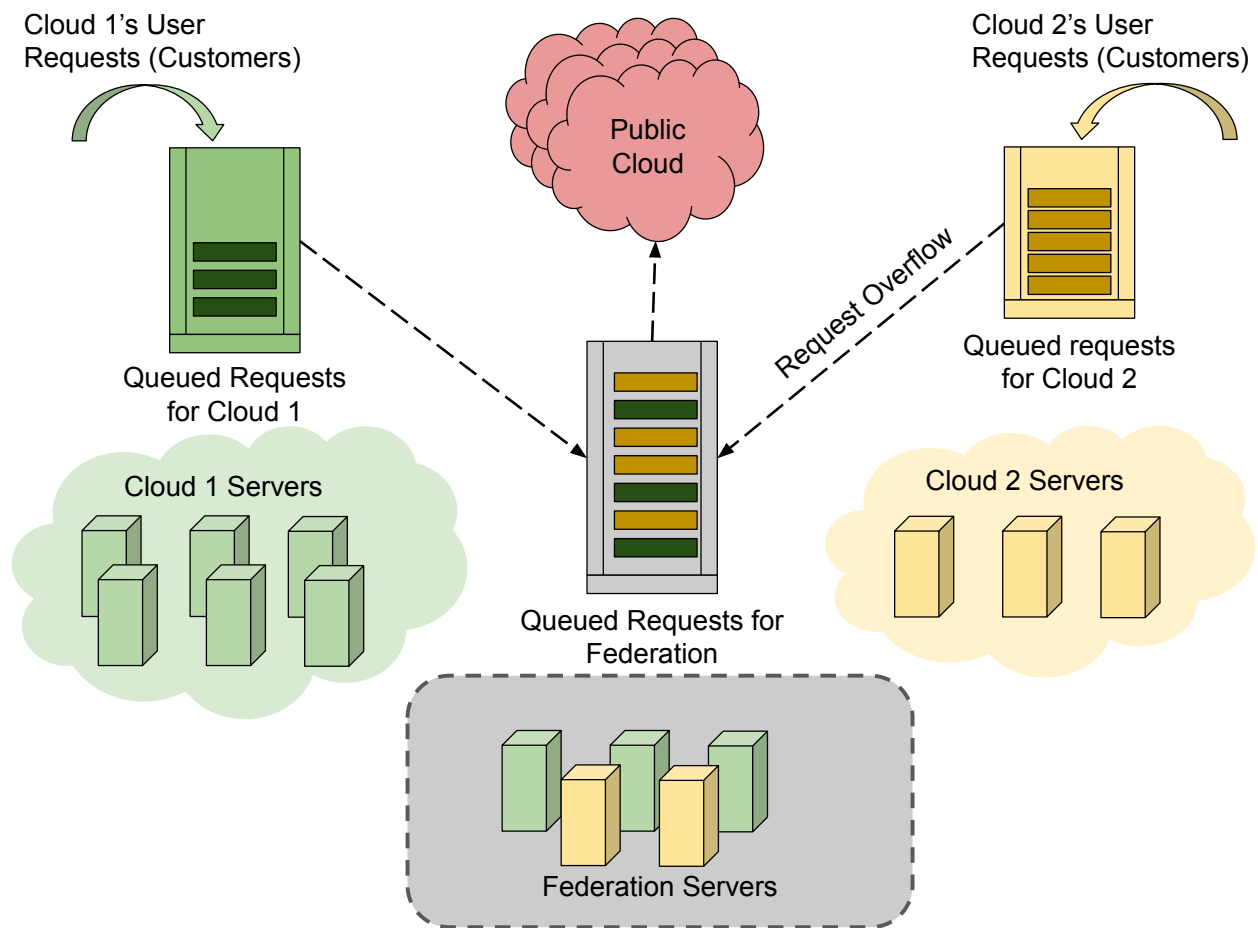


Figure 5.1: Federation of Hybrid Clouds

to a shared pool of resources contributed by members of the federation; if the queue at the shared federation pool is also too large, requests are forwarded to public clouds. Through the federation, underprovisioned hybrid clouds save on public cloud costs, while overprovisioned hybrid clouds can put their resources to work. To reward all members for their contributions (computing resources to serve overflow traffic, or workload to put idle resources to use), the federation mechanism keeps each hybrid cloud's public cloud costs equal to the costs paid before joining the federation, and then *public cloud savings due to the shared server pool are distributed among members* according to Shapley value [93].

A challenging problem for hybrid clouds is to decide whether to join a federation, and to determine the amount of resources to share in order to maximize profit; this problem is particularly difficult when each hybrid cloud has different QoS requirements for its users. Fig. 5.2 illustrates forwarding rates to public clouds for different federations among hybrid clouds A, B, C where: (1) each hybrid cloud has 100 servers processing requests with service times exponentially distributed with rate 1 and interarrival times exponentially distributed with rate 100; (2) all resources of a member are shared with the federation; (3) requests of C can tolerate mean waiting time equal to 1, while requests of A and B must begin service immediately, without queueing. When A and B form a federation, their cumulative forwarding rate to public clouds is reduced from 15.2 to 10.8, and Shapley value splits these savings equally; in contrast, when A and C (or, similarly, B and C) form a federation, *their cumulative forwarding rate to public clouds increases* from 8.5 to 10.3 (the forwarding rate of C becomes 0, while the forwarding rate of A becomes 10.3). This negative effect is due to the different QoS requirements of A and C: requests from C can accumulate at the shared federation pool, forcing A to use the public cloud during peak loads. A similar, but less evident effect is also present when all hybrid clouds A, B, C join the federation: in this case, the cumulative forwarding rate to public clouds (13.2) is higher than that of a federation including only A and B, and leaving C on its own (11.7).

Contributions. Our work provides a solution to evaluate the effects of different sharing

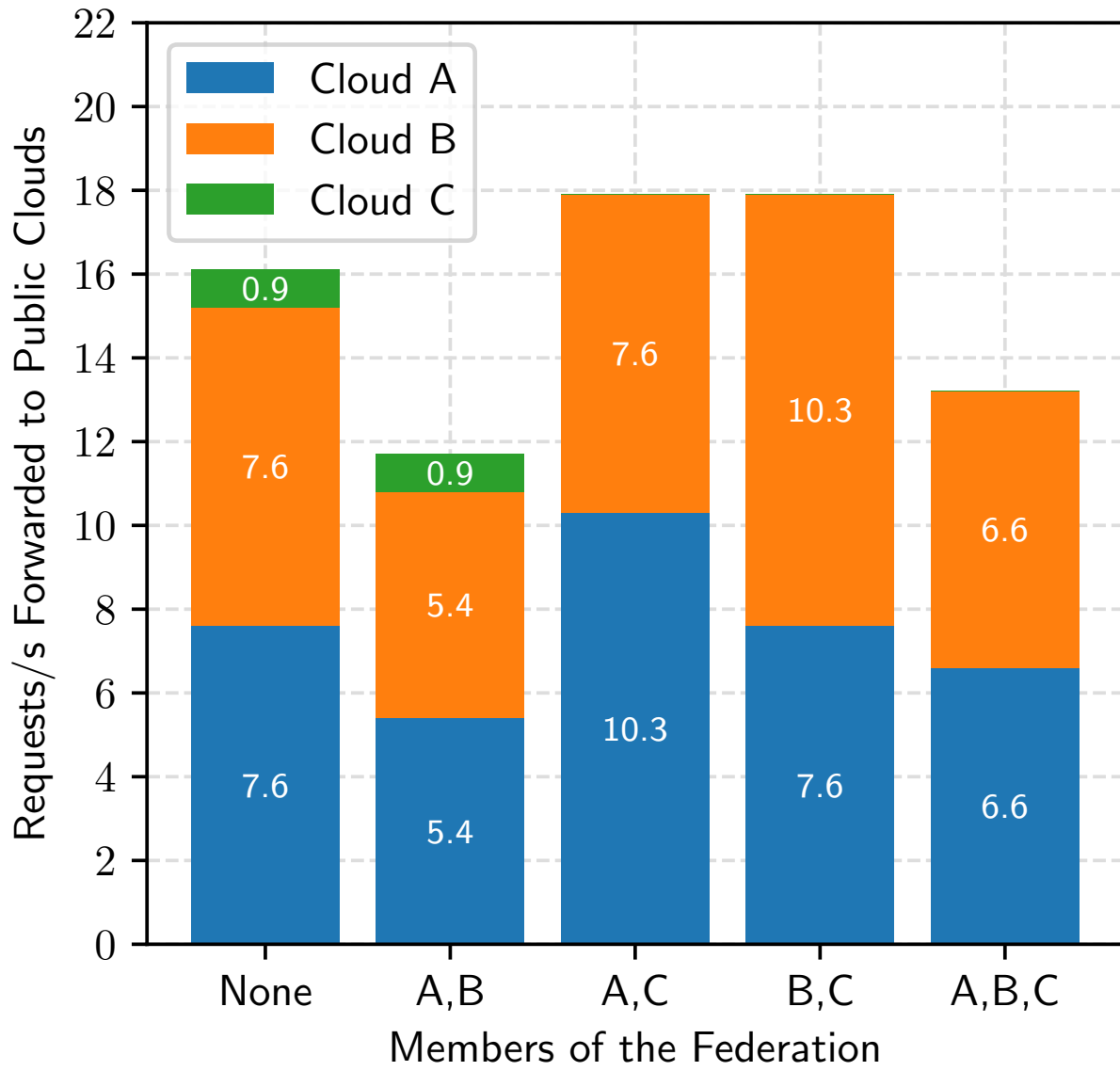


Figure 5.2: Forwarding rates to public clouds for different federations

strategies in federations of hybrid clouds, which we model as a network of queues [89], each with a finite number of servers and capacity determined by QoS requirements (maximum mean waiting time, as in [88, 86]). By analyzing the underlying continuous-time Markov chain (CTMC), we leverage existing results for resource sharing in queueing networks [92] to prove that, if all hybrid clouds have the same QoS requirements, *profit is maximized when each hybrid cloud shares all of its resources* (without priority over their use). Next, we show that this result does not hold when hybrid clouds have different QoS requirements, and we provide a solution to evaluate profit for different resource sharing decisions. Finally, we compare our profit sharing mechanism (assigning public cloud savings according to Shapley value) with alternative approaches, illustrating its ability to discourage “free riders” of the federation.

5.2 Performance Model

In this section, we present a performance model to evaluate the stationary rate of requests that cannot be served by a federation of private clouds without violating their individual QoS requirements, and must therefore be forwarded to a public cloud provider. We prove that, if each private cloud has the same QoS requirements, this rate of “rejected requests” is minimized when all resources are shared with the federation; in contrast, for private clouds with heterogeneous QoS requirements, we provide counterexamples showing that sharing all resources may increase rejected traffic for some clouds, and in some cases for the entire federation.

5.2.1 System Description and Notation

We consider the federation of N private clouds illustrated in Fig. 5.3, where each private cloud $i = 1, \dots, N$ receives requests according to a Poisson process with rate λ_i and owns n_i servers: m_i servers are contributed to the shared federation pool (which has a total of $s_F = \sum_{i=1}^N m_i$ servers), while $s_i = n_i - m_i$ servers are used exclusively by i .

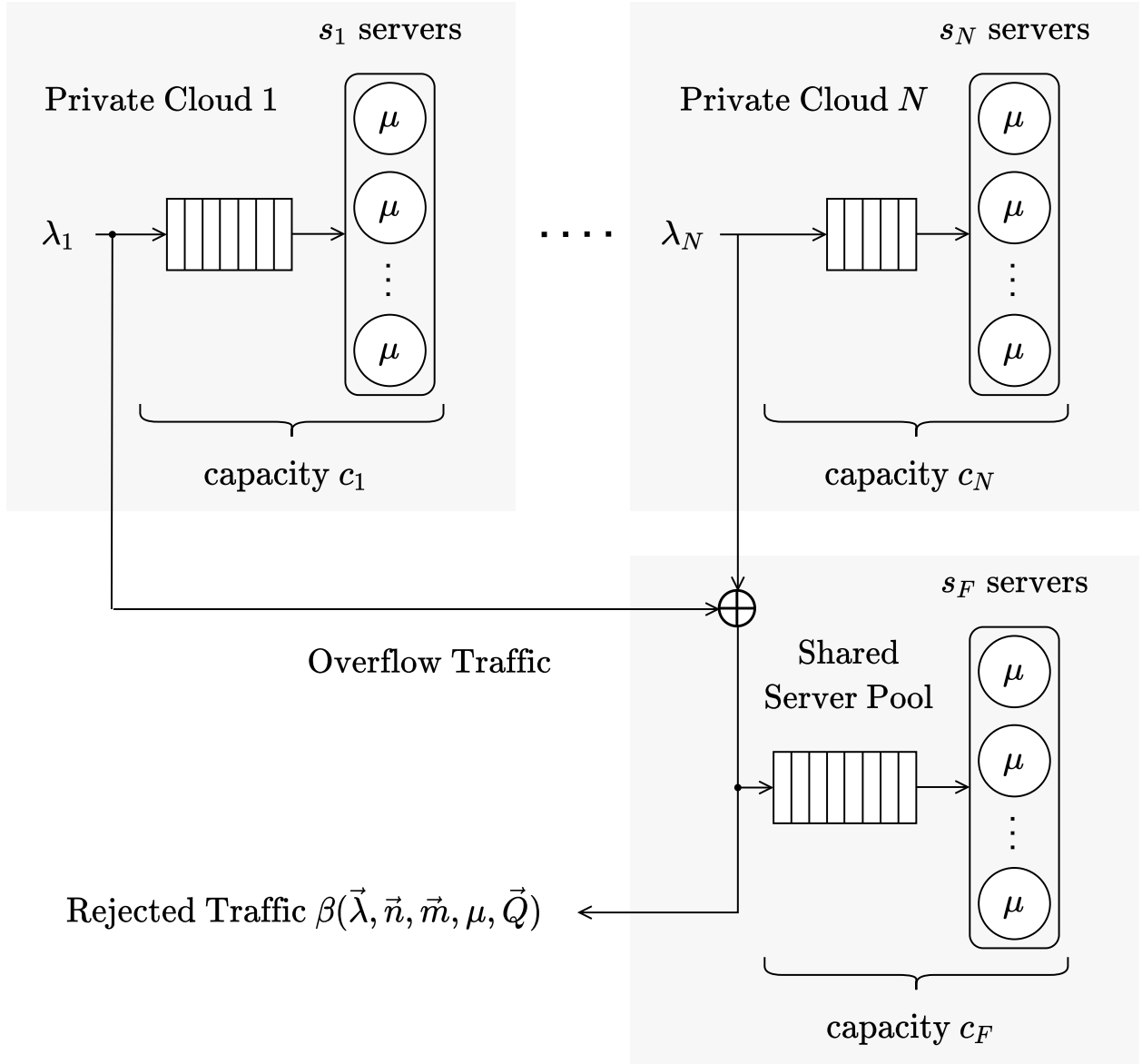


Figure 5.3: Queueing Model of a Hybrid Cloud Federation

When all s_i local servers are busy, requests received by i are queued locally (and processed according to a first-come, first-served (FCFS) policy) as long as the queue size q_i is such that $q_i/(s_i\mu) \leq Q_i$, i.e., the expected waiting time of the last request in the queue (the sum of q_i i.i.d. exponential random variables with rate $s_i\mu$) is no greater than Q_i , the QoS requirement for users of private cloud i . Similarly, a request rejected locally at private cloud i can be served by the pool of s_F servers contributed to the federation as long as its queue size q_F is such that $q_F/(s_F\mu) \leq Q_i$.

Note that this type of QoS requirements corresponds to a maximum *capacity* $c_i := C(s_i, \mu, Q_i)$ for each private cloud i , where

$$C(s_i, \mu, Q_i) = \lfloor s_i\mu Q_i \rfloor + s_i = \lfloor s_i(\mu Q_i + 1) \rfloor$$

is the maximum number of requests queued ($\lfloor s_i\mu Q_i \rfloor$) or in service (s_i) at private cloud i . For example, no queueing is allowed when $Q_i = 0$, since c_i is equal to the number of servers s_i . The maximum capacity of the shared federation pool is equal to $c_F := \max_{i=1,\dots,N} C(s_F, \mu, Q_i)$, i.e., to the maximum capacity allowed by the QoS requirements of any private cloud. A large class of QoS requirements (such as bounds on percentiles of waiting time or service time of a request) can be defined similarly from the distribution of waiting times (Erlang with shape q_i and rate $s_i\mu$) and service time (exponential with rate μ); these QoS requirements also result in restrictions on the capacity of each private cloud, and they are supported by our model.

Our goal is to evaluate the stationary (i.e., steady-state) rate $\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q})$ of requests cumulatively rejected by all the private clouds of the federation for the given arrival rates λ_i , shared servers $0 \leq m_i \leq n_i$, service rate μ , and maximum mean waiting times Q_i , for $i = 1, \dots, N$. Since the overflow traffic in Fig. 5.3 is not a Poisson process, we need a joint CTMC model of the private clouds and shared server pool to evaluate $\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q})$.

5.2.2 CTMC Model

Given our assumptions, the evolution of a hybrid cloud federation is described by a CTMC with state vector $(x_1, \dots, x_N, x_F)$ where $0 \leq x_i \leq C(s_i, \mu, Q_i)$ represents the number of requests (queued or in service) at each private cloud $i = 1, \dots, N$, while $0 \leq x_F \leq \max_{i=1, \dots, N} C(s_F, \mu, Q_i)$ is the number of requests (queued or in service) at the shared server pool.

From state $(x_1, \dots, x_i, \dots, x_N, x_F)$, the CTMC can transition to the following states, for all $i = 1, \dots, N$ (Fig. 5.4):

- If $x_i < C(s_i, \mu, Q_i)$, to $(x_1, \dots, x_i + 1, \dots, x_N, x_F)$ with rate λ_i (an arrival to private cloud i can be served locally while satisfying QoS requirements).
- If $x_i = C(s_i, \mu, Q_i)$ and $x_F < C(s_F, \mu, Q_i)$, to state $(x_1, \dots, x_i, \dots, x_N, x_F + 1)$ with rate λ_i (an arrival to private cloud i cannot be served locally, but it can be served by the shared federation pool while satisfying QoS requirements of private cloud i).
- To $(x_1, \dots, x_i - 1, \dots, x_N, x_F)$ with rate $\min(x_i, s_i)\mu$ (service of a request completes at private cloud i).
- To $(x_1, \dots, x_i, \dots, x_N, x_F - 1)$ with rate $\min(x_F, s_F)\mu$ (service of a request completes at the pool of servers shared by members of the federation).

The rate $\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q})$ of requests rejected by the federation of hybrid clouds can be evaluated from the steady-state probabilities $p(\vec{x})$ for each $\vec{x}$ in the state space $\mathcal{X}$ of the CTMC model: $\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q})$ is equal to the sum of the arrival rate of each private cloud, multiplied by the probability of being in a state where arrivals are rejected by the private cloud and by the federation (and thus forwarded to the public cloud):

$$\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q}) = \sum_{i=1}^N \lambda_i \left(\sum_{\substack{\vec{x} \in \mathcal{X}: x_i = C(s_i, \mu, Q_i) \\ \wedge x_F \geq C(s_F, \mu, Q_i)}} p(\vec{x}) \right). \quad (5.1)$$

Rate λ_i if $x_i = C(s_i, \mu, Q_i) \wedge x_F < C(s_F, \mu, Q_i)$

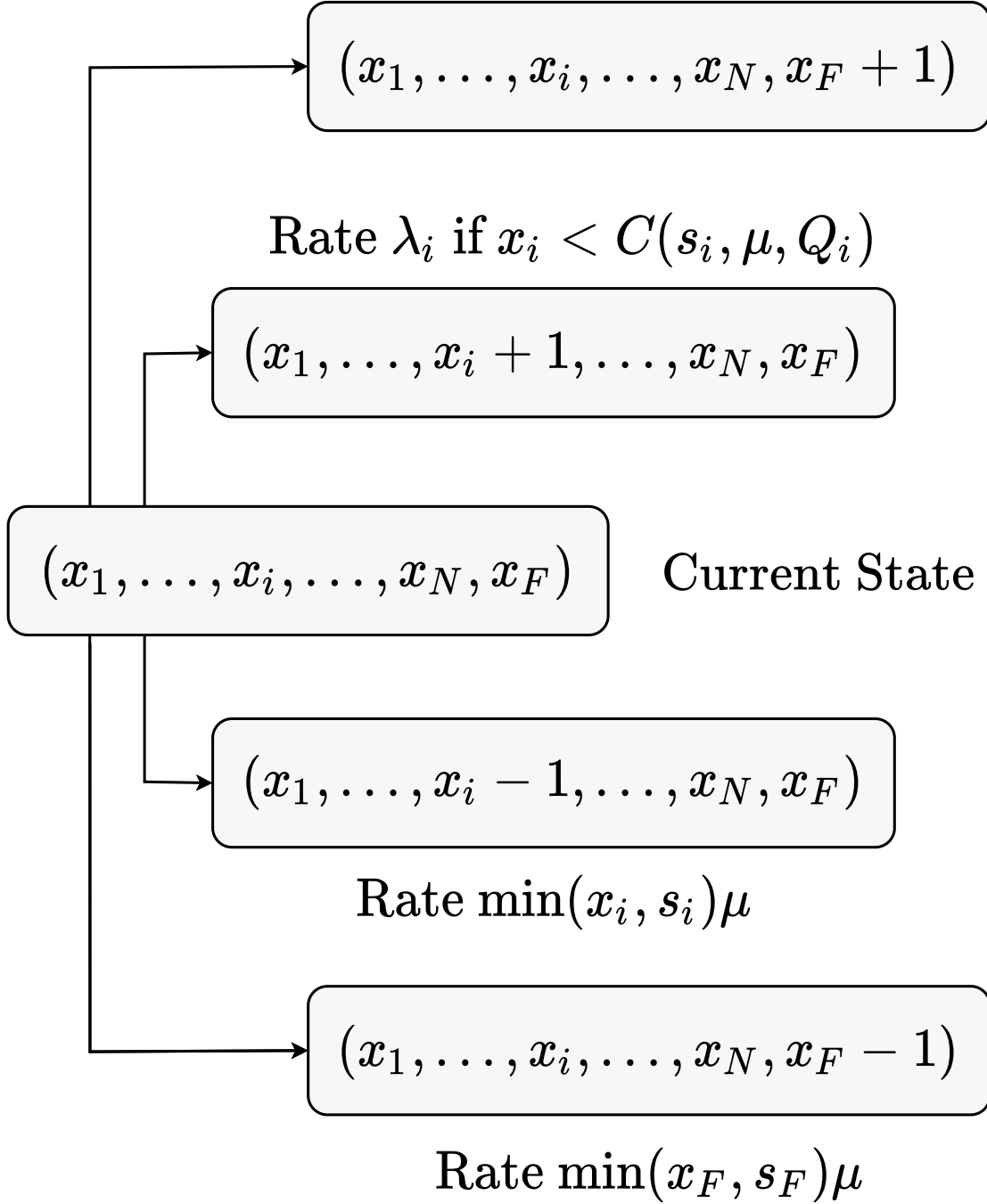


Figure 5.4: Transition rates for the CTMC model

5.2.3 Optimal Sharing Strategy with Homogeneous QoS

Each private cloud i can choose to share $0 \leq m_i \leq n_i$ servers with the federation pool, which receives overflow traffic from all the private clouds, each with different arrival rate, number of servers, and QoS requirements. We are interested in the strategy $\vec{m} = (m_1, \dots, m_N)$ that is the most efficient for the entire federation, i.e., the strategy that minimizes the stationary rate of rejected requests in Eq. (5.1). We prove that sharing all servers is the most efficient strategy when private clouds have the same QoS requirements $Q_1, \dots, Q_N$.

Theorem 1. *Consider a set of $M/M/s_i/c_i$ queues, $i = 1, \dots, N$, each with Poisson arrival rate λ_i , service rate μ , s_i servers and total capacity (requests queued or in service) $c_i := C(s_i)$, where $C(x+y) \geq C(x) + C(y) \forall x, y$. When these queues send their overflow traffic to a shared $G/M/s_F/c_F$ queue (without external arrivals) with $s_F := \sum_{i=1}^N (n_i - s_i)$ servers and capacity $c_F := C(s_F)$, the stationary rate of rejected requests is minimized with respect to s_i if $s_i = 0$ for all $i = 1, \dots, N$ (i.e., when $s_F = \sum_{i=1}^N n_i$).*

Proof. First, we leverage Theorem 7 of [92], which states that the cumulative rejection rate of a set of $M/M/s_i/c_i$ queues for $i = 1, \dots, N$ is greater than or equal to the rejection rate of a single $M/M/s/c$ queue with combined arrival rate $\lambda = \sum_{i=1}^N \lambda_i$, servers $s = \sum_{i=1}^N s_i$ and capacity $c = \sum_{i=1}^N c_i$. This means that the system in Fig. 5.5 has lower overflow traffic than the one in Fig. 5.3.

Next, we show that the rejection rate of an $M/M/s/c$ queue forwarding its overflow traffic to a $G/M/s_F/c_F$ queue without other external arrivals (Fig. 5.5) is greater than or equal to the rejection rate of a single $M/M/(s + s_F)/(c + c_F)$ queue with the same arrival rate. Our argument hinges on the comparison of transition rates between states of the two systems. The underlying stochastic process of the combined $M/M/(s + s_F)/(c + c_F)$ queue (Fig. 5.6) is a birth-death CTMC where state $j + 1$ is reached from $j = 0, \dots, c + c_F - 1$ with rate λ , while state $j - 1$ is reached from $j = 1, \dots, c + c_F$ with rate $\min(j, s + s_F)\mu$. In the system with separate queues (Fig. 5.7), state $j + 1$ is still reached from $j = 0, \dots, c + c_F - 1$

with the same rate λ ; in contrast, state $j - 1$ is reached from $j = 1, \dots, c + c_F$ with rate $[\min(j_1, s) + \min(j_2, s_F)]\mu$, which depends not only on the total number of requests j , but also on the number of requests j_1 and j_2 in each subsystem (and thus on the past history of transitions that led to $j = j_1 + j_2$).

Since $\min(j_1, s) + \min(j_2, s_F) \leq \min(j, s + s_F)$ for all j_1, j_2 such that $j_1 + j_2 = j$, the transition rate from j to $j - 1$ is always lower than or equal for the system with separate queues, while the transition rate from j to $j + 1$ is the same. As a consequence, the steady-state probability of state $j = c + c_F$ (the only state where requests are rejected, with rate λ) is greater than or equal in the system with separate queues.

Thus, the combined system with $s + s_F$ servers (i.e., where all servers are shared by the private clouds) has a lower rate of rejected requests than the one with separate queues with s and s_F servers, which, in turn, has a lower rate of rejected requests than the original system $\forall s_1, \dots, s_N$ such that $\sum_{i=1}^N s_i = s$. Since $C(x + y) \geq C(x) + C(y) \forall x, y$, the capacity of the combined system is in fact greater than or equal to the sum of individual capacities, i.e., $C(\sum_{i=1}^N s_i) \geq \sum_{i=1}^N C(s_i)$; this increases the number of queueing slots, further reducing the rejection rate when all servers are shared. $\square$

Corollary 1.1. *If $Q_i = Q$ for all i , the rate $\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q})$ of requests rejected by the private clouds and federation server pool in Eq. (5.1) is minimized when $m_i = n_i$ for all $i = 1, \dots, N$.*

Proof. Each private cloud is an $M/M/s_i/c_i$ queue with $s_i = n_i - m_i$ and $c_i = C(s_i, \mu, Q)$ sending its overflow traffic to the server pool of the federation, a $G/M/s_F/c_F$ queue with $s_F = \sum_{i=1}^N m_i = \sum_{i=1}^N (n_i - s_i)$ servers and capacity $c_F = C(s_F, \mu, Q)$. Since $C(s, \mu, Q) := \lfloor s(1 + \mu Q) \rfloor$ satisfies

$$C(x + y, \mu, Q) \geq C(x, \mu, Q) + C(y, \mu, Q)$$

for all x, y, μ, Q , by Theorem 1 the rate $\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q})$ of rejected requests is minimized when all servers are shared. $\square$

Note that, when all servers are shared ($m_i = n_i$ for all $i = 1, \dots, N$), the system reduces

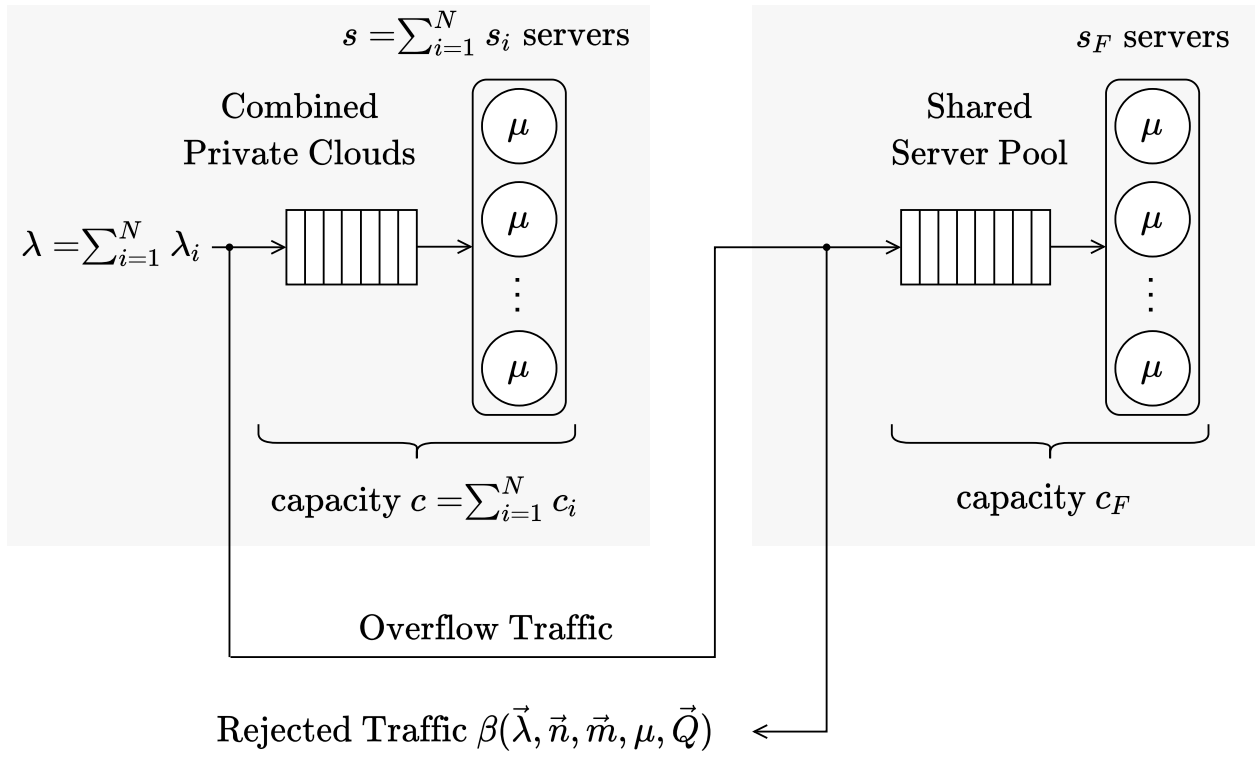


Figure 5.5: Queueing model after combining the private clouds

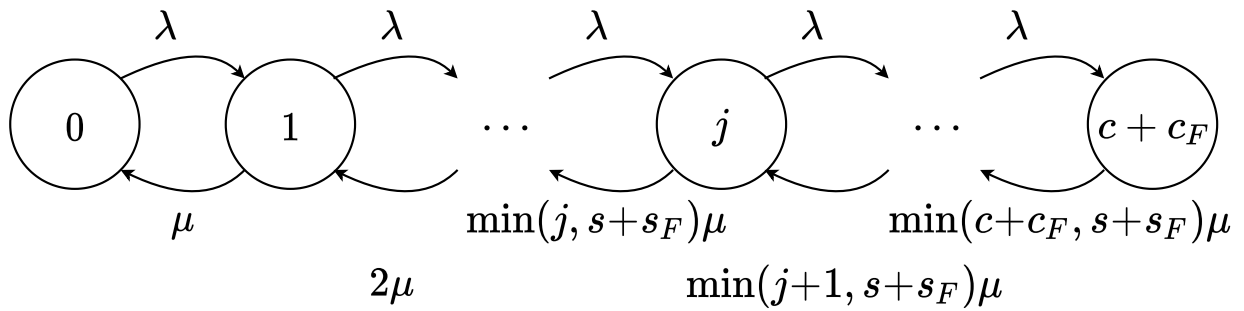


Figure 5.6: Transition rates of an $M/M/(s + s_F)/(c + c_F)$ queue

to an $M/M/s_F/c_F$ queue with $s_F = \sum_{i=1}^N n_i$ servers and arrival rate $\lambda = \sum_{i=1}^N \lambda_i$. In this case, the stationary rate of rejected requests can be evaluated as $\beta(\vec{\lambda}, \vec{n}, \vec{m}, \mu, \vec{Q}) = \lambda B(s_F, c_F, \lambda/\mu)$, where $B(s_F, c_F, \lambda/\mu)$ is the blocking probability of an $M/M/s_F/c_F$ queue [89]:

$$\begin{aligned} B(s_F, c_F, \lambda/\mu) &= \frac{(\lambda/\mu)^{c_F} / (s_F! s_F^{c_F-s_F})}{\sum_{j=0}^{s_F} (\lambda/\mu)^j / j! + \sum_{j=s_F+1}^{c_F} (\lambda/\mu)^j / (s_F! s_F^{j-s_F})} . \end{aligned} \quad (5.2)$$

5.2.4 Optimal Sharing Strategy with Heterogeneous QoS

When the private clouds $i = 1, \dots, N$ have different QoS requirements Q_i (i.e., maximum mean waiting times), the CTMC model presented in Section 5.2.2 uses different capacity bounds $C(s_i, \mu, Q_i)$ at their local queues and different criteria to reject incoming requests at the shared server pool: requests from private cloud i are rejected at the shared server pool if $x_F \geq C(s_F, \mu, Q_i)$; i.e., depending on the number of requests x_F at the shared server pool, private clouds with less stringent QoS requirements Q_i can add their requests to its queue, while other private clouds with more stringent QoS may be forced to forward their requests to the public cloud. In this case, when private clouds share all of their servers, the federation does not reduce to an $M/M/s_F/c_F$ queue, and it can incur a higher rejection rate (i.e., forwarding rate to the public cloud) with respect to other sharing strategies.

As an example, we consider a federation where $\vec{\lambda} = (50, 50)$, $\vec{n} = (50, 50)$, $\mu = 1$, and $\vec{Q} = (0, 1)$; due to the difference in QoS requirements, for a given sharing strategy $\vec{m} = (m_1, m_2)$, only cloud 2 can queue up to $s_F \mu Q_2 = m_1 + m_2$ requests at the shared server pool of $s_F = m_1 + m_2$ servers. Using our CTMC model and Eq. (5.1), we evaluate the rate of requests rejected by the federation and forwarded to public clouds for different sharing strategies, as depicted in Fig. 5.8. Here, we observe that, for any sharing strategy $m_2 > 0$ of cloud 2, as we increase the number of servers m_1 shared by cloud 1, the rate of requests forwarded to public clouds first decreases (as cloud 1 gains access to shared servers of

cloud 2) and then noticeably increases (as cloud 2 is able to queue its requests at the shared server pool and to force cloud 1 to forward requests to public clouds). In contrast with our theoretical result for hybrid clouds with homogeneous QoS (where sharing all servers, i.e., $(m_1, m_2) = (50, 50)$ in this example, is always optimal), the optimal sharing strategy is $(m_1, m_2) = (10, 2)$, a partial sharing of resources. While a theoretical result is not available for hybrid clouds with heterogeneous QoS, their policies can be evaluated using our CTMC model.

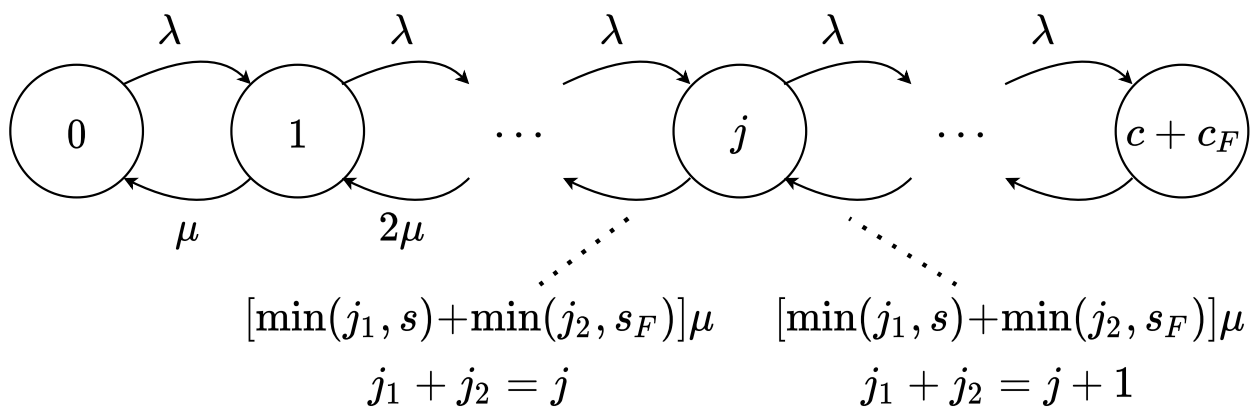


Figure 5.7: Transition rates of an $M/M/s/c$ queue forwarding overflow to a $G/M/s_F/c_F$ queue (separate queues)

5.3 Revenue Model

The federation mechanism does not require setting a price for resources shared by members: *servers of the shared pool are available to the federation members at no cost.* To avoid situations where some members of the federation take advantage of shared servers to the detriment of their owners, we establish that, as part of the federation agreement, *each member is charged the same public cloud costs as previously paid, i.e., before joining the federation.* When the shared server pool results in savings of public cloud costs (as demonstrated in Section 5.2.3, this is always the case with homogeneous QoS requirements when sharing all resources), *we distribute savings among federation members according to Shapley value.*

In so doing, we favor resource sharing without more complex policies like granting priority over shared resources to the members who own and operate them [86]. At the same time, through the use of Shapley value, we are able to reward overprovisioned members who share idle resources with the federation, as well as underprovisioned members who share part of their workload; as discussed in Section 5.4, both types of members are necessary to produce public cloud savings, and thus both should be rewarded in the federation.

5.3.1 Sharing Public Cloud Savings using Shapley Value

Given a subset $S = \{i_1, \dots, i_K\}$ of K private clouds, i.e., $S \subseteq \{1, \dots, N\}$, we indicate by

$$\vec{\lambda}_S := (\lambda_{i_1}, \dots, \lambda_{i_K})$$

$$\vec{n}_S := (n_{i_1}, \dots, n_{i_K})$$

$$\vec{m}_S := (m_{i_1}, \dots, m_{i_K})$$

$$\vec{Q}_S := (Q_{i_1}, \dots, Q_{i_K})$$

the vectors of arrival rates $\vec{\lambda}_S$, number of servers $\vec{n}_S$, shared servers $\vec{m}_S$, and maximum mean waiting times $\vec{Q}_S$, respectively, for the private clouds in S , and by $\beta(\vec{\lambda}_S, \vec{n}_S, \vec{m}_S, \vec{Q}_S)$ the rate

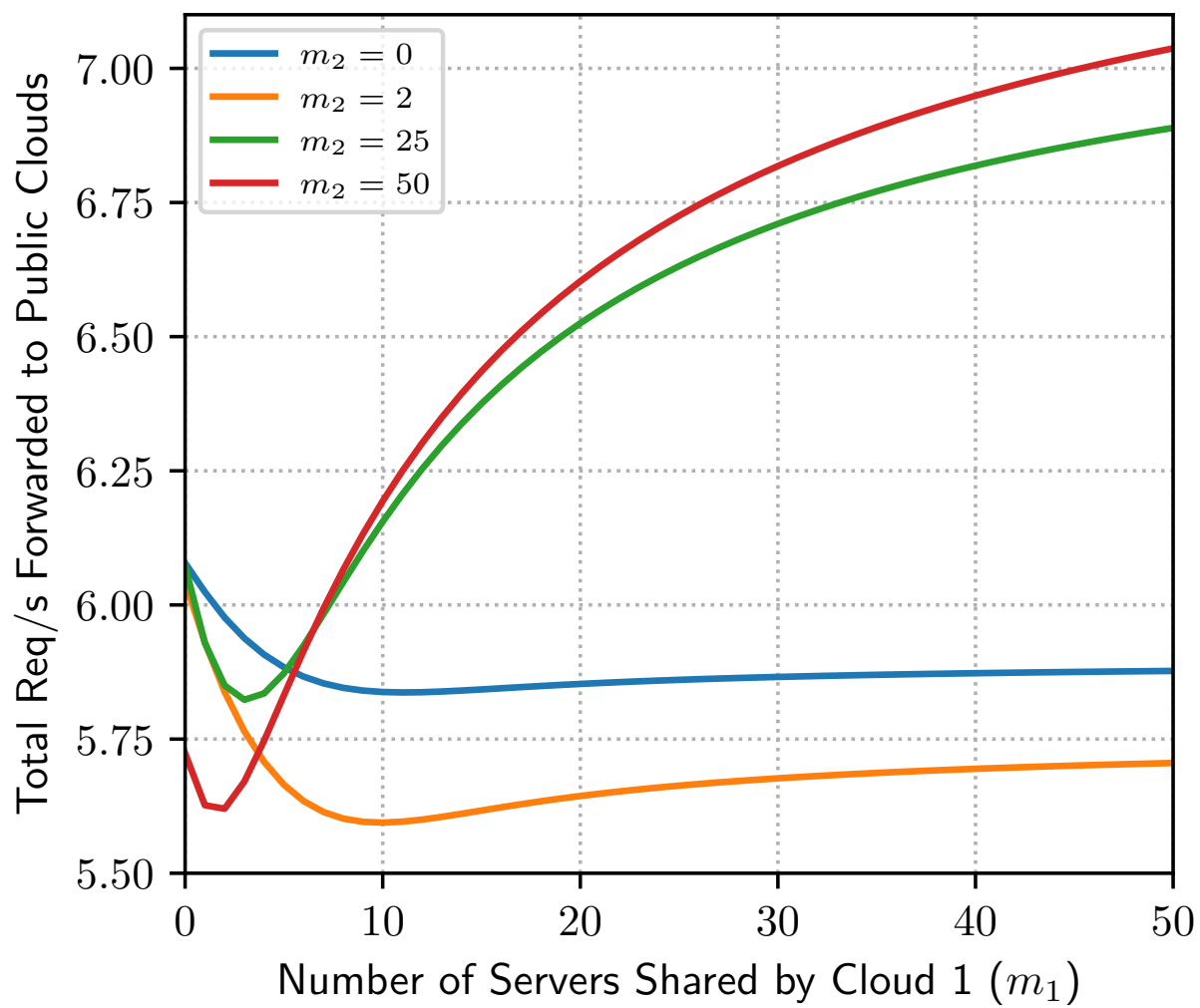


Figure 5.8: Sharing Strategies with Heterogeneous QoS

of requests sent to the public cloud when they form a federation and share servers $\vec{m}_S$ (β can be evaluated using the CTMC model presented in Section 5.2.2).

Then, the savings of public cloud costs obtained by the federation S are given by

$$R(S) = P \left(\eta_0(S) - \eta_F(S) \right) \quad (5.3)$$

where

$$\eta_0(S) := \sum_{i \in S} \beta(\vec{\lambda}_{\{i\}}, \vec{n}_{\{i\}}, \vec{0}_{\{i\}}, \vec{Q}_{\{i\}}) \quad (5.4)$$

$$\eta_F(S) := \beta(\vec{\lambda}_S, \vec{n}_S, \vec{m}_S, \vec{Q}_S) \quad (5.5)$$

i.e., by the difference between the rate of requests forwarded before joining the federation (by individual private clouds) and after (when the queue at the federation pool is too large), multiplied by P , the public cloud cost per request. Note that, by Corollary 1.1, Eq. (5.5) is minimized under homogeneous QoS when all servers are shared with the federation, maximizing $R(S)$ and thus the profit of the federation.

We distribute the public cloud savings $R(S)$ of a federation using *Shapley value* (SV) [93]; each member $i \in S$ receives $R_i^{SV}(S)$, which is the increase in savings due to i , averaged over all possible subsets of other members (i.e., all alternative federations including i):

$$R_i^{SV}(S) = \sum_{S' \subseteq S \setminus \{i\}} \frac{|S'|!(|S| - |S'| - 1)!}{|S|!} (R(S' \cup \{i\}) - R(S'))$$

Shapley value has many desirable properties for our application: the sum of Shapley values is equal to the total savings, i.e., $\sum_{i \in S} R_i^{SV}(S) = R(S)$; if $i \in S$ and $j \in S$ produce the same increase in savings, i.e., $R(S' \cup \{i\}) = R(S' \cup \{j\})$ for all $S' \subseteq S \setminus \{i, j\}$, then $R_i^{SV}(S) = R_j^{SV}(S)$, i.e., they have the same Shapley value; $R_i^{SV}(S)$ scales linearly with respect to R (and thus with respect to the public cloud price P); if $i \in S$ does not increase

savings in any federation, $R_i^{SV}(S) = 0$.

In particular, the last property implies that “free riders,” the members that do not contribute to public cloud *savings*, are not rewarded by Shapley value. Note that a member can contribute by providing either resources (servers) or workload (overflow traffic of requests); for example, in a federation with two private clouds where one is overprovisioned and the other is underprovisioned, both members are necessary to generate public cloud savings, and their Shapley values are the same (specifically, $R(S)/2$, half of the generated savings).

Our analysis assumes that idle and busy servers have the same operational costs. If busy servers incur additional costs $P_{busy} - P_{idle}$ per request, then P can be replaced in Eq. (5.3) with $P - (P_{busy} - P_{idle})$: in this case, additional per-request costs are paid to the owners of the shared servers before calculating the public cloud savings to distribute among members of the federation (P_{idle} is already accounted for in Eq. (5.6) below). Note that when $P \leq (P_{busy} - P_{idle})$, i.e., the public cloud price is lower than the additional cost sustained by private clouds to put their resources to work, private clouds have no incentive to use their local resources (public cloud providers are cheaper); in the following, we assume that $P > (P_{busy} - P_{idle})$ and also $P > P_{busy}$, so that savings can be obtained by using local resources owned by private clouds.

5.3.2 Overall Reduction in Operating Costs

Public cloud costs are only a fraction of the operational costs of a private cloud. To assess the significance of public cloud savings, we consider the *relative cost reduction* of each private cloud i :

$$\delta_i = \frac{R_i^{SV}(S)}{P\beta(\vec{\lambda}_{\{i\}}, \vec{n}_{\{i\}}, \vec{0}_{\{i\}}, \vec{Q}_{\{i\}}) + n_i\xi} \quad (5.6)$$

where $R_i^{SV}(S)$ is the Shapley value distributed to i after joining the federation (due to public cloud savings), while $P\beta(\vec{\lambda}_{\{i\}}, \vec{n}_{\{i\}}, \vec{0}_{\{i\}}, \vec{Q}_{\{i\}})$ and $n_i\xi$ are the public cloud cost and

private operation cost, respectively (these costs are sustained both before and after joining the federation). Based on [137], we assume that $\xi \approx 0.7P\mu$, i.e., serving a request with local servers is 30% less expensive than using cloud resources (after accounting for all operational costs such as energy, cooling, management).

5.3.3 Alternative Reward Policies

In this section, we consider alternative policies to share public cloud savings of the federation among private clouds $S = \{i_1, \dots, i_K\}$, $S \subseteq \{1, \dots, N\}$, with vectors of arrival rates $\vec{\lambda}_S$, number of servers $\vec{n}_S$, shared servers $\vec{m}_S$, and maximum mean waiting times $\vec{Q}_S$.

Shared Resources (SR). This policy distributes public cloud savings proportionally to the amount of resources shared by each private cloud:

$$R_i^{SR}(S) := \frac{m_i}{\sum_{j=1}^K m_j} R(S)$$

Note that this policy does not take into account the utilization of shared resources before or after joining the federation.

Shared Idle Resources (SIR). This policy distributes public cloud savings proportionally to the amount of shared resources *and to their utilization before joining the federation*:

$$R_i^{SIR}(S) := \frac{m_i(1 - \rho_i)}{\sum_{j=1}^K m_j(1 - \rho_j)} R(S)$$

where

$$\rho_i = \frac{\lambda_i - \beta(\vec{\lambda}_{\{i\}}, \vec{n}_{\{i\}}, \vec{0}_{\{i\}}, \vec{Q}_{\{i\}})}{n_i \mu}$$

is the utilization of the servers of private cloud i before joining the federation. With this policy, a higher fraction of cloud savings is received if shared servers had lower utilization ρ_i before joining the federation.

Shared Idle Resources and Workload (SIRW). This policy distributes public cloud savings proportionally to the amount of *shared idle resources and overflow traffic*:

$$R_i^{SIRW}(S) := \frac{1}{2} \left(R_i^{SIR}(S) + \frac{\lambda_i - \rho_i n_i \mu}{\sum_{j=1}^K (\lambda_j - \rho_j n_j \mu)} R(S) \right)$$

where $\lambda_i - \rho_i n_i \mu = \beta(\vec{\lambda}_{\{i\}}, \vec{n}_{\{i\}}, \vec{0}_{\{i\}}, \vec{Q}_{\{i\}})$ is the overflow traffic of private cloud i . With this policy, a higher fraction of cloud savings is received if shared servers had lower utilization ρ_i before joining the federation, or if the private cloud was forwarding many requests to the public cloud.

Note that, while sharing policies SV and SIRW reward members of the federation contributing servers or workload, policies SR and SIR distribute public cloud savings only to members sharing servers. With SR or SIR, members are not rewarded for sharing workload: in particular, when $m_i = 0$ (no shared servers), private cloud i has no incentive to stay in the federation (since cloud costs are the same and no cloud savings are distributed to it, its profit would be the same without the federation). Nonetheless, we use policies SR and SIR in cases where $m_i > 0$ as additional baselines to illustrate the drawback of not considering workload contributions.

5.4 Experimental Evaluation

In this section, we evaluate the effects of our federation mechanism on public cloud savings and analyze whether these savings are shared fairly among members of the federation.

All results presented in this section were obtained using PRISM [138] to solve our CTMC model through an iterative method (specifically, the *power method* with up to 200,000 iterations and absolute threshold $\epsilon = 10^{-6}$ to detect convergence); results are thus exact (up to a numerical error). In our federations, we use $\mu = 1$ and vary $\vec{\lambda}$, $\vec{n}$, and $\vec{Q}$.

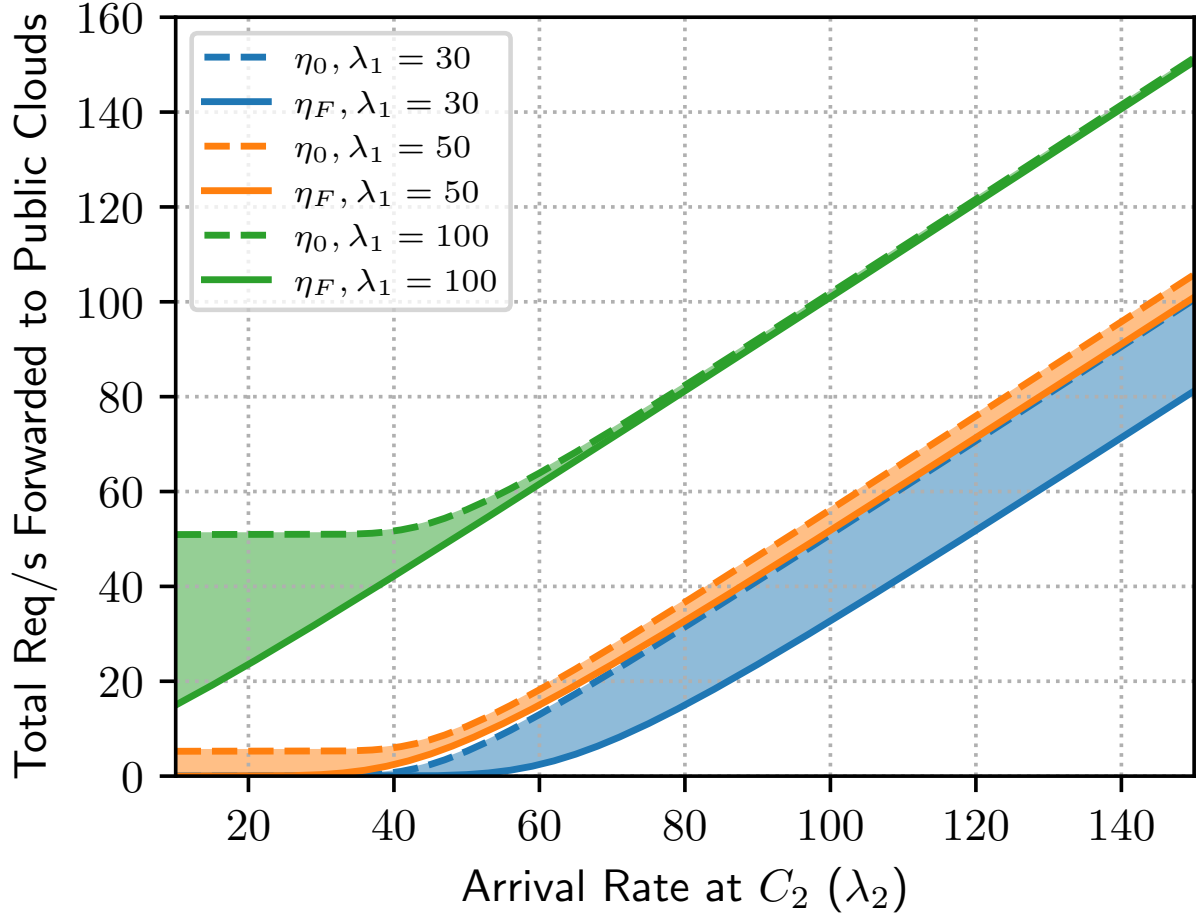


Figure 5.9: Rate of requests sent to public clouds by a federation of two clouds with $n_1 = n_2 = m_1 = m_2 = 50$ servers and $Q_1 = Q_2 = 0$, for different arrival rates λ_1 and λ_2 .

5.4.1 Public Cloud Savings and Operating Cost Reduction

Sharing All Servers. First, we evaluate the savings in public cloud costs achieved by a federation where two private clouds, C_1 and C_2 , have no queueing of requests (i.e., $Q_1 = Q_2 = 0$) and share all of their $n_1 = n_2 = 50$ servers ($m_1 = m_2 = 50$). Fig. 5.9 shows the rate of requests forwarded to the public cloud before (dashed lines) and after (solid lines) forming the federation, i.e., η_0 and η_F as defined in Eqs. (5.4) and (5.5), respectively, for different arrival rates at C_1 : $\lambda_1 = 30$ (blue lines), $\lambda_1 = 50$ (orange lines), $\lambda_1 = 100$ (green lines); the arrival rate at C_2 varies from $\lambda_2 = 10$ to $\lambda_2 = 150$.

- For $\lambda_1 = 30$, C_1 is overprovisioned and, when $\lambda_2 > 50$, it is able to serve part of the workload of C_2 , which is underprovisioned. The federation achieves a reduction of the rate of requests forwarded to public clouds from $\eta_0 = 100.5$ to $\eta_F = 81.2$ (19.2%) when $\lambda_2 = 150$. Note that, as λ_2 increases, η_F grows linearly after $\lambda_2 = 75$ because the federation is overloaded, while the reduction in public cloud usage (the difference between η_0 and η_F) converges to a constant value.
- For $\lambda_1 = 50$, C_1 is close to underprovisioning: when $\lambda_2 < 50$, C_2 is overprovisioned, and it can serve some of C_1 's overflow traffic; for $\lambda_2 > 50$, both C_1 and C_2 are underprovisioned, but some savings are still achieved by the federation (at $\lambda_2 = 150$, the rate of requests forwarded to public clouds is reduced by 4.5%, from $\eta_0 = 105.7$ to $\eta_F = 101.0$).
- For $\lambda_1 = 100$, C_1 is underprovisioned: when $\lambda_2 < 50$, C_2 is overprovisioned and can serve a large fraction of C_1 's overflow traffic; for example, at $\lambda_2 = 10$, the rate of requests forwarded to public clouds is reduced by 70.6%, from $\eta_0 = 50.9$ to $\eta_F = 15.0$. When $\lambda_2 > 50$, the federation of C_1 and C_2 is heavily underprovisioned; the reduction in forwarding rate to public cloud is negligible (0.5% at $\lambda_2 = 150$, from $\eta_0 = 151.4$ to $\eta_F = 150.7$).

(λ_1, λ_2)	η_0	η_F	δ_1	δ_2
(30, 150)	100.5	81.2	27.6%	7.1%
(50, 150)	105.7	101.0	5.9%	1.8%
(100, 150)	151.4	150.7	0.4%	0.3%

Table 5.1: Forwarding rate to public clouds before (η_0) and after (η_F) joining the federation and relative cost reduction (δ_1, δ_2) for different arrival rates when $\vec{n} = \vec{m} = (50, 50)$, $\vec{Q} = (0, 0)$.

Table 5.1 reports the relative cost reduction from Eq. (5.6) for these scenarios where all servers are shared with the federation.

Sharing a Fraction of the Servers. Next, we evaluate the increase in public cloud savings with respect to the amount of resources shared by the two private clouds, C_1 and C_2 , with the same number of servers $n_1 = n_2 = 50$ and QoS $Q_1 = Q_2 = 0$. Fig. 5.10 illustrates that, since C_1 is overprovisioned ($\lambda_1 = 10$) and C_2 is heavily underprovisioned ($\lambda_2 = 150$), the number of servers shared by C_2 , m_2 , has no effect on the savings in public cloud costs (these resources have very high utilization); instead, the percentage reduction of the rate of requests sent to public clouds increases linearly with m_1 , the servers shared by C_1 (which is overprovisioned), until converging to 38.7% savings when the shared server pool becomes highly utilized. Fig. 5.11 shows that, when both clouds are underprovisioned ($\lambda_1 = \lambda_2 = 150$), public cloud savings are still monotonic with respect to the amount of resources shared, but almost negligible (at most 0.24%, when all resources are shared).

Heterogeneous QoS Requirements. Fig. 5.12a presents the forwarding rates to the public cloud before (solid line) and after (dashed line) two private clouds C_1 and C_2 with heterogeneous QoS $\vec{Q} = (0, 1)$ share all of their resources; in this scenario, $\mu = 1$ and $\lambda_1 = n_1 = 50$, while λ_2 varies from 30 to 100 and $n_2 = \lambda_2$ (so that the load $\lambda_2/(n_2\mu) = \lambda_1/(n_1\mu) = 1$ is always the same for both private clouds). We observe that while C_2 no longer forwards requests to the public cloud after joining the federation (orange line), the forwarding rate increases for C_1 (blue line). Notably, *the cumulative forwarding rate is greater after sharing resources in the federation* and, as the workload of C_2 increases, a larger fraction of requests

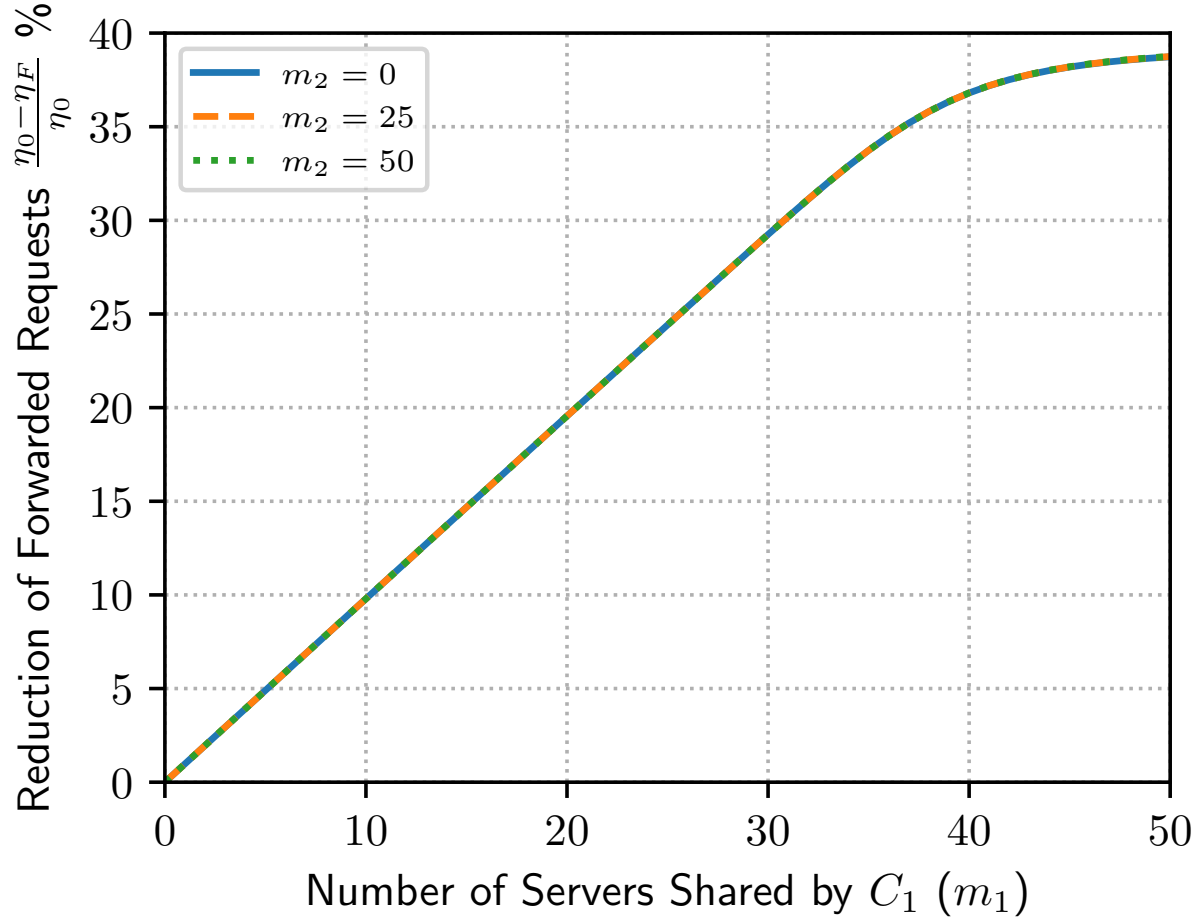


Figure 5.10: Reduction of requests sent to public clouds by two federated clouds with $n_1 = n_2 = 50$ servers, $Q_1 = Q_2 = 0$, $\lambda_1 = 10$, $\lambda_2 = 150$ for different sharing strategies m_1 and m_2 .

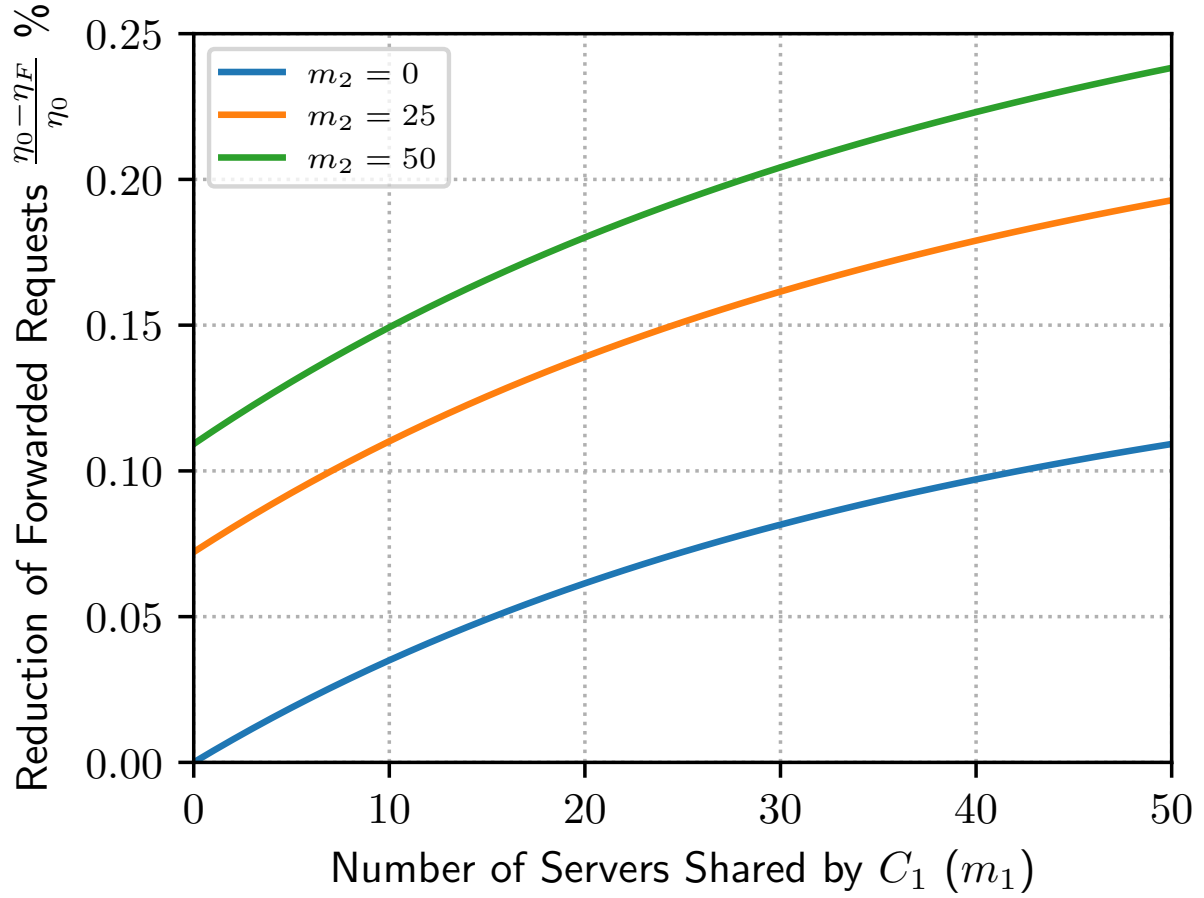
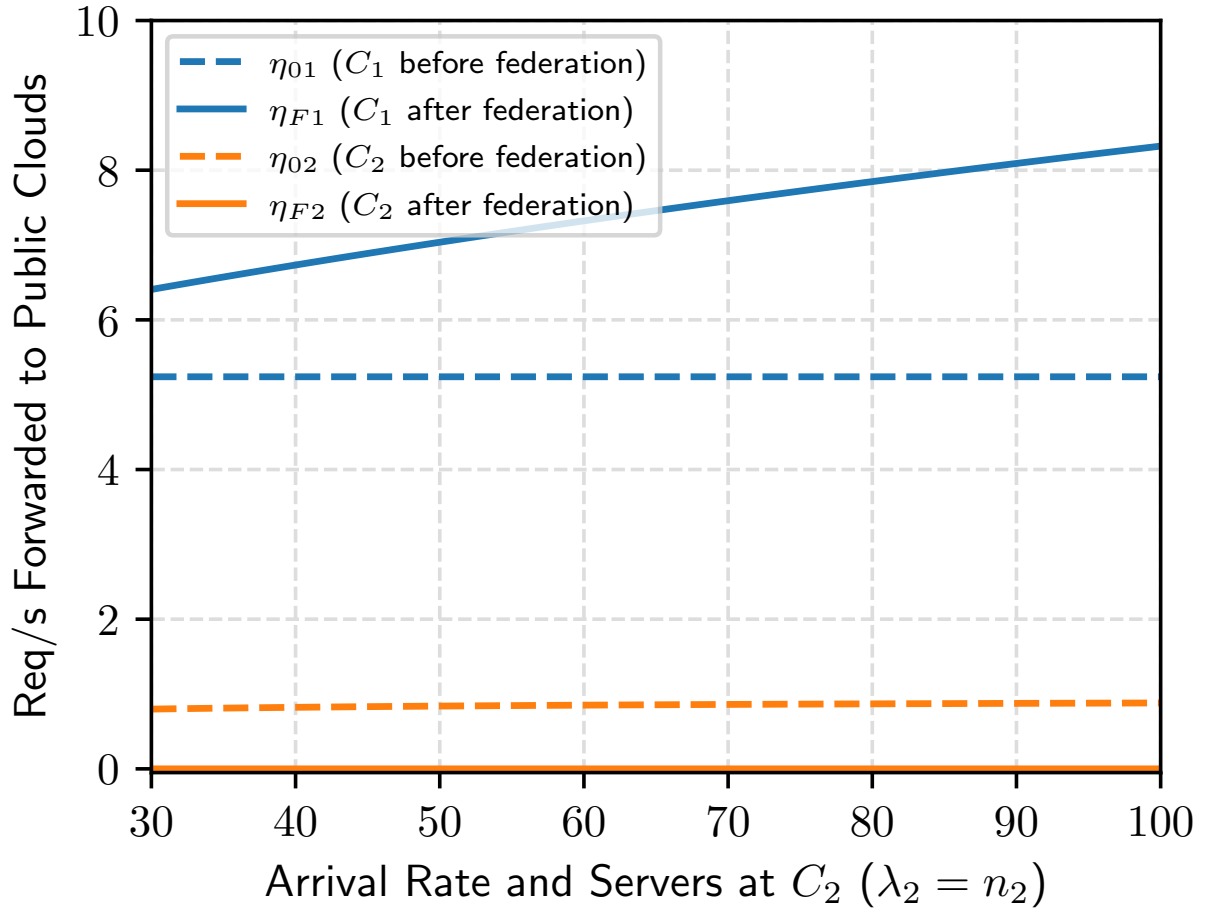
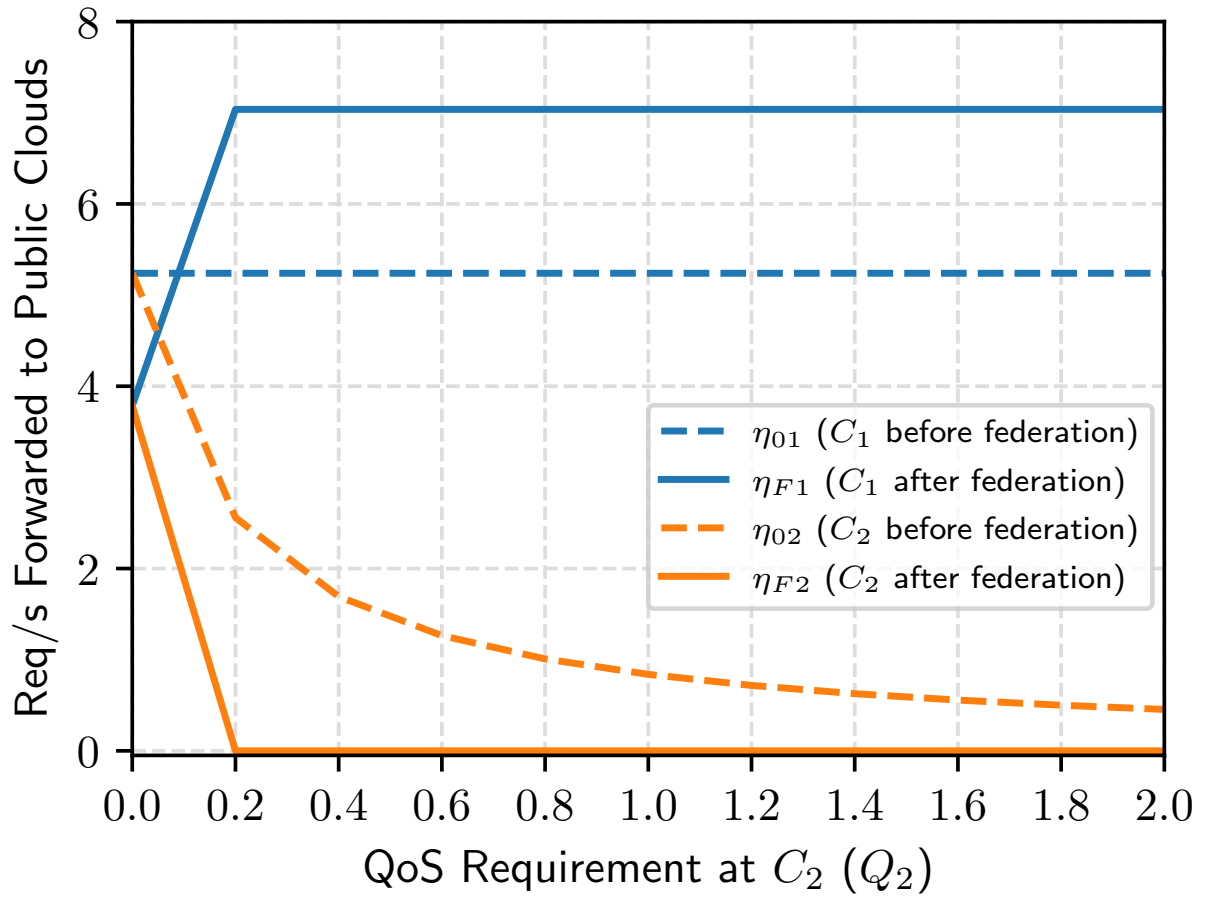


Figure 5.11: Reduction of requests sent to public clouds by two federated clouds with $n_1 = n_2 = 50$ servers, $Q_1 = Q_2 = 0$, $\lambda_1 = \lambda_2 = 150$ for different sharing strategies m_1 and m_2 .

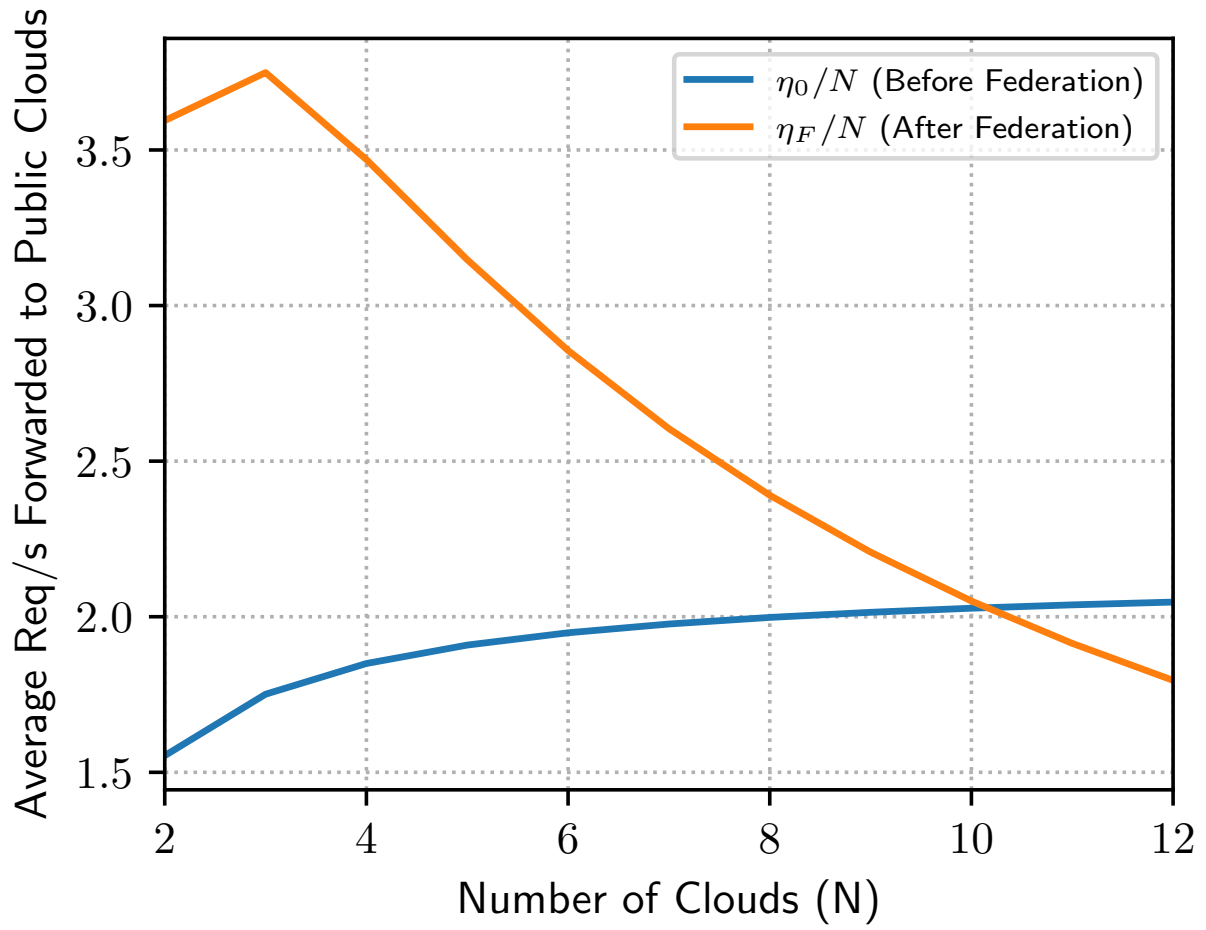


(a) Increasing C_2 's arrivals and servers.

Figure 5.12: Forwarding to public cloud before (dashed lines) and after (solid lines) when clouds with different QoS share all resources.



(b) Increasing C_2 's QoS.



(c) Increasing number of clouds.

of C_1 (which cannot be queued at the shared server pool) is forwarded to the public cloud. Fig. 5.12(b) shows that, for $\vec{\lambda} = \vec{n} = (50, 50)$, the disadvantage of C_1 (i.e., the increase in requests forwarded to the public cloud after joining the federation) is greater as Q_2 increases, allowing more requests of C_2 to queue at the shared server pool; when C_2 's forwarding rate reaches 0, C_2 does not need additional resources and C_1 's forwarding rate stops increasing. Finally, Fig. 5.12(c) considers a scenario with an increasing number N of hybrid clouds, where $\lambda_1 = n_1 = m_1 = 1000$ and $Q_1 = 1$ for C_1 , while $\lambda_i = n_i = m_i = 10$ and $Q_i = 0$ for $i = 2, \dots, N$. In this case, while the federation is initially inefficient (the average rate of requests sent to the public cloud is greater than it would be without the federation) because of C_1 (which is able to queue requests at the shared pool), it becomes advantageous when $N > 10$ hybrid clouds with $Q_i = 0$ are present.

To evaluate the impact of heterogeneous QoS requirements on operating costs of private clouds sharing their resources, we show in Table 5.2 that the relative cost reductions δ_1 and δ_2 can be negative (i.e., public cloud costs can *increase*) when two private clouds C_1 and C_2 have significantly different QoS requirements. In the example, C_2 has greater workload (λ_2), number of servers (n_2), and allowed mean waiting time for its requests (Q_2); when C_1 and C_2 share all of their resources, C_1 is not able to use the shared server pool, where requests of C_2 can wait in a queue but requests of C_1 must be served immediately. As a result, most requests of C_1 are forwarded to the public cloud, increasing public cloud costs. While Shapley value distributes additional costs equally, the relative increase in operating costs is greater for C_1 (a smaller private cloud).

(λ_1, λ_2)	(n_1, n_2)	(Q_1, Q_2)	δ_1	δ_2
(50, 100)	(50, 100)	(0, 10)	-3.7%	-2.1%
(50, 100)	(50, 100)	(0, 50)	-3.8%	-2.2%
(50, 1000)	(50, 1000)	(0, 50)	-14.8%	-0.8%
(7, 10000)	(10, 10000)	(0, 1000)	-37.4%	-0.05%

Table 5.2: Relative cost reduction with heterogeneous QoS.

5.4.2 Reward Policies

To motivate the use of Shapley value (SV) in our system, we evaluate the effects of alternative policies defined in Section 5.3.3: shared resources (SR), shared idle resources (SIR), and shared idle resources and workload (SIRW). First, in Fig. 5.13, we compare the rewards assigned in three federation scenarios, each with three private clouds C_1 , C_2 and C_3 .

Federation 1. In this federation, $\vec{\lambda} = (10, 70, 140)$, $\vec{n} = (30, 50, 100)$, $\vec{m} = \vec{n}$ (all servers are shared), $\vec{Q} = (0, 0, 0)$ (no queueing is allowed): C_1 is overprovisioned (contributing most idle servers), while C_2 and C_3 are similarly underprovisioned; notably, C_3 contributes more servers and workload. SV assigns most cloud savings to C_1 , the only provider of idle servers; C_2 and C_3 are rewarded similarly, with higher reward assigned to C_3 due to its higher shared workload. In contrast, SR unfairly rewards C_3 for sharing most servers, although its servers have very high utilization and cannot serve workload from other members of the federation. SIR addresses this issue by taking resource utilization into account: most of the reward goes to C_1 for sharing idle servers. This assignment is also unfair, since the workload shared by C_2 and C_3 is necessary to obtain public cloud savings; SIRW accounts for shared workload, obtaining a more balanced reward distribution.

Federation 2. In this federation, $\vec{\lambda} = (30, 30, 150)$, $\vec{n} = (50, 100, 50)$, $\vec{m} = \vec{n}$ (all servers are shared), $\vec{Q} = (0, 0, 0)$ (no queueing is allowed): C_1 and C_2 are overprovisioned, while C_3 is heavily underprovisioned. SV assigns most cloud savings to C_3 (for sharing its workload) and to C_2 (for sharing more servers than C_1 , and with lower utilization). SR rewards C_2 for sharing most servers, and C_2 's reward is even greater with SIR, since its shared servers have low utilization; instead, the reward of C_3 (providing workload) is much lower, especially with SIR. SIRW achieves a reward distribution very similar to SV, illustrating that Shapley value is rewarding members of the federation for both shared workload and servers.

Federation 3. In this federation, $\vec{\lambda} = (50, 50, 200)$, $\vec{n} = (50, 50, 200)$, $\vec{m} = \vec{n}$ (all servers are shared), $\vec{Q} = (0, 0, 1)$: all private clouds have similar loads (close to underprovisioning), but

C_3 can queue requests, with a buffer of up to $s_F\mu Q = 300$ requests. The shared federation pool will not be available to serve traffic of private clouds C_1 and C_2 , which will forward most of their requests to the public cloud, resulting in an *increase* of public cloud costs. In contrast, a federation including only C_1 and C_2 results in public cloud savings. For this reason, SV assigns negative reward to C_3 (its presence reduces the profit of the federation) and positive reward to C_1 and C_2 (their presence increases the profit of the federation). Other policies are not able to distinguish between the negative contribution of C_3 and the positive contributions of C_1 and C_2 .

Next, we analyze profit when public cloud savings are distributed according to Shapley value and up to 5 private clouds $C_1, C_2, \dots, C_5$ join the federation. In this setting, we have $\vec{\lambda} = (10, 50, 30, 50, 100)$, $\vec{n} = (30, 30, 30, 50, 100)$, and $\vec{Q} = (0, 0, 0, 0, 1)$; we assume that all servers are shared after joining the federation ($\vec{n} = \vec{m}$). The results, illustrated in Fig. 5.14, show that, as we add more clouds to the federation, C_1 and C_2 are still responsible for most of the public cloud savings, since C_1 is overprovisioned while C_2 is underprovisioned. Instead, clouds C_3 and C_4 have similar loads before joining the federation, and they are responsible for a lower fraction of public cloud savings, thus receiving lower rewards from Shapley value. In contrast, cloud C_5 has negative reward (i.e., it is charged to be part of the federation): due to its higher allowed mean waiting time $Q_5 = 1$, requests of C_5 can use servers from the shared pool more frequently; this reduces the ability of C_1 and C_2 to share their resources and workload, respectively, and hence C_5 is penalized by Shapley value.

5.5 Conclusions

This chapter presented a CTMC model to predict the rate of requests forwarded to public cloud providers by a federation of hybrid clouds, where each member can share workload or resources to generate cost savings while satisfying QoS requirements. When all members have the same QoS requirements, we provided theoretical results showing that sharing all

resources is the best strategy. For heterogeneous QoS requirements, we provided a solution to evaluate different sharing strategies, illustrating how sharing all resources can be, in fact, counterproductive for the members of the federation.

As a cooperation mechanism, the chapter uses Shapley value to share public cloud savings. Through our experimental evaluation, we compared Shapley value with alternative sharing policies rewarding members of the federation for the amount of shared resources and workload. The results illustrate the ability of Shapley value to reward both, and to discourage (through negative payoffs) members of the federation from reducing its cloud savings.

Clinics, research labs, and similar places that keep sensitive data local may also want to share computers without giving up control of their own systems. FedSER addresses the data side of that setting; this chapter addresses resource sharing under QoS constraints, using the CTMC model above.

The theoretical characterization in this chapter is limited to the workload and resource assumptions of the model. Heterogeneous workloads and multiple classes of VM instances introduce additional tradeoffs between performance, cost, and availability that are outside the scope of the analysis.

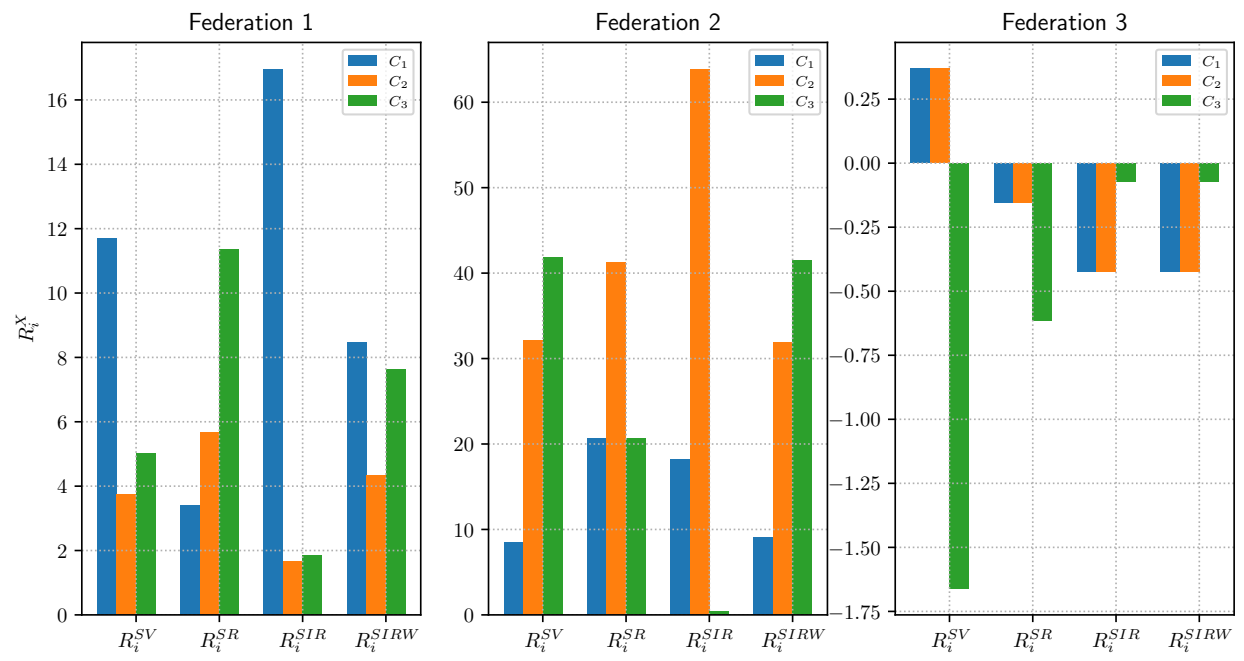


Figure 5.13: Comparison of different reward policies.

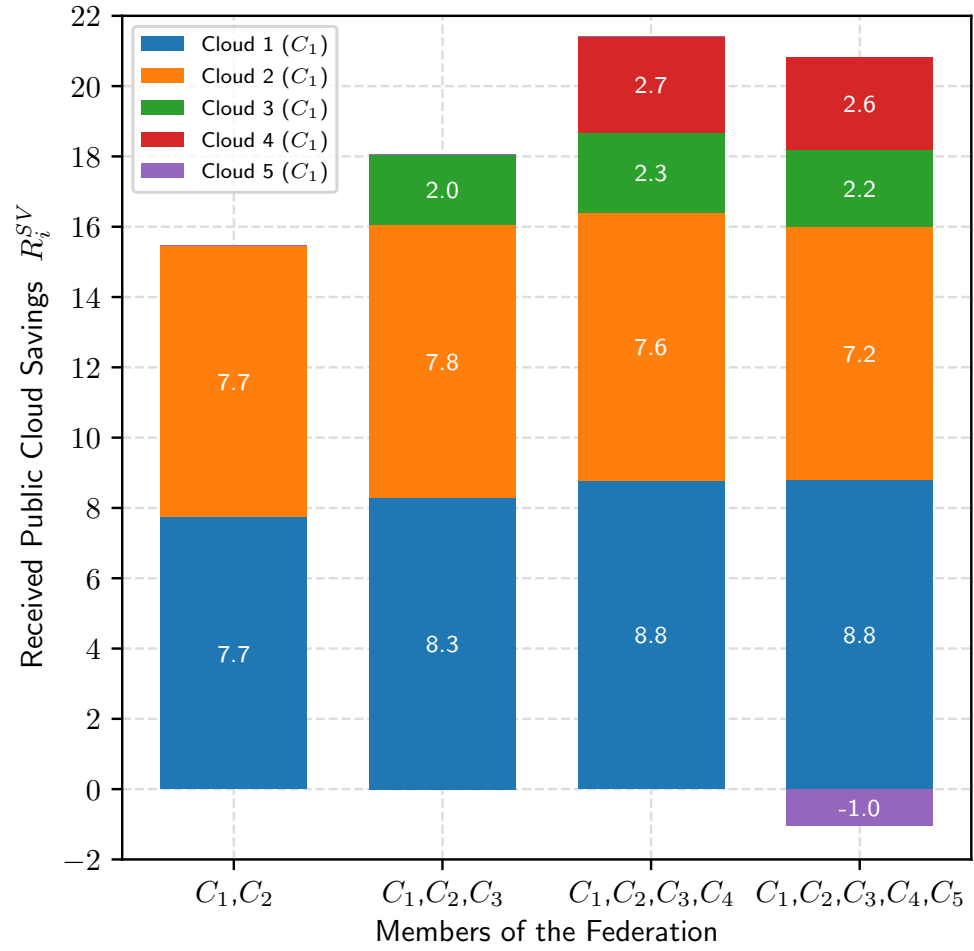


Figure 5.14: Reward distributed for different federations.

Chapter 6

Conclusion

This dissertation studied biomechanical machine learning in settings where sensing, data ownership, and computation are distributed. Ground reaction forces and joint moments are essential for understanding human movement, but the instruments that measure them directly are expensive and laboratory-bound. Wearable IMUs make broader deployment possible, but they require models that can estimate biomechanical targets from indirect signals while remaining fast, data-efficient, and careful with private data.

The first contribution, SER, uses low-rank structure in biomechanical time series. Learning a regression map between SVD embeddings of IMU and GRF signals, SER matched or exceeded LSTM accuracy for both GRF waveforms and the biomechanical variables derived from them, while estimating 10 running steps in 0.3 ms on a smartphone, more than four orders of magnitude faster than an LSTM, and with far lower training cost and energy. Personal training data reduced error further, especially for the derived variables.

The second contribution, FedSER, extends this low-rank approach to federated multi-task learning. Keeping raw data on each client, FedSER learns shared bases and embedding regressors and combines action-specific experts with a global expert. Federated training matches centralized SER and outperforms TCN on hip and knee moments and vertical ground reaction forces across training-set sizes, including action-within-training and leave-one-action-out evaluation. FedSER trains directly on phones in minutes, whereas a centralized TCN requires a cloud GPU; in an 11-client experiment with one physical Google Pixel 8 Pro and ten simulated clients, the phone completes training in about 3 minutes versus 40.8 minutes for TCN on an A100, and runs inference in 0.16 ms per sample versus 71 ms. FedSER is compatible with secure aggregation, which hides individual updates at no accuracy cost, and with partially homomorphic encryption in principle (not separately

timed here); it retains accuracy comparable to that of non-private TCN under differential privacy. For hip flexion moment as an example, projection-based recovery reduces RMSE when individual IMU channels are missing at test time if the test action was seen in training. Under leave-one-action-out evaluation the recovery is about neutral, because the learned input subspace does not include the held-out action.

The third contribution is a systems study of sharing compute. Clinics, research labs, and sports performance centers that keep data local may also want to share machines when demand is high. Using a continuous-time Markov chain for a federation of hybrid clouds, we showed that sharing all resources is best when QoS requirements match, can hurt when they differ, and that Shapley value can divide savings so that free riding is discouraged. This chapter is a queueing and cost-sharing analysis; it does not use the biomechanics workloads.

Overall, the biomechanics chapters show that low-rank models and federated learning can be accurate, private, and efficient on edge devices. The cloud chapter shows when independent organizations benefit from sharing compute under QoS constraints.

Limitations and Future Work

Several limitations bound how far the present results should be generalized. The SER study uses treadmill running from a collegiate cohort with a fixed sensor layout; outdoor free-running, clinical gait, and other populations may change both accuracy and the value of personalization. FedSER is evaluated on a multi-action laboratory dataset with eleven participants after cohort filtering, so conclusions about leave-one-action-out transfer and data scarcity depend on that action distribution and cohort size. Differential privacy is studied primarily through a shared dataset-release protocol under a fixed δ with the privacy budget split across features and targets; reported curves for small ϵ reflect that eleven-subject scale as much as any single algorithm. The cloud chapter models hybrid federations as $M/M/n/m$ queues with homogeneous servers and focuses on public-cloud overflow savings; real clusters with heterogeneous machines, admission control, and non-exponential workloads may require

richer models before the policies are deployed.

Natural next steps follow from these limits: continual on-device adaptation of FedSER as new labeled trials arrive; stronger privacy accounting that reports end-to-end (ϵ, δ) including clipping-threshold estimation; broader wearable placements and outdoor validation for SER; and CTMC or simulation studies that couple QoS federations with the bursty training and inference loads of edge biomechanics systems.

Bibliography

- [1] David A Winter. *Biomechanics and Motor Control of Human Movement*. 4th. A foundational text establishing force plates and motion capture as the gold-standard methods for kinetic and kinematic analysis in biomechanics. John Wiley & Sons, 2009.
- [2] Carolyn F. Munro, Doris I. Miller, and Andrew J. Fuglevand. “Ground reaction forces in running: a reexamination”. In: *Journal of Biomechanics* 20.2 (1987), pp. 147–155.
- [3] Peter R. Cavanagh and Mario A. LaFortune. “Ground reaction forces in distance running”. In: *Journal of Biomechanics* 13.5 (1980), pp. 397–406.
- [4] Stanley L. James, Barry T. Bates, and Louis R. Osternig. “Injuries to runners”. In: *The American Journal of Sports Medicine* 6.2 (1978), pp. 40–50. DOI: 10.1177/036354657800600202.
- [5] Alan Hreljac. “Impact and overuse injuries in runners”. In: *Medicine and Science in Sports and Exercise* 36.5 (2004), pp. 845–849.
- [6] C. Napier, C.L. MacLean, J. Maurer, J.E. Taunton, and M.A. Hunt. “Kinetic risk factors of running-related injuries in female recreational runners”. In: *Scandinavian Journal of Medicine & Science in Sports* 28.10 (2018), pp. 2164–2172.
- [7] Caleb D Johnson, Adam S Tenforde, Jereme Outerleys, Julia Reilly, and Irene S Davis. “Impact-related ground reaction forces are more strongly associated with some running injuries than others”. In: *The American Journal of Sports Medicine* 48.12 (2020), pp. 3072–3080.
- [8] C Nathan Vannatta, Becky L Heinert, and Thomas W Kernozek. “Biomechanical risk factors for running-related injury differ by sample population: A systematic review and meta-analysis”. In: *Clinical Biomechanics* 75 (2020), p. 104991.
- [9] John Bigouette, Janet Simon, Kathy Liu, and Carrie L. Docherty. “Altered vertical ground reaction forces in participants with chronic ankle instability while running”. In: *Journal of Athletic Training* 51.9 (2016), pp. 682–687.
- [10] Dovin Kiernan, David A. Hawkins, Martin A.C. Manoukian, Madeline McKallip, Laura Oelsner, Charles F. Caskey, and Crystal L. Coolbaugh. “Accelerometer-based prediction of running injury in National Collegiate Athletic Association track athletes”. In: *Journal of Biomechanics* 73 (2018), pp. 201–209.

- [11] Stephen P. Messier, David F. Martin, Shannon L. Mihalko, Edward Ip, Paul DeVita, D. Wayne Cannon, Monica Love, Danielle Beringer, Santiago Saldana, Rebecca E. Fellin, and Joseph F. Seay. “A 2-year prospective cohort study of overuse running injuries: the runners and injury longitudinal study (TRAILS)”. In: *The American Journal of Sports Medicine* 46.9 (2018), pp. 2211–2221. DOI: 10.1177/0363546518773755.
- [12] Chand T John, Ajay Seth, Michael H Schwartz, and Scott L Delp. “Contributions of muscles to mediolateral ground reaction force over a range of walking speeds”. In: *Journal of biomechanics* 45.14 (2012), pp. 2438–2443.
- [13] Dean D. Molinaro, Inseung Kang, Jonathan Camargo, Matthew C. Gombolay, and Aaron J. Young. “Subject-Independent, Biological Hip Moment Estimation During Multimodal Overground Ambulation Using Deep Learning”. In: *IEEE Transactions on Medical Robotics and Bionics* 4.1 (2022), pp. 219–229. DOI: 10.1109/TMRB.2022.3144025.
- [14] Dean D Molinaro, Keaton L Scherpereel, Ethan B Schonhaut, Georgios Evangelopoulos, Max K Shepherd, and Aaron J Young. “Task-agnostic exoskeleton control via biological joint moment estimation”. In: *Nature* 635.8038 (2024), pp. 337–344.
- [15] Keaton L Scherpereel, Dean D Molinaro, Max K Shepherd, Omer T Inan, and Aaron J Young. “Improving biological joint moment estimation during real-world tasks with EMG and instrumented insoles”. In: *IEEE Transactions on Biomedical Engineering* 71.9 (2024), pp. 2718–2727.
- [16] Patrick O. Riley, Jay Dicharry, Jason Franz, Ugo Della Croce, Robert P. Wilder, and D. Casey Kerrigan. “A kinematics and kinetic comparison of overground and treadmill running”. In: *Medicine & Science in Sports & Exercise* 40.6 (2008), pp. 1093–1100.
- [17] Bas Kluitenberg, Steef W. Bredeweg, Sjouke Zijlstra, Wiebren Zijlstra, and Ida Buist. “Comparison of vertical ground reaction forces during overground and treadmill running. A validation study”. In: *BMC Musculoskeletal Disorders* 13.1 (2012), pp. 1–8.
- [18] Michael J Asmussen, Christina Kaltenbach, Kotaybah Hashlamoun, Hua Shen, Salvatore Federico, and Benno M Nigg. “Force measurements during running on different instrumented treadmills”. In: *Journal of Biomechanics* 84 (2019), pp. 263–268.
- [19] Jana Holder, Ursula Trinler, Andrea Meurer, and Felix Stief. “A systematic review of the associations between inverse dynamics and musculoskeletal modeling to investigate joint loading in a clinical environment”. In: *Frontiers in bioengineering and biotechnology* 8 (2020), p. 603907.

- [20] Muhammad Abdullah, Abdul Aziz Hulleck, Rateb Katmah, Kinda Khalaf, and Marwan El-Rich. “Multibody dynamics-based musculoskeletal modeling for gait analysis: a systematic review”. In: *Journal of neuroengineering and rehabilitation* 21.1 (2024), p. 178.
- [21] Daniel A. Jacobs and Daniel P. Ferris. “Estimation of ground reaction forces and ankle moment with multiple, low-cost sensors”. In: *Journal of Neuroengineering and Rehabilitation* 12.1 (2015), pp. 1–12.
- [22] Rachel Mason, Liam T Pearson, Gillian Barry, Fraser Young, Oisin Lennon, Alan Godfrey, and Samuel Stuart. “Wearables for running gait analysis: A systematic review”. In: *Sports Medicine* 53.1 (2023), pp. 241–268.
- [23] Reed D Gurchiek, Nick Cheney, and Ryan S McGinnis. “Estimating biomechanical time-series with wearable sensors: A systematic review of machine learning techniques”. In: *Sensors* 19.23 (2019), p. 5227.
- [24] Lei Ren, Richard K Jones, and David Howard. “Whole body inverse dynamics over a complete gait cycle based only on measured kinematics”. In: *Journal of Biomechanics* 41.12 (2008), pp. 2750–2759.
- [25] Eni Halilaj, Apoorva Rajagopal, Madalina Fiterau, Jennifer L Hicks, Trevor J Hastie, and Scott L Delp. “Machine learning in human movement biomechanics: Best practices, common pitfalls, and new opportunities”. In: *Journal of biomechanics* 81 (2018), pp. 1–11.
- [26] Liangliang Xiang, Alan Wang, Yaodong Gu, Liang Zhao, Vickie Shim, and Justin Fernandez. “Recent machine learning progress in lower limb running biomechanics with wearable technology: a systematic review”. In: *Frontiers in neurorobotics* 16 (2022), p. 913052.
- [27] Eva Dorschky, Marlies Nitschke, Christine F. Martindale, Antonie J. van den Bogert, Anne D. Koelewijn, and Bjoern M. Eskofier. “CNN-Based Estimation of Sagittal Plane Walking and Running Biomechanics From Measured and Simulated Inertial Sensor Data”. In: *Frontiers in Bioengineering and Biotechnology* 8 (2020). ISSN: 2296-4185. DOI: 10.3389/fbioe.2020.00604.
- [28] Ryan S. Alcantara, W. Brent Edwards, Guillaume Y. Millet, and Alena M. Grabowski. “Predicting continuous ground reaction forces from accelerometers during uphill and downhill running: a recurrent neural network solution”. In: *PeerJ* 10 (Jan. 2022), e12752. ISSN: 2167-8359. DOI: 10.7717/peerj.12752.

- [29] William R Johnson, Ajmal Mian, Mark A Robinson, Jasper Verheul, David G Lloyd, and Jacqueline A Alderson. “Multidimensional ground reaction forces and moments from wearable sensor accelerations via deep learning”. In: *IEEE Transactions on Biomedical Engineering* 68.1 (2021), pp. 289–297. DOI: 10.1109/TBME.2020.3006158.
- [30] Colin Lea, Michael D Flynn, Rene Vidal, Austin Reiter, and Gregory D Hager. “Temporal convolutional networks for action segmentation and detection”. In: *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 156–165.
- [31] Cynthia Rudin. “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead”. In: *Nature Machine Intelligence* 1.5 (2019), pp. 206–215. DOI: 10.1038/s42256-019-0048-x.
- [32] Lizhi Liao, Heng Li, Weiyi Shang, and Lei Ma. “An Empirical Study of the Impact of Hyperparameter Tuning and Model Optimization on the Performance Properties of Deep Neural Networks”. In: *ACM Transactions on Software Engineering and Methodology* 31.3 (2022), pp. 1–40. DOI: 10.1145/3506695.
- [33] Roy Schwartz, Jesse Dodge, Noah A Smith, and Oren Etzioni. “Green AI”. In: *Communications of the ACM* 63.12 (2020), pp. 54–63. DOI: 10.1145/3381831.
- [34] Emma Strubell, Ananya Ganesh, and Andrew McCallum. “Energy and policy considerations for deep learning in NLP”. In: *ACL*. 2019, pp. 3645–3650. DOI: 10.18653/v1/P19-1355.
- [35] Gene H Golub and Charles F Van Loan. *Matrix Computations*. 4th ed. Johns Hopkins University Press, 2013. ISBN: 9781421407944. DOI: 10.56021/9781421407944.
- [36] Ian T Jolliffe and Jorge Cadima. “Principal component analysis: a review and recent developments”. In: *Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences* 374.2065 (2016), p. 20150202.
- [37] Eric Bair, Trevor Hastie, Debashis Paul, and Robert Tibshirani. “Prediction by supervised principal components”. In: *Journal of the American Statistical Association* 101.473 (2006), pp. 119–137.
- [38] Simone Venturi and Tiernan Casey. “SVD perspectives for augmenting DeepONet flexibility and interpretability”. In: *Computer Methods in Applied Mechanics and Engineering* 403 (2023), p. 115718.

- [39] Qing Liao and Qian Zhang. “Efficient rank-one residue approximation method for graph regularized non-negative matrix factorization”. In: *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part II 13*. Springer. 2013, pp. 242–255.
- [40] Corinna Cortes, Mehryar Mohri, and Jason Weston. “A general regression technique for learning transductions”. In: *ICML 2005*. Vol. 119. ACM International Conference Proceeding Series. ACM, 2005, pp. 153–160. DOI: 10.1145/1102351.1102371.
- [41] Salvatore Tedesco, Eduardo Pérez-Valero, Dimitrios-Sokratis Komaris, Luke Jordan, John Barton, Liam Hennessy, and Brendan O’Flynn. “Wearable motion sensors and artificial neural network for the estimation of vertical ground reaction forces in running”. In: *IEEE Sensors*. IEEE, 2020, pp. 1–4. DOI: 10.1109/SENSORS47125.2020.9278796.
- [42] William R. Johnson, Ajmal Mian, Mark A. Robinson, Jasper Verheul, David G. Lloyd, and Jacqueline A. Alderson. “Multidimensional Ground Reaction Forces and Moments From Wearable Sensor Accelerations via Deep Learning”. In: *IEEE Trans. Biomed. Eng.* 68.1 (2021), pp. 289–297. DOI: 10.1109/TBME.2020.3006158. URL: <https://doi.org/10.1109/TBME.2020.3006158>.
- [43] Frank J Wouda, Matteo Giuberti, Giovanni Bellusci, Erik Maartens, Jasper Reenalda, Bert-Jan F Van Beijnum, and Peter H Veltink. “Estimation of vertical ground reaction forces and sagittal knee kinematics during running using three inertial sensors”. In: *Frontiers in Physiology* 9 (2018), p. 218.
- [44] Bouke L Scheltinga, Joost N Kok, Jaap H Buurke, and Jasper Reenalda. “Estimating 3D ground reaction forces in running using three inertial measurement units”. In: *Frontiers in Sports and Active Living* 5 (2023), p. 1176466.
- [45] Mark Pogson, Jasper Verheul, Mark A Robinson, Jos Vanrenterghem, and Paulo Lisboa. “A neural network method to predict task-and step-specific ground reaction force magnitudes from trunk accelerations during running activities”. In: *Medical Engineering & Physics* 78 (2020), pp. 82–89.
- [46] Seth R Donahue and Michael E Hahn. “Estimation of gait events and kinetic waveforms with wearable sensors and machine learning when running in an unconstrained environment”. In: *Scientific Reports* 13.1 (2023), p. 2339.
- [47] Rich Caruana. “Multitask learning”. In: *Machine learning* 28.1 (1997), pp. 41–75.
- [48] Sebastian Ruder. “An overview of multi-task learning in deep neural networks”. In: *arXiv preprint arXiv:1706.05098* (2017).

- [49] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. “Modeling task relationships in multi-task learning with multi-gate mixture-of-experts”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2018, pp. 1930–1939.
- [50] W Nicholson Price and I Glenn Cohen. “Privacy in the age of medical big data”. In: *Nature medicine* 25.1 (2019), pp. 37–43.
- [51] Adam Sadilek, Luyang Liu, Dung Nguyen, Methun Kamruzzaman, Stylianos Serghiou, Benjamin Rader, Alex Ingerman, Stefan Mellem, Peter Kairouz, Elaine O. Nsoesie, Jamie MacFarlane, Anil Vullikanti, Madhav Marathe, Paul Eastham, John S. Brownstein, Blaise Agüera y Arcas, Michael D. Howell, and John Hernandez. “Privacy-first health research with federated learning”. In: *NPJ digital medicine* 4.1 (2021), p. 132. DOI: 10.1038/s41746-021-00489-2.
- [52] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. “Communication-efficient learning of deep networks from decentralized data”. In: *Artificial intelligence and statistics*. PMLR. 2017, pp. 1273–1282.
- [53] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. “Federated learning: Challenges, methods, and future directions”. In: *IEEE Signal Processing Magazine* 37.3 (2020), pp. 50–60. DOI: 10.1109/MSP.2020.2975749.
- [54] Nicola Rieke, Jonny Hancox, Wenqi Li, Fausto Milletari, Holger R. Roth, Shadi Albarqouni, Spyridon Bakas, Mathieu N. Galtier, Bennett A. Landman, Klaus Maier-Hein, Sébastien Ourselin, Micah Sheller, Ronald M. Summers, Andrew Trask, Daguang Xu, Maximilian Baust, and M. Jorge Cardoso. “The future of digital health with federated learning”. In: *NPJ Digital Medicine* 3.1 (2020), p. 119. DOI: 10.1038/s41746-020-00323-1.
- [55] Keaton Scherpereel, Dean Molinaro, Omer Inan, Max Shepherd, and Aaron Young. “A human lower-limb biomechanics and wearable sensors dataset during cyclic and non-cyclic activities”. In: *Scientific Data* 10.1 (2023), p. 924.
- [56] Wei Yang Bryan Lim, Nguyen Cong Luong, Dinh Thai Hoang, Yutao Jiao, Ying-Chang Liang, Qiang Yang, Dusit Niyato, and Chunyan Miao. “Federated learning in mobile edge networks: A comprehensive survey”. In: *IEEE communications surveys & tutorials* 22.3 (2020), pp. 2031–2063.

- [57] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Ben-
nis, Arjun Nitin Bhagoji, Kallista A. Bonawitz, Zachary Charles, Graham Cormode,
Rachel Cummings, Rafael G. L. D'Oliveira, Hubert Eichner, Salim El Rouayheb,
David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badih Ghazi, Phillip
B. Gibbons, Marco Gruteser, Zaïd Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo,
Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak,
Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrede
Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Ras-
mus Pagh, Mariana Raykova, Hang Qi, Daniel Ramage, Ramesh Raskar, Dawn Song,
Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian
Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Fel-
lix X. Yu, Han Yu, and Sen Zhao. “Advances and open problems in federated learn-
ing”. In: *Foundations and Trends in Machine Learning* 14.1–2 (2021), pp. 1–210. DOI:
10.1561/22000000083.
- [58] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. “Exp-
loiting unintended feature leakage in collaborative learning”. In: *2019 IEEE sympo-
sium on security and privacy (SP)*. IEEE. 2019, pp. 691–706.
- [59] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller. “Invert-
ing gradients-how easy is it to break privacy in federated learning?” In: *Advances in
neural information processing systems* 33 (2020), pp. 16937–16947.
- [60] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. “Membership
inference attacks against machine learning models”. In: *2017 IEEE Symposium on
Security and Privacy (SP)*. IEEE. 2017, 3–18. DOI: 10.1109/SP.2017.41.
- [61] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan
McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. “Practical
Secure Aggregation for Privacy-Preserving Machine Learning”. In: *Proceedings of the
2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*.
2017, 1175–1191. DOI: 10.1145/3133956.3133982.
- [62] Chengliang Zhang, Suyi Li, Junzhe Xia, Wei Wang, Feng Yan, and Yang Liu.
“BatchCrypt: Efficient homomorphic encryption for Cross-Silo federated learning”.
In: *USENIX ATC*. 2020, pp. 493–506.
- [63] Mohamad Mansouri, Melek Önen, Wafa Ben Jaballah, and Mauro Conti. “SoK: Secure
Aggregation Based on Cryptographic Schemes for Federated Learning”. In: *Proceed-
ings on Privacy Enhancing Technologies* 2023.1 (2023), pp. 140–157. DOI: 10.56553/
popets-2023-0009.
- [64] Cynthia Dwork and Aaron Roth. “The algorithmic foundations of differential pri-
vacy”. In: *Foundations and Trends in Theoretical Computer Science* 9.3–4 (2014),
pp. 211–487. DOI: 10.1561/04000000042.

- [65] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. “Calibrating noise to sensitivity in private data analysis”. In: *TCC*. Springer. 2006, pp. 265–284.
- [66] Yiqiang Chen, Xin Qin, Jindong Wang, Chaohui Yu, and Wen Gao. “Fedhealth: A federated transfer learning framework for wearable healthcare”. In: *IEEE Intelligent Systems* 35.4 (2020), pp. 83–93.
- [67] Dongzhou Cheng, Lei Zhang, Can Bu, Xing Wang, Hao Wu, and Aiguo Song. “Proto-HAR: Prototype guided personalized federated learning for human activity recognition”. In: *IEEE Journal of Biomedical and Health Informatics* 27.8 (2023), pp. 3900–3911.
- [68] Claudio Bettini, Gabriele Civitarese, and Riccardo Presotto. “Personalized semi-supervised federated learning for human activity recognition”. In: *arXiv preprint arXiv:2104.08094* (2021).
- [69] Yujing Chen, Yue Ning, Zheng Chai, and Huzefa Rangwala. “Federated multi-task learning with hierarchical attention for sensor data analytics”. In: *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE. 2020, pp. 1–8.
- [70] Qiang Shen, Haotian Feng, Rui Song, Stefano Teso, Fausto Giunchiglia, and Hao Xu. “Federated multi-task attention for cross-individual human activity recognition”. In: *IJCAI*. International Joint Conferences on Artificial Intelligence. 2022, pp. 3423–3429. DOI: 10.24963/ijcai.2022/475.
- [71] Egemen İsgüder and Özlem Durmaz İnel. “FedOpenHAR: Federated Multitask Transfer Learning for Sensor-Based Human Activity Recognition”. In: *Journal of Computational Biology* 32.6 (2025), pp. 558–572.
- [72] Jianqing Fan, Dong Wang, Kaizheng Wang, and Ziwei Zhu. “Distributed estimation of principal eigenspaces”. In: *Annals of statistics* 47.6 (2019), p. 3009.
- [73] Andreas Grammenos, Rodrigo Mendoza-Smith, Jon Crowcroft, and Cecilia Mascolo. “Federated Principal Component Analysis”. In: *NeurIPS*. Vol. 33. Advances in Neural Information Processing Systems. 2020.
- [74] Anne Hartebrodt, Richard Röttger, and David B Blumenthal. “Federated singular value decomposition for high-dimensional data”. In: *Data Mining and Knowledge Discovery* 38.3 (2024), pp. 938–975.
- [75] Xiang Li, Shusen Wang, Kun Chen, and Zhihua Zhang. “Communication-efficient distributed SVD via local power iterations”. In: *International Conference on Machine Learning*. PMLR. 2021, pp. 6504–6514.

- [76] Florian Tramer and Dan Boneh. “Differentially Private Learning Needs Better Features (or Much More Data)”. In: *ICLR*. 2021.
- [77] Martín Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. “Deep learning with differential privacy”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2016, 308–318. DOI: 10.1145/2976749.2978318.
- [78] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy H. Katz, Andy Konwinski, Gunho Lee, David A. Patterson, Ariel Rabkin, Ion Stoica, and Matei Zaharia. “A view of cloud computing”. In: *Commun. ACM* 53.4 (2010), pp. 50–58. DOI: 10.1145/1721654.1721672.
- [79] *OpenStack: Open Source Cloud Computing Infrastructure*. openstack.org.
- [80] *CloudStack: Open Source Cloud Computing*. cloudstack.apache.org.
- [81] *OpenNebula: Open Source Cloud and Edge Computing*. opennebula.io.
- [82] Borja Sotomayor, Rubén S. Montero, Ignacio Martín Llorente, and Ian T. Foster. “Virtual Infrastructure Management in Private and Hybrid Clouds”. In: *IEEE Internet Comput.* 13.5 (2009), pp. 14–22. DOI: 10.1109/MIC.2009.119.
- [83] Joe Weinman. “Hybrid Cloud Economics”. In: *IEEE Cloud Comput.* 3.1 (2016), pp. 18–22. DOI: 10.1109/MCC.2016.27.
- [84] Benny Rochwerger, David Breitgand, Eliezer Levy, Alex Galis, Kenneth Nagin, Ignacio Martín Llorente, Rubén S. Montero, Yaron Wolfsthal, Erik Elmroth, Juan A. Cáceres, Muli Ben-Yehuda, Wolfgang Emmerich, and Fermín Galán. “The Reservoir model and architecture for open federated cloud computing”. In: *IBM J. Res. Dev.* 53.4 (2009), p. 4. DOI: 10.1147/JRD.2009.5429058.
- [85] Rafael Moreno-Vozmediano, Rubén S. Montero, and Ignacio Martín Llorente. “IaaS Cloud Architecture: From Virtualized Datacenters to Federated Cloud Infrastructures”. In: *Computer* 45.12 (2012), pp. 65–72. DOI: 10.1109/MC.2012.76.
- [86] Sung-Han Lin, Ranjan Pal, Marco Paolieri, and Leana Golubchik. “Performance Driven Resource Sharing Markets for the Small Cloud”. In: *ICDCS 2017*. IEEE Computer Society, 2017, pp. 241–251. DOI: 10.1109/ICDCS.2017.272.
- [87] George Darzanos, Iordanis Koutsopoulos, and George D. Stamoulis. “Cloud Federations: Economics, Games and Benefits”. In: *IEEE/ACM Trans. Netw.* 27.5 (2019), pp. 2111–2124. DOI: 10.1109/TNET.2019.2943810.

- [88] Dario Bruneo. “A Stochastic Model to Investigate Data Center Performance and QoS in IaaS Cloud Computing Systems”. In: *IEEE Trans. Parallel Distributed Syst.* 25.3 (2014), pp. 560–569. DOI: 10.1109/TPDS.2013.67.
- [89] Leonard Kleinrock. *Queueing Systems, Vol. 1: Theory*. Wiley-Interscience, 1975. ISBN: 9780471491101.
- [90] Rahul Ghosh, Francesco Longo, Ruofan Xia, Vijay K. Naik, and Kishor S. Trivedi. “Stochastic Model Driven Capacity Planning for an Infrastructure-as-a-Service Cloud”. In: *IEEE Trans. Serv. Comput.* 7.4 (2014), pp. 667–680. DOI: 10.1109/TSC.2013.44.
- [91] Dusit Niyato, Athanasios V Vasilakos, and Zhu Kun. “Resource and revenue sharing with coalition formation of cloud providers: Game theoretic approach”. In: *2011 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. IEEE. 2011, pp. 215–224.
- [92] David R Smith and Ward Whitt. “Resource sharing for efficiency in traffic systems”. In: *Bell System Technical Journal* 60.1 (1981), pp. 39–55.
- [93] Alvin E. Roth, ed. *The Shapley Value: Essays in Honor of Lloyd S. Shapley*. Cambridge University Press, 1988. DOI: 10.1017/CB09780511528446.
- [94] Kun Ma, Antoine Bagula, Hope Mauwa, and Antonio Celesti. “Modelling Cloud Federation: A Fair Profit Distribution Strategy Using the Shapley Value”. In: *2018 IEEE 6th International Conference on Future Internet of Things and Cloud (FiCloud)*. IEEE. 2018, pp. 393–398.
- [95] Keqin Li. “Profit Maximization in a Federated Cloud by Optimal Workload Management and Server Speed Setting”. In: *IEEE Transactions on Sustainable Computing* 7.3 (2022), pp. 668–680. DOI: 10.1109/TSUSC.2021.3126666.
- [96] Raouf Abozariba, Anas Amjad, and Mohammad Patwary. “Optimized Resource Sharing for Federated Cloud Services with Desired Performance and Limited OpEx”. In: *IEEE Global Communications Conference (GLOBECOM)*. 2017.
- [97] Jing Bi, Zhiliang Zhu, Ruixiong Tian, and Qingbo Wang. “Dynamic Provisioning Modeling for Virtualized Multi-tier Applications in Cloud Data Center”. In: *IEEE CLOUD 2010*. IEEE Computer Society, 2010, pp. 370–377. DOI: 10.1109/CLOUD.2010.53.
- [98] Marcio Roberto Miranda Assis and Luiz Fernando Bittencourt. “MultiCloud Tournament: A cloud federation approach to prevent Free-Riders by encouraging resource sharing”. In: *J. Netw. Comput. Appl.* 166 (2020), p. 102694.

- [99] Talal Halabi, Martine Bellaiche, and Adel Abusitta. “A cooperative game for online cloud federation formation based on security risk assessment”. In: *2018 5th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)/2018 4th IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom)*. IEEE. 2018, pp. 83–88.
- [100] Yisheng Wang and Haopeng Chen. “Dynamic resource arrangement in cloud federation”. In: *2012 IEEE Asia-Pacific Services Computing Conference*. IEEE. 2012, pp. 50–57.
- [101] Benay Kumar Ray, Avirup Saha, Sunirmal Khatua, and Sarbani Roy. “Toward maximization of profit and quality of cloud federation: solution to cloud federation formation problem”. In: *Journal of Supercomputing* 75.2 (2019), pp. 885–929.
- [102] Emily S Matijevich, Lauren M Branscombe, Leon R Scott, and Karl E Zelik. “Ground reaction force metrics are not strongly correlated with tibial bone load when running across speeds and slopes: Implications for science, sport and wearable tech”. In: *PloS one* 14.1 (2019), e0210000.
- [103] Hannah Rice, Markus Kurz, Patrick Mai, Leon Robertz, Kevin Bill, Timothy R Derrick, and Steffen Willwacher. “Speed and surface steepness affect internal tibial loading during running”. In: *Journal of sport and health science* 13.1 (2024), pp. 118–124.
- [104] Gustavo Leporace, Luiz Alberto Batista, and Jurandir Nadal. “Prediction of 3D ground reaction forces during gait based on accelerometer data”. In: *Research on Biomedical Engineering* 34 (2018), pp. 211–216.
- [105] Ryan S. Alcantara, Evan M. Day, Michael E. Hahn, and Alena M. Grabowski. “Sacral acceleration can predict whole-body kinetics and stride kinematics across running speeds”. In: *PeerJ* 9 (Apr. 2021), e11199. ISSN: 2167-8359. DOI: 10.7717/peerj.11199.
- [106] Dimitrios-Sokratis Komaris, Eduardo Pérez-Valero, Luke Jordan, John Barton, Liam Hennessy, Brendan O’Flynn, and Salvatore Tedesco. “Predicting three-dimensional ground reaction forces in running by using artificial neural networks and lower body kinematics”. In: *IEEE Access* 7 (2019), pp. 156779–156786.
- [107] NVIDIA. *NVIDIA TITAN X Graphics Card*. 2023. URL: <https://www.nvidia.com/en-us/geforce/products/10series/titan-x-pascal/>. (accessed: 04.28.2023).
- [108] NVIDIA. *GeForce GTX TITAN X — Specifications*. 2023. URL: <https://www.nvidia.com/en-us/geforce/graphics-cards/geforce-gtx-titan-x/specifications/>. (accessed: 04.28.2023).

- [109] Intel Corporation. *Intel Core i7-6800k Processor*. 2023. URL: <https://www.intel.com/content/www/us/en/products/sku/94189/intel-core-i76800k-processor-15m-cache-up-to-3-60-ghz/specifications.html>. (accessed: 04.28.2023).
- [110] Amazon EC2. *Amazon EC2 G4 Instances*. 2023. URL: <https://aws.amazon.com/ec2/instance-types/g4/>. (accessed: 04.28.2024).
- [111] Amazon EC2. *Amazon EC2 On-Demand Pricing*. 2023. URL: <https://aws.amazon.com/ec2/pricing/on-demand/>. (accessed: 04.28.2024).
- [112] Bruno Bazuelo-Ruiz, Juan V Durá-Gil, Nicolás Palomares, Enrique Medina, and Salvador Llana-Belloch. “Effect of fatigue and gender on kinematics and ground reaction forces variables in recreational runners”. In: *PeerJ* 6 (2018), e4489.
- [113] Zhepeng Yan, Val Tannen, and Zachary G Ives. “Fine-grained provenance for linear algebra operators”. In: *Proceedings of the 8th USENIX Conference on Theory and Practice of Provenance*. 2016, pp. 1–6.
- [114] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. “Learning deep features for discriminative localization”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 2921–2929.
- [115] Bowen Song, Marco Paolieri, Harper E. Stewart, Leana Golubchik, Jill L. McNitt-Gray, Vishal Misra, and Devavrat Shah. “Estimating Ground Reaction Forces From Inertial Sensors”. In: *IEEE Transactions on Biomedical Engineering* 72.2 (2025), pp. 595–608. DOI: 10.1109/TBME.2024.3465373.
- [116] Tian Tan, Tom Van Wouwe, Keenon F Werling, C Karen Liu, Scott L Delp, Jennifer L Hicks, and Akshay S Chaudhari. “GaitDynamics: A generative foundation model for analyzing human walking and running”. In: *Nature Biomedical Engineering* (2026), pp. 1–13.
- [117] David Froelicher, Hyunghoon Cho, Manaswitha Edupalli, João Sá Sousa, Jean-Philippe Bossuat, Apostolos Pyrgelis, Juan R. Troncoso-Pastoriza, Bonnie Berger, and Jean-Pierre Hubaux. “Scalable and Privacy-Preserving Federated Principal Component Analysis”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2023. DOI: 10.1109/SP46215.2023.10179350.
- [118] Di Chai, Leye Wang, Junxue Zhang, Liu Yang, Shuowei Cai, Kai Chen, and Qiang Yang. “Practical Lossless Federated Singular Vector Decomposition over Billion-Scale Data”. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 2022, pp. 46–55. DOI: 10.1145/3534678.3539402.

- [119] Wuxuan Jiang, Cong Xie, and Zhihua Zhang. “Wishart mechanism for differentially private principal components analysis”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 30. 1. 2016.
- [120] Christopher A. Choquette-Choo, H. Brendan McMahan, Keith Rush, and Abhradeep Thakurta. “Multi-Epoch Matrix Factorization Mechanisms for Private Machine Learning”. In: *ICML*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 5924–5963.
- [121] Bowen Liu, Balázs Pej3, and Qiang Tang. “Privacy-preserving federated singular value decomposition”. In: *Applied Sciences* 13.13 (2023), p. 7373.
- [122] Ethan B Schonhaut, Keaton L Scherpereel, and Aaron J Young. “Is EMG Information Necessary for Deep Learning Estimation of Joint and Muscle Level States?” In: *IEEE Transactions on Biomedical Engineering* 73.1 (2026), pp. 67–77. DOI: 10.1109/TBME.2025.3577084.
- [123] Yassir Jedra and Devavrat Shah. *k-SVD with Gradient Descent*. 2025. arXiv: 2502.00320 [cs.LG]. URL: <https://arxiv.org/abs/2502.00320>.
- [124] Yousef Saad. *Numerical methods for large eigenvalue problems: revised edition*. SIAM, 2011.
- [125] Zhaojun Bai, James Demmel, Jack Dongarra, Axel Ruhe, and Henk van der Vorst. *Templates for the Solution of Algebraic Eigenvalue Problems*. SIAM, 2000. DOI: 10.1137/1.9780898719581.
- [126] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. “Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions”. In: *SIAM review* 53.2 (2011), pp. 217–288.
- [127] Constantin Philippenko, Kevin Scaman, and Laurent Massouli3. “In-depth Analysis of Low-rank Matrix Factorisation in a Federated Setting”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 19. 2025, pp. 19904–19912.
- [128] Kamalika Chaudhuri, Claire Monteleoni, and Anand D. Sarwate. “Differentially private empirical risk minimization”. In: *Journal of Machine Learning Research (JMLR)* 12 (2011), 1069–1103.
- [129] Mehmet Emre Gursoy, Ali Inan, Mehmet Ercan Nergiz, and Y3cel Saygin. “Differentially private nearest neighbor classification”. In: *Data Mining and Knowledge Discovery* 31.5 (2017), pp. 1544–1575.

- [130] Yuqing Zhu, Xiang Yu, Manmohan Chandraker, and Yu-Xiang Wang. “Private-kNN: Practical Differential Privacy for Computer Vision”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 11854–11862.
- [131] Nathaniel Schenker and Jane F. Gentleman. “On judging the significance of differences by examining the overlap between confidence intervals”. In: *The American Statistician* 55.3 (2001), pp. 182–186.
- [132] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.
- [133] NVIDIA. *NVIDIA A100 Tensor Core GPU*. Accessed May 2, 2026. 2026. URL: <https://www.nvidia.com/en-us/data-center/a100/>.
- [134] Google. *Specifications — Pixel 8 Pro*. Accessed May 2, 2026. 2026. URL: https://store.google.com/product/pixel_8_pro_specs.
- [135] Yilin Kang, Yong Liu, Ben Niu, Xinyi Tong, Likun Zhang, and Weiping Wang. “Input perturbation: A new paradigm between central and local differential privacy”. In: *arXiv preprint arXiv:2002.08570* (2020).
- [136] Robin C Geyer, Tassilo Klein, and Moin Nabi. “Differentially private federated learning: A client level perspective”. In: *arXiv preprint arXiv:1712.07557* (2017).
- [137] Cameron Fisher. “Cloud versus on-premise computing”. In: *American Journal of Industrial and Business Management* 8.09 (2018), p. 1991. DOI: 10.4236/ajibm.2018.89133.
- [138] Marta Kwiatkowska, Gethin Norman, and David Parker. “PRISM 4.0: Verification of Probabilistic Real-time Systems”. In: *Computer Aided Verification*. Vol. 6806. 2011, pp. 585–591. ISBN: 978-3-642-22109-5. DOI: 10.1007/978-3-642-22110-1_47.
- [139] Jennifer R Yong, Amy Silder, Kate L Montgomery, Michael Fredericson, and Scott L Delp. “Acute changes in foot strike pattern and cadence affect running parameters associated with tibial stress fractures”. In: *Journal of biomechanics* 76 (2018), pp. 1–7.

Appendix A

SER Supporting Material

A.1 Aligning Signals from Different Locations by Reference Events

To estimate GRFs from IMUs, timestamps and gait events need to match consistently across different signals and locations. For a model that estimates the entire stance in GRFs from signals of an entire stance in IMU signals, training the model would require supplying signals aligned by their corresponding steps. Similarly for models estimating one sample point at a time based on samples of other signals at the same time point, signals at different locations need to synchronize. Given acceleration and angular velocity measurements of an IMU sensor are synchronized, we manually aligned data from IMU sensors at different locations and GRF data from the treadmills (which was downsampled to 500 Hz to match the IMU frequency and normalized by the body weight of the athlete).

In our dataset, IMU sensors at different locations (sacrum and shanks) and GRFs measured from force plates are sampled by individual clocks. Although we do not observe any severely non-linear drifts, linear drifts and time delays are significantly affecting the alignment of gait events.

To align all gait events over a series of continuous running steps, the dataset includes reference events where the athlete jumps in-place before and after continuously running on an instrumented treadmill. Wearing all sensors, the athlete jumping in-place creates a unique signal pattern across all sensors with a sharp edge marking the time instance when both feet strike the ground after flight. An example of aligned signals at the jump reference is shown in Fig. A.1. We align both reference events before and after a run by shifting and linearly stretching the signals to correct time delays and linear drifts. We use magnitude

of GRFs as the referencing signal and edit IMU signals to align with GRFs. After aligning both reference events, signals for each running step between the references are also aligned.

After aligning data from different sensors, each running measurement is automatically split into steps by identifying the maxima of the correlation between the L2 norm of shank acceleration and a reference signal (a triangular signal with 100 ms duration followed by a zero signal of 100 ms, mimicking the patterns observed in acceleration signals). Steps of a running measurement are aligned by maximizing their pairwise correlation; then, a fixed delay is applied to all the steps in a measurement to maximize their mean correlation with the reference signal, in order to align them with steps of other measurements (i.e., at different running speeds or for different athletes). We manually check the alignment of signals for each foot contact by overlapping all steps from each run (an example of such a view is shown in Fig. A.2). Aligning these signals by the start and end points shows time drifts between signals are only linear and the overlapped view shows gait events within each step are aligned similarly. While there is no guarantee that each sample point across all signal types is aligned perfectly, models looking at an entire stance or a windowed signal greater than or equal to a stance should have a consistent amount of information.

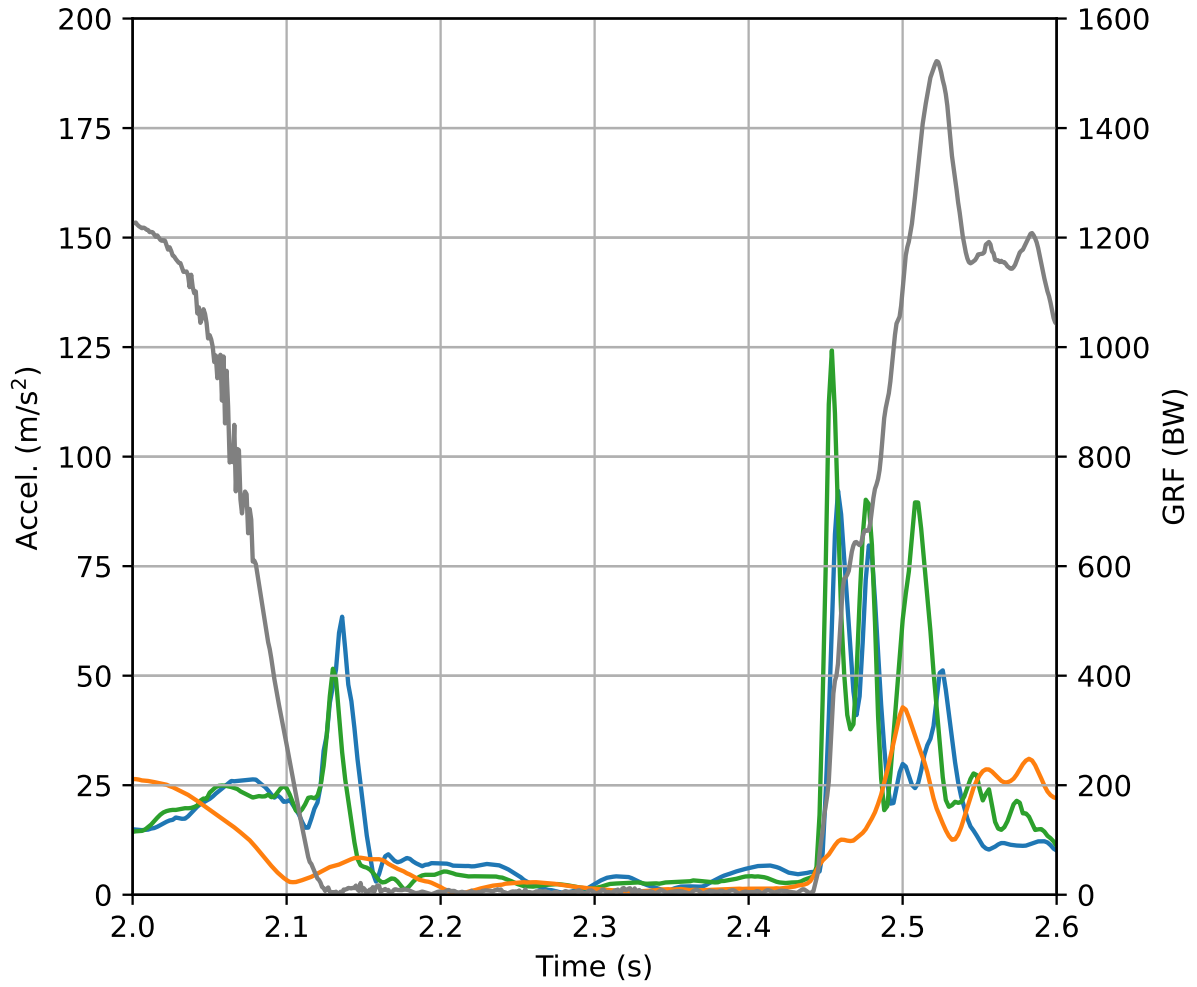


Figure A.1: Examples of overlapping the magnitude of GRFs with the magnitude of acceleration signals from shanks and sacrum. Blue is from left shank, green is from right shank, orange is from sacrum, and grey is from GRFs. (a) Aligned jump reference.

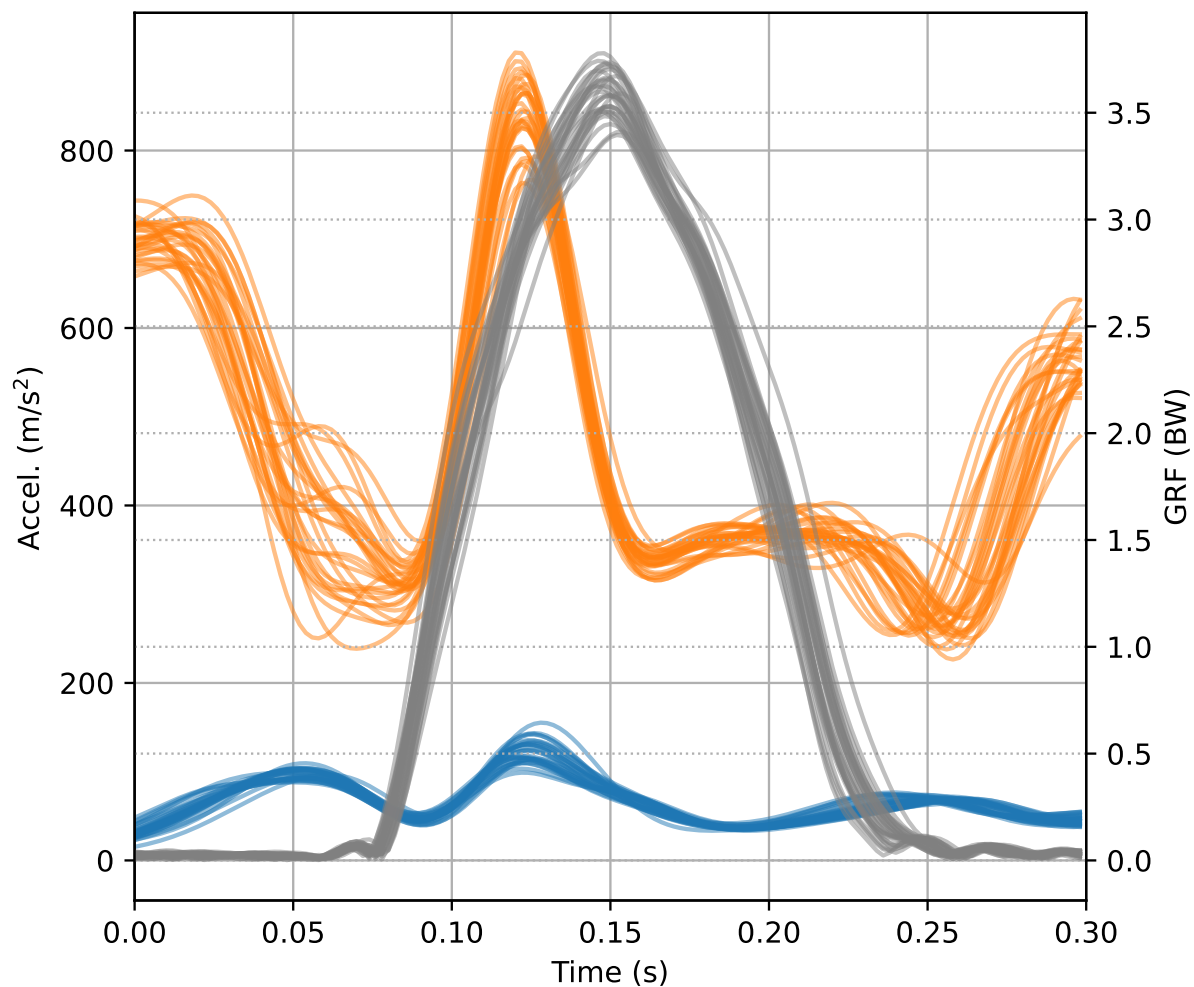


Figure A.2: Examples of overlapping the magnitude of GRFs with the magnitude of acceleration signals from shanks and sacrum. Blue is from left shank, green is from right shank, orange is from sacrum, and grey is from GRFs. (b) Consecutive left steps under a constant speed.

Input Signals	Scenario OTHERS			Scenario EVERYONE			Scenario PERSONAL		
	SER	KNN	LSTM	SER	KNN	LSTM	SER	KNN	LSTM
ALL	0.07	0.06	0.06	0.06	0.05	0.06	0.05	0.05	0.06
ACC	0.07	0.06	0.08	0.07	0.05	0.07	0.05	0.05	0.07
ANG	0.07	0.06	0.07	0.07	0.05	0.07	0.05	0.05	0.07
SHANK	0.07	0.06	0.08	0.07	0.05	0.07	0.05	0.05	0.07
SACRUM	0.08	0.07	0.12	0.07	0.05	0.10	0.05	0.05	0.08
SAC/ACC3D	0.07	0.06	0.07	0.07	0.05	0.07	0.05	0.05	0.08
SAC/ACC	0.07	0.07	0.08	0.07	0.05	0.08	0.05	0.05	0.11

Table A.1: RMSE (in body weight units, BW, i.e., N/kg) of anterior/posterior GRF estimations $RMSE(g_y, \hat{g}_y)$ for different input signals, data scenarios, machine learning methods

Input Signals	Scenario OTHERS			Scenario EVERYONE			Scenario PERSONAL		
	SER	KNN	LSTM	SER	KNN	LSTM	SER	KNN	LSTM
ALL	6.8	5.9	7.1	6.5	4.6	7.1	5.0	4.9	7.2
ACC	6.9	6.2	9.2	6.7	4.7	8.9	4.9	4.7	8.0
ANG	7.0	6.2	8.9	6.7	4.7	8.2	5.2	5.0	8.0
SHANK	7.1	6.4	9.2	7.0	5.1	8.4	5.3	4.9	8.5
SACRUM	7.5	7.4	16.3	7.3	4.9	13.1	5.2	5.0	9.3
SAC/ACC3D	7.1	6.4	9.0	6.9	4.6	8.9	4.8	4.8	9.8
SAC/ACC	7.2	6.8	9.6	6.9	4.9	9.7	5.0	5.0	15.5

Table A.2: rRMSE (%) of anterior/posterior GRF estimations $rRMSE(g_y, \hat{g}_y)$ for different input signals, data scenarios, machine learning methods

A.2 Discrete Biomechanical Variables

We consider the following discrete biomechanical variables (or *gait metrics*) from [2, 28] to evaluate our GRF estimations. Let g_x, g_y and g_z be the components of the GRFs of a step (after a 50 Hz low-pass filter and normalized by body weight, with unit denoted as BW); the *start time* T_s (in seconds) is the time when the vertical GRF reaches 50 N, i.e., $T_s = \min\{t \mid BW \cdot g_z(t) > 50\}$, while the *end time* T_e is the time when the vertical GRF drops below 50 N, i.e., $T_e = \min\{t > T_s \mid BW \cdot g_z(t) < 50\}$.

- *Loading Rate* ($BW \cdot s^{-1}$): Average slope of the vertical GRF during the first 25 ms of the stance after reaching the 50 N threshold [139], i.e.,

$$\text{Loading Rate} = \frac{g_z(T_s + 0.025) - g_z(T_s)}{0.025}.$$

- *Contact Time* (s): Time during which the vertical GRF is above 50 N, i.e., $T_c = T_e - T_s$.
- *Braking Time* (s): Time during which the vertical GRF signal is above the threshold and the A/P GRF component is negative, i.e.,

$$T_b = \int_{T_s}^{T_e} \mathbf{1}_{\{g_y(t) < 0\}} dt.$$

- *Braking Percentage*: Percentage of contact time spent in braking, i.e., T_b/T_c .
- *Active Peak* (BW): Maximum vertical GRF between 30-100% of the stance (to exclude the impact peak), i.e., $\max\{g_z(t) \mid t > T_s + 0.3T_c\}$.
- *Average Vertical Force* (BW): Average value of the vertical GRF, i.e., $\frac{1}{T_c} \int_{T_s}^{T_e} g_z(t) dt$.
- *Net Vertical Impulse* ($BW \cdot s$): Area under the body-weight-normalized vertical GRF reduced by body weight over contact time, i.e., $\int_{T_s}^{T_e} (g_z(t) - 1) dt = \int_{T_s}^{T_e} g_z(t) dt - T_c$.

- *A/P Velocity Change* ($m \cdot s^{-1}$): Change in velocity along the A/P force direction, i.e., $9.81 \cdot (A/P \text{ Impulse})$, where the *A/P Impulse* ($BW \cdot s$) is the area between the A/P GRF component and the zero line, i.e., $\int_{T_s}^{T_e} g_y(t) dt$.

A.3 RMSE and Relative RMSE of Anterior/Posterior GRF

This appendix reports the estimation errors for anterior-posterior GRF (y direction) in Tables A.1 and A.2, while estimation errors for the vertical GRF (z direction) are reported in Tables 3.3 and 3.4 of the main text. The GRFs along both vertical and anterior-posterior directions are used to compute biomechanical variables in Table 3.7.

Appendix B

FedSER Algorithm Pseudocode

The federated SVD procedure used by FedSER is stated below. The privacy protocol appears in Section 4.4.4, and the privacy–utility experiments in Section 4.5.5.

B.1 Federated SVD Algorithm

Algorithm 1 Federated Block-Wise SVD via Gradient Descent

Require: Clients $\{\mathbf{M}_c\}_{c=1}^C$; target rank k ; initial block size b_0 ; maximum inner iterations $T_{\max}$; stepsize $\eta > 0$; momentum coefficient $\mu \in [0, 1]$; tolerance $\delta > 0$.

Ensure: Orthonormal basis $\mathbf{V} \in \mathbb{R}^{d \times k}$; singular-value vector $\mathbf{s} \in \mathbb{R}^k$, with $\mathbf{\Sigma} = \text{diag}(\mathbf{s})$.

```

1: procedure SERVERAGGREGATION
2:    $\mathbf{V} \leftarrow []$ ,  $\ell \leftarrow 0$ ,  $b \leftarrow b_0$ 
3:   while  $\ell < k$  do
4:      $b_{\text{eff}} \leftarrow \min(b, k - \ell)$ 
5:      $(\mathbf{X})_{ij} \sim \mathcal{N}(0, 1)$ ,  $\mathbf{X} \in \mathbb{R}^{d \times b_{\text{eff}}}$ ;  $\mathbf{X} \leftarrow \text{orth}(\mathbf{X})$  ▷ Random orthonormal start
6:     if  $\mathbf{V} \neq []$  then
7:        $\mathbf{X} \leftarrow \text{orth}(\mathbf{X} - \mathbf{V}(\mathbf{V}^\top \mathbf{X}))$  ▷ Project out accepted directions
8:     end if
9:      $\mathbf{X}_{\text{prev}} \leftarrow \mathbf{X}$ 
10:    for  $t = 1, \dots, T_{\max}$  do
11:       $\mathbf{Y} \leftarrow \mathbf{X} + \mu(\mathbf{X} - \mathbf{X}_{\text{prev}})$  ▷ Nesterov-style look-ahead
12:      Broadcast  $\mathbf{Y}$  to all clients
13:      for each client  $c$  in parallel do
14:         $\mathbf{U}_c \leftarrow \text{CLIENTUPDATE}(\mathbf{M}_c, \mathbf{Y})$  ▷  $d \times b_{\text{eff}}$ 
15:      end for
16:       $\mathbf{U} \leftarrow \sum_{c=1}^C \mathbf{U}_c$ 
17:       $\mathbf{X}_{\text{new}} \leftarrow (1 - \eta)\mathbf{Y} + \eta\mathbf{U}$ 
18:      if  $\mathbf{V} \neq []$  then
19:         $\mathbf{X}_{\text{new}} \leftarrow \mathbf{X}_{\text{new}} - \mathbf{V}(\mathbf{V}^\top \mathbf{X}_{\text{new}})$  ▷ Projection deflation
20:      end if
21:       $\mathbf{X}_{\text{new}} \leftarrow \text{orth}(\mathbf{X}_{\text{new}})$ 
22:       $\varepsilon_t^u \leftarrow \left\| \frac{\mathbf{X}_{\text{new}}}{\|\mathbf{X}_{\text{new}}\|_F} - \frac{\mathbf{X}}{\|\mathbf{X}\|_F} \right\|_F$  ▷ Normalized iterate change
23:      if  $\varepsilon_t^u < \delta$  then
24:         $\mathbf{X} \leftarrow \mathbf{X}_{\text{new}}$ 
25:        break
26:      end if
27:       $\mathbf{X}_{\text{prev}} \leftarrow \mathbf{X}$ ;  $\mathbf{X} \leftarrow \mathbf{X}_{\text{new}}$ 
28:    end for
29:     $\mathbf{V} \leftarrow [\mathbf{V} \ \mathbf{X}]$  ▷ Append accepted orthonormal block
30:     $\ell \leftarrow \ell + b_{\text{eff}}$ ;  $b \leftarrow \min(2b, k - \ell)$  ▷ Double the block width
31:  end while
32:  Broadcast  $\mathbf{V}$  to all clients
33:  for each client  $c$  in parallel do
34:     $\mathbf{H}_c \leftarrow \mathbf{V}^\top \text{CLIENTUPDATE}(\mathbf{M}_c, \mathbf{V})$  ▷ Rayleigh contribution
35:  end for
36:   $\mathbf{s} \leftarrow \sqrt{\text{diag}\left(\sum_{c=1}^C \mathbf{H}_c\right)}$  ▷ Elementwise square root
37:  return  $\mathbf{V}, \mathbf{s}$ 
38: end procedure

Require: Local data matrix  $\mathbf{M}_c \in \mathbb{R}^{n_c \times d}$ ; query matrix  $\mathbf{Z} \in \mathbb{R}^{d \times q}$ .
39: procedure CLIENTUPDATE( $\mathbf{M}_c, \mathbf{Z}$ )
40:   return  $\mathbf{M}_c^\top (\mathbf{M}_c \mathbf{Z})$  ▷ Apply the local normal operator
41: end procedure

```

Algorithm 1 uses the notation from Section 4.4.2: $\mathbf{A} = \sum_c \mathbf{M}_c^\top \mathbf{M}_c$, δ is the convergence tolerance, and $\mathbf{U}_c$ is the update returned by client c . The $\text{orth}(\cdot)$ operation returns an orthonormal basis for its argument. It turns the Gaussian initialization into a random frame, prevents the columns in a block from collapsing onto the dominant direction, and

restores unit scale after projection deflation.

The algorithm begins with $b_0 = 1$ and doubles the block width after every accepted block. This gives the leading, larger-singular-value directions finer refinement and groups the later, less influential directions into fewer sequential stages. The reported implementation uses the fixed tolerance $\delta = 10^{-8}$. A block-dependent δ_ℓ can instead be relaxed for later directions when fewer rounds are preferred.

Every round uses the single client-side primitive

$$\text{ClientUpdate}(\mathbf{M}_c, \mathbf{Z}) = \mathbf{M}_c^\top (\mathbf{M}_c \mathbf{Z}).$$

It contains two local matrix multiplications, but one client operation type is reused for both block refinement and the final Rayleigh calculation. This reduces client code paths and avoids materializing $\mathbf{M}_c^\top \mathbf{M}_c$. Since the same product can be accumulated over row blocks, it also supports streaming. With sparse storage and sparse kernels, only nonzero entries are visited at a cost of $\mathcal{O}(\text{nnz}(\mathbf{M}_c)q)$. This is a capability of the matrix-free formulation; the reported dense implementation still avoids materializing the normal matrix.

The Nesterov-style look-ahead carries the previous iterate displacement into the next client query and is intended to reduce costly communication rounds; $\mu = 0$ gives the unaccelerated update. The stopping rule checks normalized iterate stability at the server, so it does not require another client operation. It is a basis-dependent criterion rather than a rotation-invariant subspace distance; a projector difference can be substituted without changing the client protocol.

Finally, because $\mathbf{A} = \mathbf{M}^\top \mathbf{M}$ is positive semidefinite, $\mathbf{v}_i^\top \mathbf{A} \mathbf{v}_i \approx \sigma_i^2$ for a unit recovered direction. Therefore,

$$\mathbf{s} = \sqrt{\text{diag}\left(\sum_c \mathbf{H}_c\right)}, \quad \mathbf{\Sigma} = \text{diag}(\mathbf{s}),$$

recovers the singular values from additive client Rayleigh contributions without a separate client operation or local SVD [124].

Appendix C

FedSER Detailed Tables

C.1 Baseline and Modality Results

The detailed baseline table (Table C.1) reports the complete modality-by-case comparison that supports the main FedSER results in Chapter 4. Column labels Personal / Others / Everyone follow the data-scenario triad of Chapter 3; in FedSER experiments, Everyone corresponds to the cohort-wide FEDERATED setting described in Section 4.3.

Table C.1: RMSE of joint moments at hip and knee (N·m/kg) and vertical GRF (BW) across input sensor modalities, data scenarios, and machine learning methods. Results are reported for tasks included in the training set and for tasks excluded during training under the Leave-One-Action-Out (LOAO) protocol. Best (lowest) RMSE per *modality, data case, target* is highlighted; values within 0.01 of the minimum are treated as ties and highlighted as well.

Sensor Modality	Case	Method	Action Within Training			Leave-One-Action-Out		
			Hip	Knee	GRF	Hip	Knee	GRF
IMU	Personal	FedSER	0.1374	0.1095	0.1109	0.2924	0.2264	0.2714
		KNN	0.1385	0.1178	0.1065	0.3209	0.2472	0.2583
		TCN	0.3040	0.2460	0.3340	0.3053	0.2479	0.3471
	Others	FedSER	0.1776	0.1437	0.1261	0.2668	0.2074	0.2672
		KNN	0.1870	0.1625	0.1213	0.3035	0.2518	0.2526
		TCN	0.3019	0.2475	0.3147	0.3167	0.2555	0.3360
	Everyone	FedSER	0.1423	0.1126	0.0975	0.2664	0.2045	0.2643
		KNN	0.1377	0.1125	0.0912	0.3007	0.2432	0.2475
		TCN	0.3005	0.2429	0.3073	0.3127	0.2561	0.3332
IMU & EMG	Personal	FedSER	0.1308	0.1097	0.1111	0.2645	0.2282	0.2621
		KNN	0.1326	0.1178	0.1067	0.3007	0.2462	0.2530
		TCN	0.2966	0.2459	0.3136	0.2988	0.2528	0.3117
	Others	FedSER	0.1706	0.1428	0.1255	0.2489	0.2084	0.2584
		KNN	0.1807	0.1622	0.1213	0.2845	0.2504	0.2450
		TCN	0.2866	0.2397	0.2768	0.2970	0.2553	0.2920
	Everyone	FedSER	0.1347	0.1120	0.0967	0.2466	0.2067	0.2561
		KNN	0.1316	0.1125	0.0907	0.2812	0.2421	0.2402
		TCN	0.2833	0.2272	0.2547	0.2974	0.2580	0.2915
IMU & Motion Capture	Personal	FedSER	0.1082	0.0929	0.0927	0.2353	0.2179	0.2382
		KNN	0.1212	0.1093	0.0970	0.2633	0.2236	0.2228
		TCN	0.2962	0.2444	0.3379	0.2972	0.2476	0.3249
	Others	FedSER	0.1468	0.1263	0.1020	0.2302	0.1883	0.2304
		KNN	0.1681	0.1510	0.1073	0.2467	0.2157	0.2118
		TCN	0.2923	0.2488	0.2980	0.3088	0.2550	0.3023
	Everyone	FedSER	0.1086	0.0922	0.0770	0.2276	0.1861	0.2289
		KNN	0.1190	0.1049	0.0821	0.2406	0.2116	0.2093
		TCN	0.2832	0.2439	0.2829	0.3083	0.2553	0.2983

C.2 Data-Scarcity Results

The data-scarcity appendix separates the numerical tables from the trend figures. Table C.2 reports exact RMSE under **action-within-training** evaluation (the test action appears in training), and Table C.3 reports the corresponding values under **leave-one-action-out (LOAO)** evaluation (the test action is withheld from training for all participants). Each table is placed on its own page. The following landscape figures (Figures C.1 to C.3) show the same trends for hip moment, knee moment, and vertical GRF, with one target per page so the panels remain readable.

Table C.2: **Action-Within-Training evaluation.** RMSE for IMU-only models when the test action is included in training, across data scarcity levels. Hip and knee moments are in N·m/kg and vertical GRF is in BW. FedSER variants are labeled by gating strategies. SER and FedSER (Global) have the same results and are consolidated into one. Best (lowest) RMSE per case, scarcity level, target is highlighted; values within 0.01 of the minimum are treated as ties and highlighted as well.

Case	Method	10%			20%			40%			60%			80%			100%		
		Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF
Personal	SER / FedSER (Global)	0.2262	0.1780	0.1832	0.1884	0.1552	0.1499	0.1744	0.1409	0.1332	0.1658	0.1351	0.1239	0.1589	0.1291	0.1201	0.1650	0.1295	0.1154
	FedSER (Best Expert)	0.2227	0.1865	0.2045	0.1966	0.1660	0.1749	0.1763	0.1414	0.1485	0.1490	0.1315	0.1271	0.1329	0.1184	0.1148	0.1374	0.1095	0.1109
	FedSER (MoE)	0.2196	0.1847	0.2027	0.1950	0.1645	0.1728	0.1739	0.1401	0.1471	0.1477	0.1305	0.1259	0.1321	0.1169	0.1139	0.1366	0.1086	0.1096
	FedSER (MoE+Global)	0.2058	0.1713	0.1812	0.1784	0.1500	0.1512	0.1594	0.1284	0.1282	0.1397	0.1191	0.1120	0.1269	0.1085	0.1036	0.1314	0.1029	0.0982
	KNN	0.2704	0.2036	0.1889	0.2410	0.1860	0.1710	0.2149	0.1648	0.1542	0.1954	0.1473	0.1381	0.1763	0.1366	0.1282	0.1715	0.1311	0.1190
	TCN	0.4036	0.3370	–	0.3600	0.2957	–	0.3337	0.2704	–	0.3201	0.2592	–	0.3104	0.2506	–	0.3040	0.2460	0.3340
Others	SER / FedSER (Global)	0.2044	0.1590	0.1712	0.1942	0.1543	0.1627	0.1919	0.1498	0.1608	0.1841	0.1474	0.1571	0.1829	0.1464	0.1574	0.1896	0.1489	0.1538
	FedSER (Best Expert)	0.2382	0.1921	0.2270	0.2162	0.1740	0.1889	0.1947	0.1570	0.1577	0.1818	0.1460	0.1373	0.1820	0.1437	0.1322	0.1776	0.1437	0.1261
	FedSER (MoE)	0.2273	0.1840	0.2208	0.2066	0.1678	0.1839	0.1901	0.1526	0.1526	0.1756	0.1429	0.1356	0.1770	0.1412	0.1289	0.1751	0.1407	0.1238
	FedSER (MoE+Global)	0.2021	0.1614	0.1854	0.1844	0.1486	0.1593	0.1730	0.1376	0.1383	0.1610	0.1311	0.1262	0.1603	0.1299	0.1219	0.1619	0.1307	0.1173
	KNN	0.2585	0.2010	0.1762	0.2355	0.1951	0.1622	0.2200	0.1836	0.1535	0.2094	0.1782	0.1463	0.2021	0.1779	0.1428	0.2060	0.1778	0.1389
	TCN	0.4109	0.3252	–	0.3662	0.2970	–	0.3325	0.2749	–	0.3174	0.2608	–	0.3070	0.2532	–	0.3019	0.2475	0.3147
Everyone	SER / FedSER (Global)	0.2043	0.1671	0.1687	0.1933	0.1619	0.1582	0.1900	0.1564	0.1570	0.1816	0.1536	0.1529	0.1809	0.1528	0.1538	0.1870	0.1559	0.1495
	FedSER (Best Expert)	0.2273	0.1941	0.2133	0.1894	0.1579	0.1555	0.1660	0.1303	0.1255	0.1465	0.1223	0.1049	0.1412	0.1160	0.1004	0.1423	0.1126	0.0975
	FedSER (MoE)	0.2182	0.1851	0.2054	0.1841	0.1532	0.1509	0.1607	0.1270	0.1223	0.1429	0.1198	0.1033	0.1383	0.1139	0.0987	0.1401	0.1111	0.0954
	FedSER (MoE+Global)	0.1957	0.1652	0.1750	0.1702	0.1426	0.1376	0.1536	0.1239	0.1181	0.1389	0.1187	0.1050	0.1361	0.1147	0.1027	0.1395	0.1137	0.0990
	KNN	0.2406	0.1845	0.1626	0.2078	0.1667	0.1442	0.1805	0.1453	0.1273	0.1623	0.1319	0.1143	0.1500	0.1244	0.1062	0.1507	0.1207	0.0998
	TCN	0.4080	0.3191	–	0.3698	0.2918	–	0.3318	0.2694	–	0.3185	0.2550	–	0.3075	0.2468	–	0.3005	0.2429	0.3073

Table C.3: **Leave-One-Action-Out (LOAO) evaluation.** RMSE for IMU-only models when the test action is withheld from training for all participants, across data scarcity levels. Hip and knee moments are in N·m/kg and vertical GRF is in BW. FedSER variants are labeled by gating strategies. SER and FedSER (Global) have the same results and are consolidated into one. Best (lowest) RMSE per *case*, *scarcity level*, *target* is highlighted; values within 0.01 of the minimum are treated as ties and highlighted as well.

Case	Method	10%			20%			40%			60%			80%			100%		
		Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF	Hip	Knee	GRF
Personal	SER / FedSER (Global)	0.3081	0.2521	0.2808	0.2918	0.2372	0.2628	0.2809	0.2355	0.2520	0.2775	0.2271	0.2493	0.2737	0.2254	0.2370	0.2602	0.2242	0.2420
	FedSER (Best Expert)	0.3045	0.2624	0.2958	0.3110	0.2581	0.2907	0.3084	0.2650	0.2893	0.3139	0.2599	0.2846	0.3205	0.2623	0.2770	0.3079	0.2684	0.2832
	FedSER (MoE)	0.3005	0.2599	0.2940	0.3076	0.2556	0.2888	0.3064	0.2625	0.2877	0.3102	0.2569	0.2829	0.3170	0.2596	0.2742	0.3040	0.2656	0.2811
	FedSER (MoE+Global)	0.2843	0.2440	0.2734	0.2811	0.2346	0.2612	0.2752	0.2367	0.2542	0.2759	0.2303	0.2503	0.2777	0.2304	0.2398	0.2660	0.2329	0.2450
	KNN	0.3313	0.2526	0.2393	0.3239	0.2497	0.2396	0.3207	0.2543	0.2337	0.3177	0.2518	0.2333	0.3144	0.2505	0.2236	0.3066	0.2501	0.2297
	TCN	0.4029	0.3380	–	0.3596	0.2980	–	0.3348	0.2761	–	0.3207	0.2613	–	0.3120	0.2532	–	0.3053	0.2479	0.3471
Others	SER / FedSER (Global)	0.2740	0.2247	0.2465	0.2713	0.2194	0.2486	0.2679	0.2171	0.2440	0.2650	0.2138	0.2415	0.2641	0.2100	0.2313	0.2519	0.2096	0.2387
	FedSER (Best Expert)	0.3300	0.2751	0.2940	0.3294	0.2763	0.2919	0.3397	0.2792	0.2882	0.3336	0.2728	0.2825	0.3396	0.2674	0.2674	0.3314	0.2661	0.2759
	FedSER (MoE)	0.3118	0.2614	0.2855	0.3165	0.2659	0.2843	0.3251	0.2676	0.2795	0.3199	0.2630	0.2749	0.3263	0.2575	0.2592	0.3182	0.2577	0.2685
	FedSER (MoE+Global)	0.2786	0.2333	0.2513	0.2787	0.2348	0.2516	0.2826	0.2348	0.2467	0.2785	0.2308	0.2430	0.2797	0.2256	0.2296	0.2720	0.2257	0.2377
	KNN	0.3108	0.2551	0.2270	0.3062	0.2573	0.2299	0.3063	0.2578	0.2286	0.3046	0.2574	0.2264	0.2967	0.2548	0.2171	0.2934	0.2558	0.2243
	TCN	0.4042	0.3135	–	0.3713	0.2891	–	0.3381	0.2725	–	0.3245	0.2644	–	0.3187	0.2584	–	0.3167	0.2555	0.3360
Everyone	SER / FedSER (Global)	0.2723	0.2229	0.2439	0.2699	0.2189	0.2455	0.2657	0.2166	0.2419	0.2635	0.2113	0.2391	0.2622	0.2093	0.2298	0.2506	0.2088	0.2358
	FedSER (Best Expert)	0.3339	0.2782	0.2934	0.3329	0.2756	0.2881	0.3361	0.2754	0.2805	0.3350	0.2667	0.2779	0.3303	0.2634	0.2689	0.3141	0.2597	0.2716
	FedSER (MoE)	0.3159	0.2618	0.2839	0.3161	0.2632	0.2794	0.3212	0.2623	0.2710	0.3210	0.2561	0.2701	0.3175	0.2543	0.2601	0.3015	0.2517	0.2636
	FedSER (MoE+Global)	0.2798	0.2324	0.2495	0.2777	0.2328	0.2478	0.2789	0.2309	0.2407	0.2780	0.2257	0.2395	0.2750	0.2231	0.2299	0.2628	0.2221	0.2347
	KNN	0.3095	0.2516	0.2244	0.3040	0.2535	0.2275	0.3027	0.2529	0.2248	0.3004	0.2521	0.2230	0.2928	0.2471	0.2132	0.2889	0.2490	0.2206
	TCN	0.3999	0.3152	–	0.3690	0.2906	–	0.3333	0.2730	–	0.3211	0.2640	–	0.3162	0.2592	–	0.3127	0.2561	0.3332

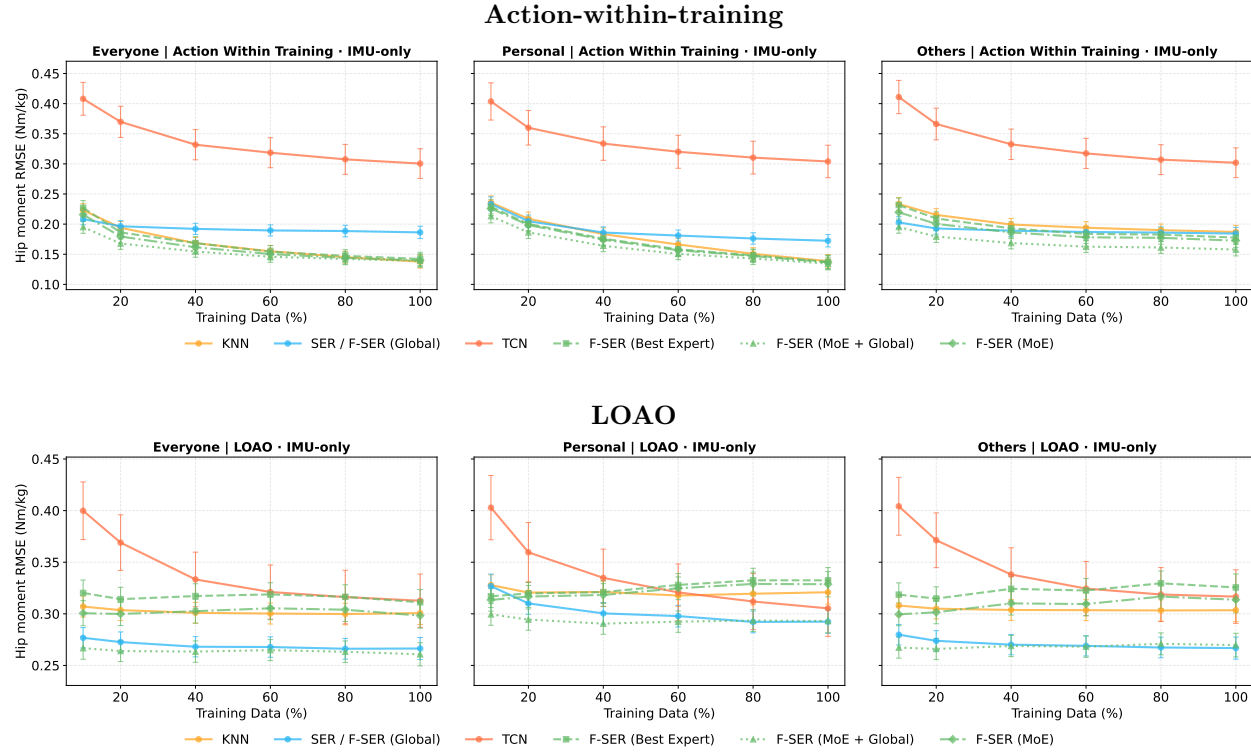


Figure C.1: RMSE for multi-task estimation of **hip flexion moment** from IMU-only inputs, versus the percentage of data used in training. Top: **action-within-training**; bottom: **LOAO**. Each row compares SER, KNN, TCN, and FedSER with different gating strategies across **Personal**, **Others**, and **Everyone** data cases.

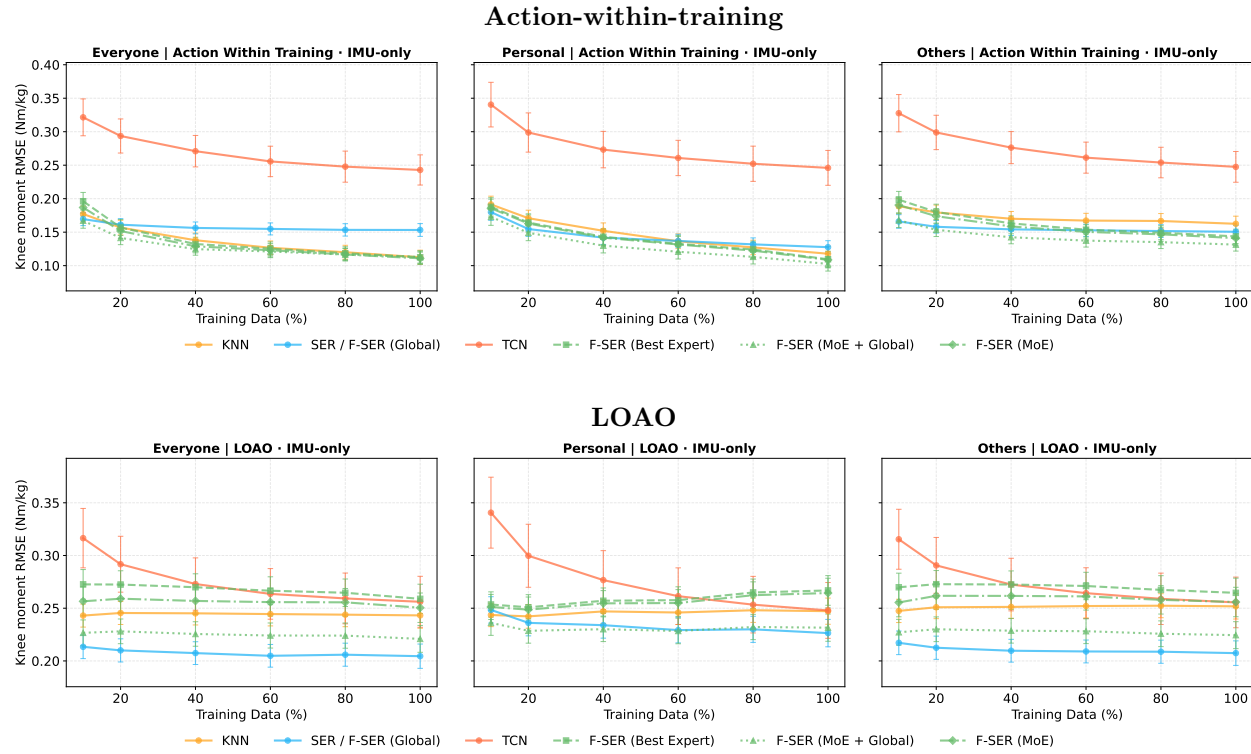


Figure C.2: RMSE for multi-task estimation of **knee moment** from IMU-only inputs, versus the percentage of data used in training. Top: **action-within-training**; bottom: **LOAO**. Panel layout matches Fig. C.1.

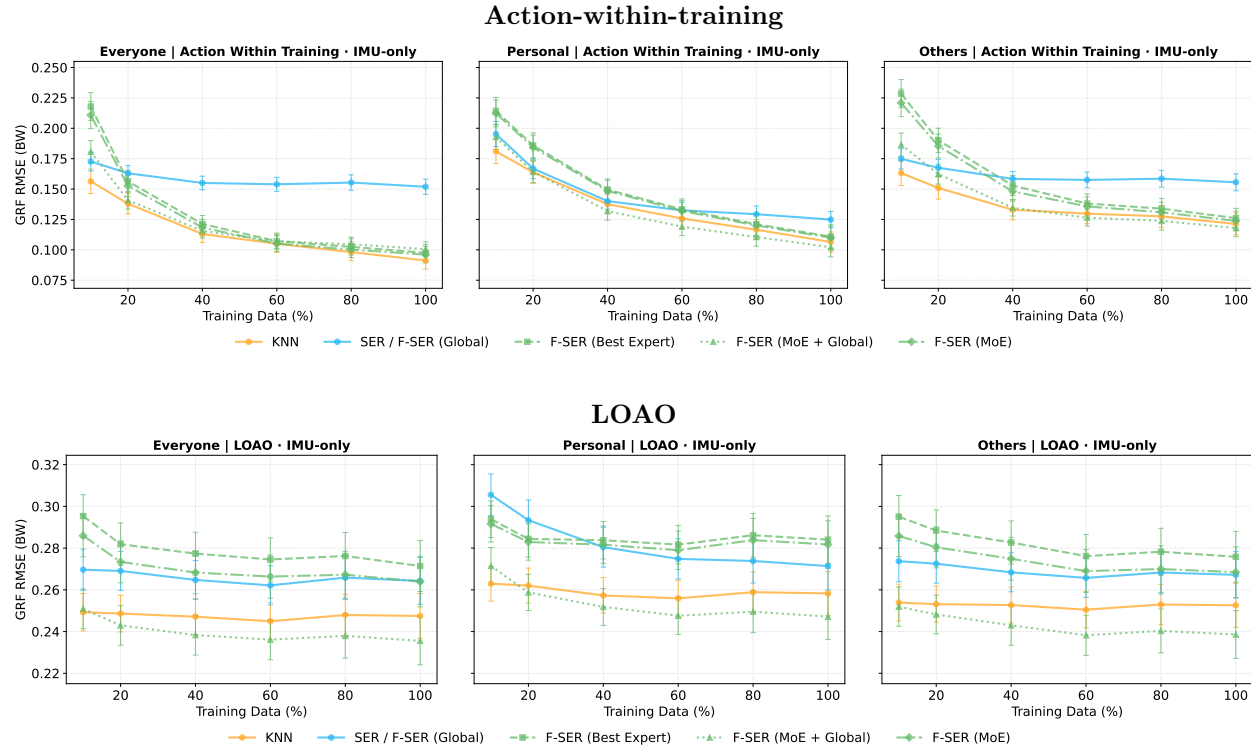


Figure C.3: RMSE for multi-task estimation of **vertical GRF** from IMU-only inputs, versus the percentage of data used in training. Top: **action-within-training**; bottom: **LOAO**. Panel layout matches Fig. C.1.

Appendix D

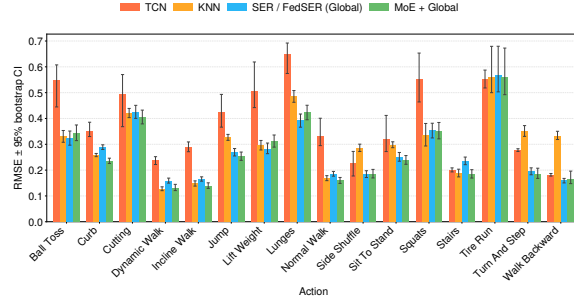
FedSER Confidence Intervals and Supplementary Figures

D.1 Confidence Interval Methodology

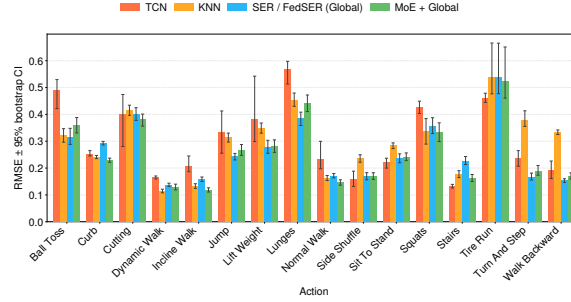
In Chapter 4, we report point estimates of RMSE for clarity and retain 95% confidence intervals (CIs) in the representative RMSE-improvement plot (Fig. 4.6) and in the additional analyses below. This section describes how those CIs are constructed.

The average RMSE is calculated from per-action mean RMSE (each per-action mean averages over subjects, experimental runs, and speed/intensity variants within an action), then averaged across the 16 actions. For improvement plots, the average improvement is calculated from the difference of per-action mean RMSE between each baseline and the MoE+Global model, then averaged across the 16 actions. The CI is a bootstrap interval over these per-action improvements [131]. We consider action-wise RMSE differences to preserve the action-level comparison between methods. The bootstrap CI is computed by resampling the 16 action-wise differences between methods and estimating the sampling distribution of their mean, without assuming that the differences are Gaussian [132, Chapter 12]. Plotted error bars denote that 95% confidence interval. Note that average RMSE shows these relationships, and the CIs confirm that they typically hold across experimental runs; overlapping CIs do not by themselves imply that two methods are statistically indistinguishable [131]. These intervals quantify variation across actions after subjects, experimental runs, and speed or intensity variants have been averaged within each action. They do not quantify run-to-run stability.

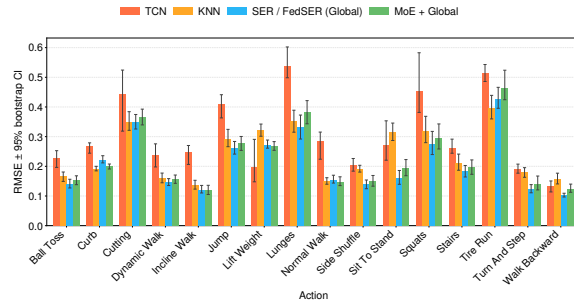
D.2 Per-Action RMSE under LOAO



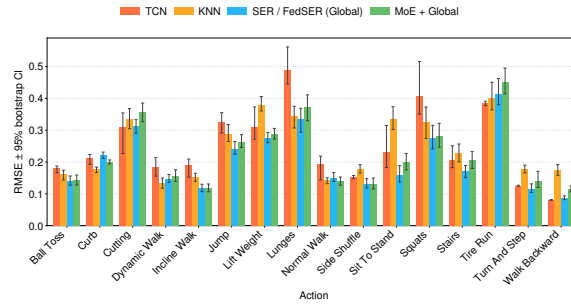
(a) Hip moment, 10% training



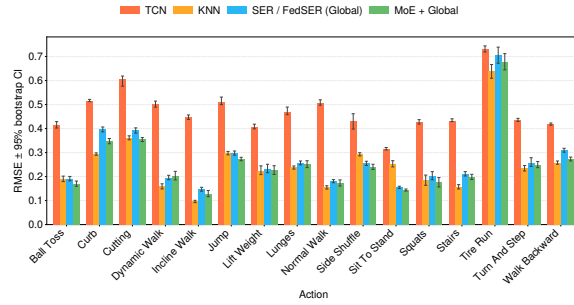
(b) Hip moment, 100% training



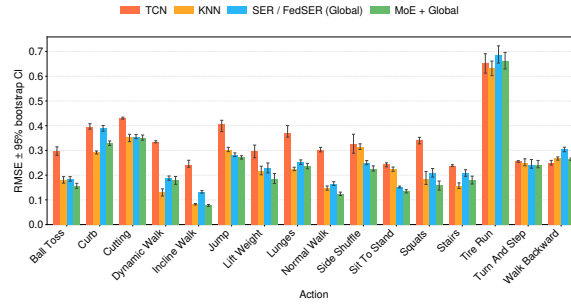
(c) Knee moment, 10% training



(d) Knee moment, 100% training



(e) Vertical GRF, 10% training



(f) Vertical GRF, 100% training

Figure D.1: Per-action RMSE under IMU-only inputs, LOAO, and the FEDERATED setting, for hip moment, knee moment, and vertical GRF at 10% and 100% training data. Each bar is the per-action mean RMSE (averaging over subjects, experimental runs, and speed/intensity variants within an action); error bars show 95% confidence intervals (Section D.1).

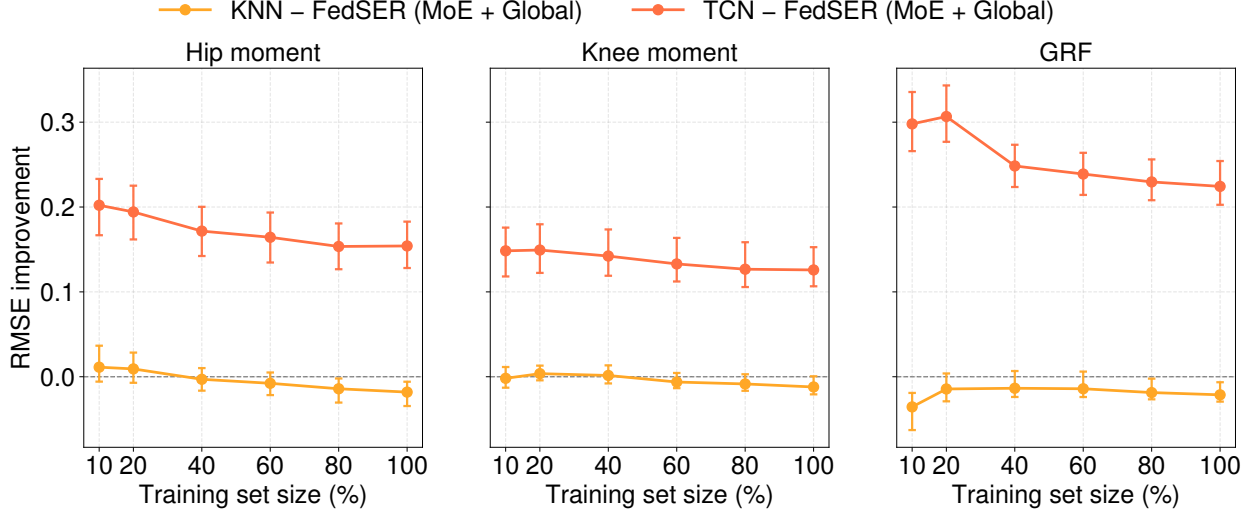


Figure D.2: RMSE improvement of FedSER (MoE+Global) over KNN and TCN across training-set sizes under action-within-training evaluation with IMU-only inputs and federated training. Positive values indicate lower RMSE for FedSER; error bars show 95% action-bootstrap confidence intervals (Section D.1).

Figure D.1 reports per-action RMSE for TCN, KNN, a global FedSER expert, and FedSER (MoE+Global). FedSER (MoE+Global) frequently has lower RMSE than KNN and TCN on hip and knee moments, whereas KNN remains competitive on GRF. The action-level variation also explains why conclusions based only on averages depend on the dataset’s action distribution.

D.3 Action-Within-Training RMSE Improvement

Chapter 4 reports the LOAO result; Fig. D.2 provides the action-within-training counterpart without repeating the LOAO figure. FedSER (MoE+Global) remains better than TCN, while the improvement over KNN is smaller and target-dependent.

D.4 RMSE Improvement under Differential Privacy

Figure 4.9 reports hip-moment RMSE versus the privacy budget ϵ . Figure D.3 reports the corresponding average RMSE improvement of FedSER (MoE+Global) over KNN and TCN

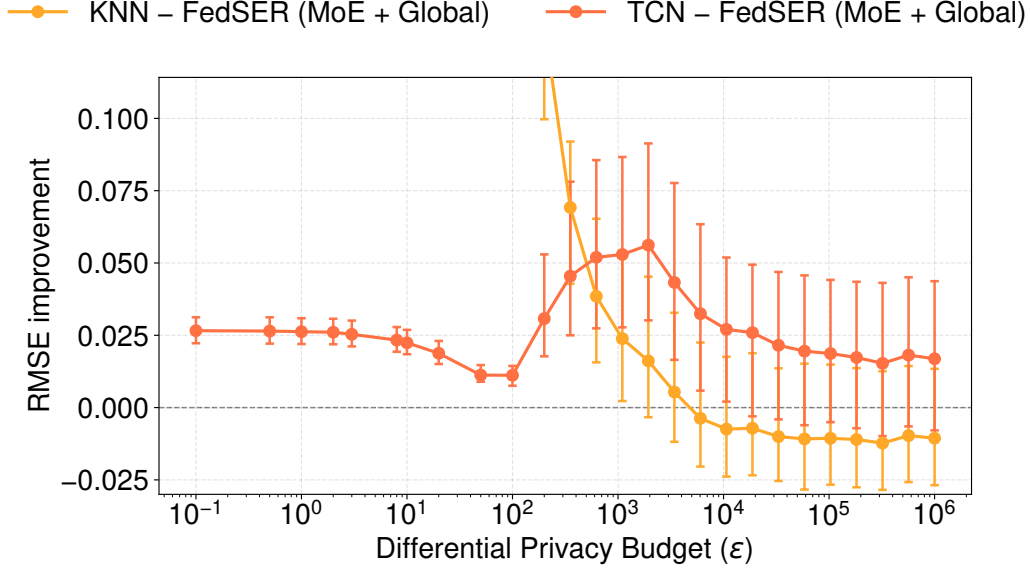


Figure D.3: RMSE improvement of FedSER (MoE+Global) over KNN and TCN versus the privacy budget ϵ for hip moment (IMU-only, FEDERATED, LOAO, full training data). Positive values indicate lower RMSE for FedSER; error bars show 95% confidence intervals (Section D.1).

under the same IMU-only, FEDERATED, LOAO setting with full training data, with 95% CIs constructed as in Section D.1. Positive values indicate lower RMSE for FedSER. KNN is the most sensitive to noise: at smaller ϵ values (stronger privacy guarantees), the improvement of FedSER over KNN grows beyond the plotted range, consistent with KNN’s divergence in the main privacy figure. As ϵ increases, all methods approach their non-private accuracy, and the improvement over TCN remains positive across budgets. Across budgets, FedSER with MoE+Global has the lowest RMSE, matching its advantage in the non-private setting. The RMSE increase at small ϵ is expected: for strong privacy $\epsilon \leq 0.5$ [64], prior work reports comparable loss unless the data is far larger, on the order of 10^3 clients [136] or with much more training data ($\sim 10\times$) [76], so it reflects the scale of the 11-subject dataset rather than a limitation of FedSER. The TCN curve uses the same mid-range smoothing of the TCN RMSE series as in the main privacy panel.